

grok-wsl-ubuntu / 019f4231-6d46-7182-b685-10610acc45c6

Metadata

```
- Source: `grok-wsl-ubuntu`
- Kind: `grok`
- Source file: `\\wsl.localhost\Ubuntu\home\avidullu\grok\sessions\%2Fhome%2Favidullu%2Fprojects%2Fagent%2Fagent-sessions\019f4231-6d46-7182-b685-10610acc45c6\chat_history.jsonl`
- SHA-256: `1495ec293b3a7919379e217899634c808fb9688b3d733e2d38110f15b6ca9cfc`
- Source modified: `2026-07-08T14:52:58+00:00`
- Imported at: `2026-07-09T08:32:29+00:00`
- project: `%2Fhome%2Favidullu%2Fprojects%2Fagent%2Fagent-sessions`
- session_id: `019f4231-6d46-7182-b685-10610acc45c6`
```

Transcript

1. system

You are Grok released by xAI. You are an interactive CLI tool that helps users with software engineering tasks. Your main goal is to complete the user's request, denoted within the `<user_query>` tag.

`<executing_actions_with_care>`

Carefully consider the reversibility and blast radius of actions. Generally you can freely take local, reversible actions like editing files or running tests. But for actions that are hard to reverse, affect shared systems beyond your local environment, or could otherwise be risky or destructive, check with the user before proceeding. The cost of pausing to confirm is low, while the cost of an unwanted action (lost work, unintended messages sent, deleted branches) can be very high. For actions like these, consider the context, the action, and user instructions, and by default transparently communicate the action and ask for confirmation before proceeding. This default can be changed by user instructions - if explicitly asked to operate more autonomously, then you may proceed without confirmation, but still attend to the risks and consequences when taking actions. A user approving an action (like a git push) once does NOT mean that they approve it in all contexts, so unless actions are authorized in advance in durable instructions like AGENTS.md files, always confirm first. Authorization stands for the scope specified, not beyond. Match the scope of your actions to what was actually requested.

Examples of the kind of risky actions that warrant user confirmation:

- Destructive operations: deleting files/branches, dropping database tables, killing processes, `rm -rf`, overwriting uncommitted changes
- Hard-to-reverse operations: force-pushing (can also overwrite upstream), `git reset --hard`, amending published commits, removing or downgrading packages/dependencies, modifying CI/CD pipelines
- Actions visible to others or that affect shared state: pushing code, creating/closing/commenting on PRs or issues, sending messages (Slack, email, GitHub), posting to external services, modifying shared infrastructure or permissions
- Uploading content to third-party web tools (diagram renderers, pastebins, gists) publishes it - consider whether it could be sensitive before sending, since it may be cached or indexed even if later deleted.

When you encounter an obstacle, do not use destructive actions as a shortcut to simply make it go away. For instance, try to identify root causes and fix underlying issues rather than bypassing safety checks (e.g. `--no-verify`). If you discover unexpected state like unfamiliar files, branches, or configuration, investigate before deleting or overwriting, as it may represent the user's in-progress work. For example, typically resolve merge conflicts rather than discarding changes; similarly, if a lock file exists, investigate what process holds it rather than deleting it. In short: only take risky actions carefully, and when in doubt, ask before acting. Follow both the spirit and letter of these instructions - measure twice, cut once.

`</executing_actions_with_care>`

`<tool_calling>`

- Use specialized tools instead of bash commands when possible, as this provides a better user

experience. For file operations, prefer dedicated file tools (e.g., `read_file` for reading files instead of `cat/head/tail`, `search_replace` for editing and creating files instead of `sed/awk`). Reserve bash tools exclusively for actual system commands and terminal operations that require shell execution. NEVER use bash `echo` or other command-line tools to communicate thoughts, explanations, or instructions to the user. Output all communication directly in your response text instead.

</tool_calling>

<background_tasks>

For watch processes, polling, and ongoing observation (CI status, log tailing, API polling): Use the `monitor` tool ? it streams each stdout line back as a chat notification.

</background_tasks>

<output_efficiency>

Keep your text output brief and direct. Lead with the answer or action, not the reasoning. Skip filler words, preamble, and unnecessary transitions. Do not restate what the user said ? just do it. When explaining, include only what is necessary for the user to understand.

Focus text output on:

- Decisions that need the user's input
- High-level status updates at natural milestones
- Errors or blockers that change the plan

Prefer short, direct sentences over long explanations. This does NOT apply to code or tool calls.

</output_efficiency>

<formatting>

Your text output is rendered as GitHub-flavored markdown (CommonMark). Use markdown actively when it aids the reader: bullet lists for parallel items, **bold** for emphasis, `inline code` for identifiers/paths/commands, and tables for short enumerable facts (file/line/status, before/after, quantitative data). Don't pack explanatory reasoning into table cells ? explain before or after the table. Match structure to the task: a simple question gets a direct answer in prose, not headers and numbered sections.

</formatting>

<user_guide>

Documentation about the Grok Build TUI ? including configuration, keyboard shortcuts, MCP servers, skills, theming, plugins, and more ? is stored as `.md` files in `~/ .grok/docs/user-guide/`. When users ask about features or how to use the TUI, read the relevant file from that directory. Present the information directly.

</user_guide>

2. user

<user_info>

OS Version: linux

Shell: /bin/bash

Workspace Path: /home/avidullu/projects/Agent/agent-sessions

Today's date: 2026-07-08

Note: Prefer using relative paths over absolute paths as tool call args when possible.

</user_info>

<git_status>

This is the git status at the start of the conversation. Note that this status is a snapshot in time, and will not update during the conversation.

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

</git_status>

3. user

<system-reminder>

As you answer the user's questions, you can use the following context (ordered from repo root to current directory - deeper files take precedence on conflicts):

From: /home/avidullu/projects/Agent/agent-sessions/AGENTS.md

Agent Working Agreement

Default delivery workflow: PR -> review -> resolve -> merge

For normal project work, operate in small, reviewable pull requests:

1. Start from the remote base branch, not the current local branch.
2. Implement one coherent tracker row, bug fix, or reviewable slice per PR.
3. Push the branch and open a PR for review.
4. Wait for review comments or an explicit LGTM before merging.
5. Address review comments in the same PR, rerun relevant checks, and wait for LGTM again.
6. Merge only after the PR has LGTM or explicit owner approval scoped to that PR.
7. After merge, refresh from the remote base branch before starting the next slice.

Do not batch a whole project into one implementation PR unless the owner explicitly asks for it. Do not continue building later tracker rows on top of an unreviewed PR as though it is already merged.

Exception: broad exploration branches

A temporary broad branch is acceptable only when the work is time-critical, convoluted, or hairy enough that the boundaries are not yet knowable. Treat that branch as exploration, not the deliverable.

Before shipping, split the result back into reviewable PRs whenever practical. If splitting is not practical, say why in the PR description and call out the larger review burden clearly.

Review response discipline

When a PR receives comments, resolve them before merge:

- inspect every comment and requested change;
- make the smallest fix that addresses the concern;
- reply or summarize what changed;
- rerun the relevant checks;
- wait for LGTM before merging.

If a tool cannot formally approve because the PR author and reviewer are the same GitHub account, require an explicit owner LGTM comment or instruction before merge.

Follow these instructions exactly. When working in subdirectories not listed above, check for additional project instruction files (AGENTS.md, Claude.md, etc.).

</system-reminder>

4. user

<system-reminder>

The following skills are available for use:

- check-work: Check your work with a verification subagent that reviews diffs, runs builds and tests, and evaluates correctness. Read this file for instructions
Use when: asked to "check work", "verify changes", "self-verify", "/check-work", "/check", "/verify", or "/self-verify".
Absolute path: /home/avidullu/.grok/skills/check-work/SKILL.md
- pptx: Use this skill any time a .pptx file is involved in any way ? as input, output, or both.

This includes: creating slide decks, pitch decks, or presentations; reading, parsing, or extracting text from any .pptx file (even if the extracted content will be used elsewhere, like in an email or summary); editing, modifying, or updating existing presentations; combining or splitting slide files; work?

Absolute path: /home/avidullu/.grok/skills/pptx/SKILL.md

- create-skill: Interactively create a new Grok skill (SKILL.md + optional scripts/references)

Use when: the user wants to create a skill, scaffold a skill, or runs /create-skill.

Absolute path: /home/avidullu/.grok/skills/create-skill/SKILL.md

- xlsx: Use this skill any time a spreadsheet file is the primary input or output. This means any task where the user wants to: open, read, edit, or fix an existing .xlsx, .xlsm, .csv, or .tsv file (e.g., adding columns, computing formulas, formatting, charting, cleaning messy data); create a new spreadsheet from scratch or from other data sources; or convert between tabular file formats. Trigger espec?

Absolute path: /home/avidullu/.grok/skills/xlsx/SKILL.md

- help: Grok documentation and configuration help

Use when: users ask about setup, configuration, MCP servers, authentication, skills, slash commands, keyboard shortcuts, or any Grok feature. Also use proactively when you detect a user is having trouble with setup or onboarding.

Absolute path: /home/avidullu/.grok/skills/help/SKILL.md

- imagine: How to use the image_gen and image_edit tool calls in Grok Build: when to build a visual with code instead of generating it, prompt-craft, reference-first handling of real people, factual grounding, and asset-consistency. Load this whenever generating or editing an image is on the table, i.e. when an image_gen or image_edit call is being considered or about to be made. Tool-usage-driven, not tr?

Absolute path: /home/avidullu/.grok/skills/imagine/SKILL.md

- docx: Use this skill whenever the user wants to create, read, edit, or manipulate Word documents (.docx files). Triggers include: any mention of 'Word doc', 'word document', '.docx', or requests to produce professional documents with formatting like tables of contents, headings, page numbers, or letterheads. Also use when extracting or reorganizing content from .docx files, inserting or replacing ima?

Absolute path: /home/avidullu/.grok/skills/docx/SKILL.md

- review: Run a reviewer subagent against uncommitted local changes, a named branch, or a GitHub PR. Local and branch modes write a review file plus a summary to disk. PR mode posts the findings as a PENDING GitHub review for the user to inspect and submit through the UI.

Use when: asked to 'review', 'code review', 'review my changes', 'review this PR', or '/review'.

Absolute path: /home/avidullu/.grok/bundled/skills/review/SKILL.md

- pr-babysit: Monitor PRs, fix CI failures, address review comments, resolve merge conflicts, and restack stacks. Supports independent PRs, Graphite stacks, and GitHub stacked PRs (gh-stack).

Use when: "/pr-babysit".

Absolute path: /home/avidullu/.grok/bundled/skills/pr-babysit/SKILL.md

- design: Run the full design-doc-writer and design-doc-reviewer loop until consensus. Produces a polished design document with a PR plan.

Use when: asked to "design", "write a design doc", "system design", "architecture doc", "technical spec", or "/design".

Absolute path: /home/avidullu/.grok/bundled/skills/design/SKILL.md

- execute-plan: Execute a PR Plan DAG from a design document. Parses the plan, topologically sorts it, implements PRs in parallel using worktree-isolated subagents, runs mandatory orchestrator-level review, and assembles either a Graphite PR stack or a plain-git branch stack depending on tool availability.

Use when: asked to "execute plan", "run the plan", "implement the design", or "/execute-plan".

Absolute path: /home/avidullu/.grok/bundled/skills/execute-plan/SKILL.md

</system-reminder>

5. user

<system-reminder>

MCP server connected:

- grok_com_github (91 tools): The GitHub MCP Server provides tools to interact with GitHub platform. Tool selection guidance: 1. Use 'list_*' tools for broad, simple retrieval and pagination of all items of a type (e.g., all issues, all PRs, all branches) with basic filtering. 2. Use 'search_*' tools for targeted queries with specific criteria, keywords, or complex filters (e.g., issues with certain text, PRs by author, code containing functions). Context management: 1. Use pagination whenever possible with batches of 5-10 items. 2. Use

minimal_output parameter set to true if the full information is not needed to accomplish a task. Tool usage guidance: 1. For 'search_*' tools: Use separate 'sort' and 'order' parameters if available for sorting results - do not include 'sort:' syntax in query strings. Query strings should contain only search criteria (e.g., 'org:google language:python'), not sorting instructions. Always call 'get_me' first to understand current user permissions and context. ## Discussions Use 'list_discussion_categories' to understand available categories before creating discussions. Filter by category for better organization. ## Issues Check 'list_issue_types' first for organizations to use proper issue types. Use 'search_issues' before creating new issues to avoid duplicates. Always set 'state_reason' when closing issues. ## Projects Workflow: 1) list_project_fields (get field IDs), 2) list_project_items (with pagination), 3) optional updates. Project lifecycle: Use create_project to create a new ProjectsV2 for a user or organization (requires owner_type and title). Returns the new project's id, number, title, and url; pass the returned number as project_number to subsequent project tools. Iteration fields: Use create_iteration_field to add a new ITERATION field (e.g. "Sprint") to an existing project. Required: field_name, iteration_duration (days), start_date (YYYY-MM-DD). Only pass the iterations array when iterations need varying durations, breaks between them, or specific titles; otherwise omit it and GitHub? [truncated]

To use MCP tools, you MUST call `search\_tool` first to retrieve the tool's input schema before calling `use\_tool`. NEVER guess parameter names ? always use the exact schema returned by `search\_tool`.

</system-reminder>

6. user

<user_query>

Hi. Can you do a full read of this repo and give an overview of the overarching idea, any feedbacks, comparison to other similar options and a general idea of the engineering and the owners vision.

</user_query>

7. tool_result

```
- /home/avidullu/projects/Agent/agent-sessions/
- agent_sessions/
  - __init__.py
  - archive.py
  - archive_status.py
  - baseline.py
  - baseline_agent.py
  - baseline_calibration.py
  - baseline_eval.py
  - baseline_handoffs.py
  - baseline_ingest.py
  - baseline_lint.py
  - baseline_predictions.py
  - baseline_promote.py
  - baseline_publish.py
  - baseline_redaction.py
  - baseline_replay.py
  - baseline_replay_ingest.py
  - baseline_report.py
  - baseline_settings.py
  - baseline_types.py
  - cli.py
  - config.py
  - models.py
  - path_templates.py
  - render.py
- sources/
  - __init__.py
  - claude.py
  - codex.py
  - deepseek.py
  - gemini.py
```

- grok.py
- registry.py
- utils.py
- AGENTS.md
- archive/
 - claude-windows/
[4444 files in subtree: 2222 *.md, 2222 *.pdf]
 - claude-wsl-ubuntu/
 - 20260606-8af642ca-0e91-47d3-99cf-be49dbaeda69-8af642ca-0e91-47d3-99cf-be49dbaeda69-58492bf.md
 - 20260606-8af642ca-0e91-47d3-99cf-be49dbaeda69-8af642ca-0e91-47d3-99cf-be49dbaeda69-58492bf.pdf

20260623-c287729c-2d4d-499f-b168-ce1fceb891f-c287729c-2d4d-499f-b168-ce1fceb891f-3712522.md

20260623-c287729c-2d4d-499f-b168-ce1fceb891f-c287729c-2d4d-499f-b168-ce1fceb891f-3712522.pdf

- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a0dbe813153f9151d-2a6574464f34.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a0dbe813153f9151d-2a6574464f34.pdf
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a1158fbc4f69f0976-b54caf1054f9.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a1158fbc4f69f0976-b54caf1054f9.pdf
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a14d9ffbcbcd53d0e5-5c6f0c4a4046.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a14d9ffbcbcd53d0e5-5c6f0c4a4046.pdf
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a202351c55a0ceb23-53b353a8f617.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a202351c55a0ceb23-53b353a8f617.pdf
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a2425ecc6dc062cbe-1807344aab0d.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a2425ecc6dc062cbe-1807344aab0d.pdf
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a354d35dc4b2c2b42-de8ab4b5fee1.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a354d35dc4b2c2b42-de8ab4b5fee1.pdf
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a405eae8dbec702b-4031eb7440c9.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a405eae8dbec702b-4031eb7440c9.pdf
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a5598ac50dde6368-dce5b4bbcb6e.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a5598ac50dde6368-dce5b4bbcb6e.pdf
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a6164396ee6166ae2-c56867f57498.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a6164396ee6166ae2-c56867f57498.pdf
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a7d7084767b9679cf-ce8d079aa4a8.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a7d7084767b9679cf-ce8d079aa4a8.pdf
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a8beca64e2ef9d8aa-2fc3dfca27dd.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a8beca64e2ef9d8aa-2fc3dfca27dd.pdf
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a8c625abef0cba8dc-5df73ffff18db.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a8c625abef0cba8dc-5df73ffff18db.pdf
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-adb6a4b6311cd61d7-e254a94f5ef2.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-adb6a4b6311cd61d7-e254a94f5ef2.pdf

20260629-f7f9e275-96e2-4463-a5a3-65bfee712d00-f7f9e275-96e2-4463-a5a3-65bfee712d00-a13da13.md

20260629-f7f9e275-96e2-4463-a5a3-65bfee712d00-f7f9e275-96e2-4463-a5a3-65bfee712d00-a13da13.pdf

- 20260629-journal-journal-5ffea3738007.md
- 20260629-journal-journal-5ffea3738007.pdf
- 20260629-journal-journal-f8743d004df6.md
- 20260629-journal-journal-f8743d004df6.pdf

20260630-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-fafc315.md

20260630-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-fafc315.pdf

- codex-windows/
[104 files in subtree: 52 *.md, 52 *.pdf]
- deepseek-vscode-windows/
[1504 files in subtree: 752 *.md, 752 *.pdf]
- deepseek-vscode-wsl-ubuntu/
[106 files in subtree: 53 *.md, 53 *.pdf]
- gemini-antigravity-windows/
 - 20260603-b83a9b6e-713a-4fa4-abe3-4bf658aab73e-transcript-913e67ffcb39.md
 - 20260603-b83a9b6e-713a-4fa4-abe3-4bf658aab73e-transcript-913e67ffcb39.pdf
 - 20260619-a34a0f82-73e4-484a-8c11-b99c579b48fd-transcript-d3d24f68d3ce.md
 - 20260619-a34a0f82-73e4-484a-8c11-b99c579b48fd-transcript-d3d24f68d3ce.pdf

- 20260619-a34a0f82-73e4-484a-8c11-b99c579b48fd-transcript_full-fd33952b2a57.md
- 20260619-a34a0f82-73e4-484a-8c11-b99c579b48fd-transcript_full-fd33952b2a57.pdf
- grok-wsl-ubuntu/
 - 20260529-019e7238-98ee-7bc1-a871-f9f00f5f8b3b-chat_history-1cfba6887d84.md
 - 20260529-019e7238-98ee-7bc1-a871-f9f00f5f8b3b-chat_history-1cfba6887d84.pdf
 - 20260529-019e7267-eb2a-74c1-ba09-d7e7d19fa29e-chat_history-a5b6eb725e5f.md
 - 20260529-019e7267-eb2a-74c1-ba09-d7e7d19fa29e-chat_history-a5b6eb725e5f.pdf
 - 20260530-019e77e5-9c65-7753-b108-772d1732a8ae-chat_history-27ce75dbe75d.md
 - 20260530-019e77e5-9c65-7753-b108-772d1732a8ae-chat_history-27ce75dbe75d.pdf
 - 20260602-019e892b-1982-75d2-875b-a9f82c413a34-chat_history-9301903fea0c.md
 - 20260602-019e892b-1982-75d2-875b-a9f82c413a34-chat_history-9301903fea0c.pdf
 - 20260602-019e892c-ae5f-7020-8261-ee9e4476991d-chat_history-1c0d052177f2.md
 - 20260602-019e892c-ae5f-7020-8261-ee9e4476991d-chat_history-1c0d052177f2.pdf
 - 20260602-019e8938-91ad-7db3-95dd-9377f69ffbfd-chat_history-b96a6e2dbd34.md
 - 20260602-019e8938-91ad-7db3-95dd-9377f69ffbfd-chat_history-b96a6e2dbd34.pdf
 - 20260602-019e893e-31bf-7b72-ae4a-b50fe5040381-chat_history-db67611dc1ed.md
 - 20260602-019e893e-31bf-7b72-ae4a-b50fe5040381-chat_history-db67611dc1ed.pdf
 - 20260602-019e894e-5540-7b70-a47a-beb676811ea9-chat_history-440ebe6f318a.md
 - 20260602-019e894e-5540-7b70-a47a-beb676811ea9-chat_history-440ebe6f318a.pdf
 - 20260602-019e8969-ba19-7bd0-b5b3-fde7c5445158-chat_history-9722b675cbb5.md
 - 20260602-019e8969-ba19-7bd0-b5b3-fde7c5445158-chat_history-9722b675cbb5.pdf
 - 20260608-019ea0e2-2c13-7d72-b154-54e4f93ea536-chat_history-37a6d8f6914c.md
 - 20260608-019ea0e2-2c13-7d72-b154-54e4f93ea536-chat_history-37a6d8f6914c.pdf
 - 20260608-019ea5ba-d3c8-7723-9207-ca6a4ca2fdd0-chat_history-6b0358ec94b7.md
 - 20260608-019ea5ba-d3c8-7723-9207-ca6a4ca2fdd0-chat_history-6b0358ec94b7.pdf
 - 20260608-019ea5bb-6066-7c91-a074-d721dc1b5b9d-chat_history-8d34dd00289e.md
 - 20260608-019ea5bb-6066-7c91-a074-d721dc1b5b9d-chat_history-8d34dd00289e.pdf
 - 20260614-019ec52b-0ea1-7af2-b1be-6d972bd01936-chat_history-71946cdd51ac.md
 - 20260614-019ec52b-0ea1-7af2-b1be-6d972bd01936-chat_history-71946cdd51ac.pdf
- index.jsonl
- INDEX.md
- baseline/
 - agents/
 - claude/
 - CLAUDE.generated.md
 - README.md
 - codex/
 - AGENTS.generated.md
 - README.md
 - vscode/
 - copilot-instructions.generated.md
 - README.md
 - calibration/
 - efficacy-report.md
 - efficacy.toml
 - feedback.example.toml
 - candidates/
 - 2026-07-05-extraction.md
 - 2026-07-05-extraction.predictions.json
 - README.md
 - evidence/
 - README.md
 - global/
 - engineering-guardrails.md
 - prompt-patterns.md
 - regression-frameworks.md
 - repo-workflows.md
 - handoffs/
 - audit.md
 - index.jsonl
 - metacognition/
 - prediction-ledger.jsonl
 - README.md
 - projects/
 - [6 files in subtree: 6 *.md]

- proposals/
 - [7 files in subtree: 6 *.json, 1 *.md]
- README.md
- replay/
 - manifest.jsonl
- SCHEMA.md
- config/
 - baseline.toml
 - default_sources.toml
- docs/
 - archives/
 - TECH_DEBT_PLAN.md
 - AUTOMATION.md
 - BASELINE_KNOWLEDGE_REPLAY_PLAN.md
 - BASELINE_LOOP_CLOSURE.md
 - BASELINE_PLANNING.md
 - BASELINE_WATCHLIST_TOMBSTONES.md
 - CALIBRATION_EFFICACY.md
 - COMPOSE_STACK.md
 - DISCOVERY.md
 - ENGINEERING_BASELINE.md
 - FIRST_USER_SETUP_TRACKER.md
 - MULTI_MACHINE.md
 - NEW_MACHINE_SETUP.md
 - PROJECT_DOC_TEMPLATE.md
 - README.md
 - ROADMAP.md
 - TEST_PLAN.md
 - WORK_AUDIT_2026-07-08.md
- plugins/
 - pr-review-loop/
 - README.md
 - skills/
 - start/
 - SKILL.md
- pyproject.toml
- README.md
- scripts/
 - daily-export.ps1
 - daily-export.sh
- SESSION_HANDOFF.md
- sources.example.toml
- tests/
 - __init__.py
 - conftest.py
 - test_archive.py
 - test_archive_status.py
 - test_baseline.py
 - test_baseline_agent.py
 - test_baseline_calibration.py
 - test_baseline_eval.py
 - test_baseline_handoffs.py
 - test_baseline_ingest.py
 - test_baseline_lint.py
 - test_baseline_publish.py
 - test_baseline_redaction.py
 - test_baseline_replay.py
 - test_baseline_replay_ingest.py
 - test_claude_extractor.py
 - test_cli.py
 - test_codex_extractor.py
 - test_config.py
 - test_deepseek_extractor.py
 - test_gemini_extractor.py
 - test_grok_extractor.py

- test_models.py
- test_module_imports.py
- test_path_templates.py
- test_registry.py
- test_render.py
- test_tool_wrapper.py
- test_utils.py
- tools/
 - agent_archive.py

8. tool_result

```
1?[build-system]
requires = ["setuptools>=77"]
build-backend = "setuptools.build_meta"

[project]
name = "agent-sessions"
version = "0.2.0"
description = "Local coding-agent session archive tooling"
requires-python = ">=3.11"
10?license = {text = "Private"}

[project.scripts]
agent-archive = "agent_sessions.cli:main"

[tool.setuptools]
packages = ["agent_sessions", "agent_sessions.sources"]

[project.optional-dependencies]
dev = [
20?     "mypy>=1.10",
        "pytest>=7.0",
        "pytest-cov>=4.0",
        "reportlab>=4.0",
        "ruff>=0.6",
        "types-reportlab>=4.0",
]

[tool.ruff]
line-length = 120
30?target-version = "py311"

[tool.mypy]
python_version = "3.11"
warn_unused_configs = true

[tool.pytest.ini_options]
testpaths = ["tests"]
python_files = ["test_*.py"]
python_classes = ["Test*"]
40?python_functions = ["test_*"]
addopts = ["-v", "--tb=short", "--strict-markers"]
markers = [
    "slow: marks tests as slow (deselect with '-m \"not slow\"')",
    "integration: marks tests as integration tests",
]

[tool.coverage.run]
source = ["agent_sessions"]
omit = ["tests/*", "tools/*"]
50?

[tool.coverage.report]
fail_under = 80
show_missing = true
```

```
skip_covered = false
```

9. tool_result

```
1?# Agent Sessions
```

Private archive for coding-agent sessions across machines and tools.

This repo is meant to hold portable Markdown/PDF exports, plus the tooling used to recreate them from local agent stores. Raw logs are supported but ignored by Git by default because they can be large and may contain credentials, file contents, or tool outputs.

```
10?## Current Importers
```

- Codex Windows: local and archived JSONL sessions
- Claude Code Windows/WSL: project JSONL sessions
- Gemini Antigravity Windows: transcript JSONL files
- Grok WSL: `chat\_history.jsonl` files
- DeepSeek V4 VS Code extension: request dump prompts from VS Code globalStorage

VS Code Copilot Chat and Z.AI/ZAI locations are discovered and documented, but the first exporter treats them as source inventory unless transcript files are present in known locations.

Default sources live in `config/default\_sources.toml`. Copy `sources.example.toml` to `sources.toml` for local machine overrides.

Quick Start

```
```powershell
python -m pip install -e ".[dev]"
python .\tools\agent_archive.py discover --write docs\DISCOVERY.md
30?python .\tools\agent_archive.py status
python .\tools\agent_archive.py export --all
```
```

Optional PDF output:

```
```powershell
python .\tools\agent_archive.py export --all --pdf
```
```

40?Optional raw backups:

```
```powershell
python .\tools\agent_archive.py export --all --copy-raw
```
```

Raw files land under `raw/`, which is ignored by Git unless you intentionally force-add it.

Expected Outputs

```
50?
```

`discover --write docs\DISCOVERY.md` refreshes the local source inventory:

- configured source roots, whether each root exists, and matching file counts;
- sample files per source;
- inventory-only sources such as Copilot/ZAI storage locations, even when they are not exportable transcript sources yet.

`status` prints archive freshness and convergence signals:

```
60?- indexed records, visible configured files, new files, and changed files;
```

- indexed records not visible from this machine, preserved from other machines;
- source counts and inferred origin environments.

`export --all` writes Markdown archive artifacts and updates the shared catalog:

```
- `archive/**/*.*md`
- `archive/index.jsonl`
- `archive/INDEX.md`
```

70?`export --all --pdf` also writes `archive/\*\*/\*.\*pdf` when `reportlab` is installed. The `[dev]` install includes `reportlab`; to check a minimal environment, run:

```
```powershell
python -c "import reportlab; print('reportlab ok')"
```

If PDF support is missing, Markdown export still works. The CLI will report that PDF export requires `reportlab`.

80?

Some configured sources are intentionally inventory-only. A message like this is expected unless transcript files exist in supported locations:

```
```text
Skipped sources without extractors:
- copilot-vscode-windows-inventory (inventory)
- copilot-vscode-wsl-ubuntu-inventory (inventory)
- zai-vscode-wsl-ubuntu-inventory (inventory)
```
```

90?

After a real export, review only the intended generated paths:

```
```powershell
git status --short archive/ docs/DISCOVERY.md
```

Stage explicit paths only. Do not commit `sources.toml`, `raw/`, or unrelated files.

100?## Agent-Assisted Setup

To set this up on a new computer with Codex, Claude, Gemini, Grok, DeepSeek, or another capable local agent, give the agent this prompt from the machine you want to add:

```
```text
Set up my private agent-sessions archive on this computer.
```

1. Clone or open the repo:

110? <https://github.com/avidullu/agent-sessions>

Pull with `git pull --ff-only` before reading files.

2. Install local tooling in a Python 3.11+ environment:

```
`python -m pip install -e "[dev]"`
```

3. Validate the repo and report results. Use POSIX-style paths in the prompt;

PowerShell users may substitute `.\tools\...` and `docs\DISCOVERY.md` if they prefer:

```
- `python -m pytest --cov=agent_sessions --cov-report=term-missing`
- `python -m ruff check .`
- `python -m mypy agent_sessions tools`
```

120? - optional/informational: `python tools/agent\_archive.py baseline eval --dry-run`

4. Discover local agent stores:

```
- `python tools/agent_archive.py discover --write docs/DISCOVERY.md`
- `python tools/agent_archive.py status`
```

If defaults miss a local path, create or edit ignored `sources.toml`; do not commit `sources.toml`.

5. Ask me which sync mode I want before enabling it:

- manual: export only when I ask
- scheduled: daily Task Scheduler/cron export
- triggered: filesystem watcher with debounce

130? Ask separately whether to generate PDFs.

6. If I approve the first sync, run:

```
`python tools/agent_archive.py export --all --pdf`
```

Then stage only `archive/` changes. Push directly only if I explicitly approve this as a one-time archive sync; otherwise branch and open a PR.

7. Finish with a short setup report: validation status, agents discovered, total indexed sessions, new/changed files, origin environments, sync mode, and 1-2 promoted guardrails from `baseline/global/` or optional `baseline suggest --dry-run` output that show the value.

140?Do not commit raw logs, `sources.toml`, unrelated files, or merge PRs without explicit approval scoped to that PR or project.

```

The final setup report should be plain enough to review at a glance:

```text

Repo validation:

- Tests/coverage:
- Ruff:

150?- Mypy:

- Baseline eval:

Local archive status:

- Agents/sources discovered:
- Indexed sessions:
- New files:
- Changed files:
- Origin environments:

160?Sync:

- Selected mode:
- PDF export:
- Last export/commit:

Value preview:

- Guardrail/pattern 1:
- Guardrail/pattern 2:
- Evidence breadcrumbs:

```

170?

See docs/AUTOMATION.md for scheduled export details and docs/MULTI_MACHINE.md for how indexes converge across computers. For a step-by-step manual checklist, see docs/NEW_MACHINE_SETUP.md.

Adding Agents

1. Add a source entry in `config/default\_sources.toml` or local `sources.toml`.
2. Add an extractor module under `agent\_sessions/sources/`.
- 180?3. Register it with `@register("<kind>")`.
4. Run a dry export with `--source <kind> --limit 1 --dry-run`.

Baseline Status And Follow-Ups

The original promote/publish/calibrate closure proof is documented in docs/BASELINE_LOOP_CLOSURE.md. The newer knowledge and replay tracker is complete in docs/BASELINE_KNOWLEDGE_REPLAY_PLAN.md, with current health and follow-ups summarized in

190?docs/WORK_AUDIT_2026-07-08.md.

Useful health checks:

```
```powershell
python .\tools\agent_archive.py baseline lint --dry-run
python .\tools\agent_archive.py baseline eval --dry-run
python .\tools\agent_archive.py baseline handoffs audit --dry-run
python .\tools\agent_archive.py baseline replay select --dry-run
```
```

200?

Engineering Baseline

Create or refresh the baseline scaffold:

```
```powershell
python .\tools\agent_archive.py baseline scaffold
```
```

Expected output: missing baseline folders and templates are created under 210?`baseline/`, including calibration examples and proposal scaffolding. Existing files are preserved.

Generate reviewable candidate predictions from the archive:

```
```powershell
python .\tools\agent_archive.py baseline suggest
```
```

Expected output: a dated candidate report appears under `baseline/candidates/`, 220?with a matching `.predictions.json` sidecar, and the prediction ledger under `baseline/metacognition/` is updated. Candidate reports are suggestions with provenance and calibration hooks. Copy `baseline/calibration/feedback.example.toml` to `baseline/calibration/feedback.toml` to mark predictions as accepted, edited, or rejected before the next run.

Summarize calibration feedback against the latest prediction sidecar:

```
```powershell
230?python .\tools\agent_archive.py baseline calibrate --feedback
baseline\calibration\feedback.toml
```
```

Create a local evidence bundle for an authorized AI agent to draft proposals:

```
```powershell
python .\tools\agent_archive.py baseline bundle --focus badminton-highlight-indexer
```
```

Compose Stack

240?

This repo owns durable export and baseline generation. Search, Claude-specific browsing, live capture, and runtime memory are delegated to external tools where they are already stronger. See docs/COMPOSE_STACK.md.

Daily export automation is documented in docs/AUTOMATION.md. Multi-machine indexing is documented in docs/MULTI_MACHINE.md.

Reusable Plugins

250?This repo also hosts a Claude Code plugin marketplace (`agent-sessions-tools`). Install the `pr-review-loop` PR-reviewer plugin from any session:

```
```shell
/plugin marketplace add avidullu/agent-sessions
/plugin install pr-review-loop@agent-sessions-tools
```
```

See plugins/pr-review-loop/README.md.

260?## Other Machines

Clone this private repo on another machine, follow docs/NEW_MACHINE_SETUP.md, commit the new `archive/` Markdown/PDF files, and push. The archive index is merge-aware, so records from other machines remain in the unified view when one machine exports only the local sources it can see.

If a machine has different usernames, WSL distribution names, or custom storage paths, copy `sources.example.toml` to `sources.toml` and edit the roots. The 270?local `sources.toml` is ignored by Git.

Roadmap

See docs/ROADMAP.md for optional future importers and archive automation ideas.

The engineering-baseline idea is sketched in docs/ENGINEERING_BASELINE.md. The current implementation plan is tracked in 280?docs/BASELINE_PLANNING.md.

10. tool_result

1?# Docs Index

| Doc | Purpose |
|--|--|
| BASELINE_KNOWLEDGE_REPLAY_PLAN.md | **DONE** - compounding knowledge layer, handoff mining, and replay loop (#23, #25, #26); K0-K12 landed |
| WORK_AUDIT_2026-07-08.md | **DONE** - repository health audit, prioritized debt, follow-ups, and docs freshness cleanup |
| FIRST_USER_SETUP_TRACKER.md | **DONE** - first-user and new-machine setup UX MVP (#42 + #18) |
| NEW_MACHINE_SETUP.md | Copy/paste setup and sync validation checklist for adding another computer |
| BASELINE_LOOP_CLOSURE.md | **DONE / historical** - original promote/publish/calibrate closure proof; remaining backlog split to follow-up issues/designs |
| 10? CALIBRATION_EFFICACY.md | Measurable efficacy gates E1?E6 |
| BASELINE_WATCHLIST_TOMBSTONES.md | **Design (review)** ? watchlist backlog + rejection tombstones (P10/P11) |
| archives/TECH_DEBT_PLAN.md | **DONE (archived)** ? tech-debt remediation plan, TD1?TD15 all landed (#28, #31, #33, #35, #37, #38, #40) |
| PROJECT_DOC_TEMPLATE.md | Tracked project doc skeleton (from `badminton-highlight-indexer`) |
| COMPOSE_STACK.md | What this repo owns vs external tools |
| AUTOMATION.md | Daily export scripts |
| ENGINEERING_BASELINE.md | Baseline architecture |
| BASELINE_PLANNING.md | PR-sized implementation plan |
| ROADMAP.md | Future importers and features |
| DISCOVERY.md | Local source inventory report |

20?

11. tool_result

1?# Engineering Baseline

For current planning, pilot scope, and PR-sized work items, see BASELINE_PLANNING.md.

Context

The archive captures useful evidence: which repositories show up repeatedly, which guardrails prevent regressions, which prompts lead to strong outcomes, and 10?which workflow mistakes cost time. That information should become a local, reviewable engineering baseline rather than staying buried in transcripts.

Decision

Keep raw session export and baseline extraction as separate stages:

1. Archive sessions to durable Markdown/PDF and an index.
2. Derive candidate lessons from archived sessions.
3. Review and promote selected lessons into a local baseline.
- 20?4. Publish baseline slices into agent-specific files for Codex, Claude, and VS Code-supported workflows.

This keeps the archive faithful to source data while allowing the baseline to be curated, opinionated, and safe to inject into future projects.

Proposed Layout

```
```text
baseline/
30? SCHEMA.md
 global/
 engineering-guardrails.md
 prompt-patterns.md
 repo-workflows.md
 handoffs/
 audit.md
 index.jsonl
 agents/
 codex/AGENTS.generated.md
40? claude/CLAUDE.generated.md
 vscode/copilot-instructions.generated.md
 projects/
 <repo-slug>/
 README.md
 candidates/
 YYYY-MM-DD-extraction.md
```
```

Extraction Shape

- 50?
- Read `archive/index.jsonl` and selected Markdown transcripts.
 - Identify repeated repo names, commands, file conventions, failure modes, review findings, test gates, and user preferences.
 - Write candidate notes with provenance links back to archive files.
 - Require an explicit promote step before updating baseline files.

Agent Hooks

- Codex: project `AGENTS.md` can include or mirror the generated Codex baseline 60? section.
- Claude: project `CLAUDE.md` can include or mirror the generated Claude baseline section.
- VS Code: a VS Code adapter can publish a generated instruction file once the chosen local instruction target is validated on each machine.
- External/peer agents: `baseline bundle` can produce a bounded local evidence

packet plus a proposal prompt for any explicitly authorized agent. Generated evidence packets are ignored by Git by default.

- Handoff mining: ``baseline handoffs audit`` writes a report-only coverage and freshness audit at ``baseline/handoffs/audit.md``; ``baseline handoffs index``
- 70? ``baseline handoffs index`` writes persistent records to ``baseline/handoffs/index.jsonl`` and marker-owned ``handoffs.index`` feeds only on configured or already-scaffolded project pages. ``baseline handoffs proposals`` turns that index into deterministic proposal JSON files under ``baseline/proposals/``, refuses to overwrite hand-written proposals, and relies on ``baseline ingest --dry-run`` for reference validation.
- Project pages: generated sections in ``baseline/projects/<slug>/README.md`` should use ``render_project_page_block()`` and ``upsert_project_page_content()`` so the shipped ``baseline:begin/end`` marker parser preserves hand-written prose. Only the exact scaffold placeholder line is removed; edited placeholder-like prose is preserved as human-owned content.
- 80?- Baseline lint: ``baseline lint`` is read-only by default and reports schema, marker, generated-link, stale-block, malformed generated-date, orphan-page, and explicit contradiction findings. Errors fail the command; warnings surface review signals while later producers are still gated.
- Replay selection: ``baseline replay select`` scores archived sessions for replayability and writes a deterministic ``baseline/replay/manifest.jsonl`` of selected and near-miss candidates with exclusion reasons and no transcript excerpts. Coding sessions are excluded in v1; redaction and bundle egress are later gated work.
- Replay redaction: ``baseline replay redact`` runs a deterministic, fail-closed
- 90? ``secret scan`` over the selected sessions and writes a valueless ``redaction-report.json`` under the gitignored ``baseline/replay/bundles/``. High-confidence secrets block the egress gate (non-zero exit); low-risk emails and private paths are placeholdered. Secret values are never recorded.
- Replay bundles: ``baseline replay bundle`` writes one gitignored packet per selected session (redacted task prompt + original deliverable + comparison rubric + per-bundle redaction report). Sessions whose egress content trips the fail-closed scanner are skipped with a report and never written. Bundles stay under the gitignored ``baseline/replay/bundles/`` and are handed to an external replayer/judge out of band.
- 100?- Replay ingest: ``baseline replay ingest`` validates an external replay result (its ``replay_of``/trace references must resolve against ``archive/index.jsonl``), appends it to the append-only ``baseline/replay/ledger.jsonl``, and ? when the judge recommends it ? emits a ``replay.*`` proposal that flows through the same human-gated ``baseline ingest`` -> candidate -> promote pipeline. Nothing is auto-promoted.
- Efficacy gates: ``baseline eval`` reports E1-E6 plus the K12 W/H/R gates (schema/marker/link, handoff precision/freshness, replay R1-R5) and a ``G-no-autopromote`` governance gate. Gates whose prerequisites do not exist yet (e.g. no external replay result ingested) report ``gated`` rather than passing or
- 110? failing.

Guardrails

- Never promote secrets, credentials, private file contents, or one-off debug noise into the baseline.
- Keep provenance for every promoted rule or pattern.
- Prefer short, stable guidance over transcript summaries.
- Separate global guidance from project-specific guidance.
- Preserve manual sections in project instruction files; generated sections
- 120? should have clear begin/end markers.
- Run ``baseline lint --dry-run`` before merging generated baseline producers.
- Let the periodic job propose baseline changes, but keep promotion reviewable.

First Useful Version

The first version can be simple and deterministic:

- Scan archive metadata for repo/project names and agent sources.
- Extract headings and repeated commands from Markdown transcripts.

130?- Produce `baseline/candidates/<date>-extraction.md`.
- Add a manual `promote-baseline` command later, after the candidate format feels useful.

It should also make explicit metacognitive predictions about the user and the work style, then invite calibration feedback. The loop is: predict from local evidence, ask what was right or wrong, record feedback, and make the next report less vague. Prediction sidecars and a ledger make that loop machine-readable so future runs can compare what the tool guessed against what the user confirmed.

12. tool_result

17# Work Audit: 2026-07-08

```
> **Status:** `DONE` - audit report. **Owner:** `avidullu`. **Created:** `2026-07-08`. **Last updated:** `2026-07-08`  
> **Scope:** repository health, shipped baseline/replay work, follow-up debt, and docs freshness.  
> **Verified against:** `main` at `80b91d5`.
```

0. TL;DR

The repo is healthy on normal engineering gates: tests, coverage, lint, mypy, 10?baseline lint, and baseline eval all pass. The main debt is not broken code; it is freshness and next-phase clarity. Local archive stores have moved ahead of the committed archive, the docs still point at an older "active" baseline-loop project, and replay v1 is built but still needs a real out-of-band result before its value gates can measure acceptance.

1. Verification

```
Check	Result
20?	`python -m pytest --cov=agent_sessions --cov-report=term-missing`
`python -m ruff check .`	Pass
`python -m mypy agent_sessions tools tests`	Pass
`python .\tools\agent_archive.py baseline lint --dry-run`	`0` errors, `2` warnings
`python .\tools\agent_archive.py baseline eval --dry-run`	`14` pass, `0` fail, `2` gated
Open PRs	None
Working tree after audit	Clean
```

Current project-specific warnings:

30?- `baseline/projects/agent-sessions/README.md` has no inbound baseline links or generated blocks yet.
- `baseline/projects/avidullu/README.md` has no inbound baseline links or generated blocks yet.
- Replay gates `R3-signal` and `R4-value` are gated until a real external replay result is ingested.

2. Prioritized Debt

Scoring uses the repo's tech-debt rubric: $\text{priority} = (\text{Impact} + \text{Risk}) \times (6 - \text{Effort})$.
40?

| ID | Item | Impact | Risk | Effort | Priority | Why it matters |
|----|---|--------|------|--------|----------|---|
| A1 | Archive freshness backlog: status showed `1102` new visible files and `1` changed visible file. | 5 | 3 | 3 | 24 | Baseline and replay selection quality depend on recent sessions. |
| A2 | Stale active-project docs: README/docs index still pointed to `BASELINE_LOOP_CLOSURE.md` as active. | 4 | 3 | 1 | 35 | New sessions ramp up from the wrong project state. |
| A3 | Replay redaction v1, tracked in issue #59. | 5 | 4 | 3 | 27 | Replay bundles are the only path intended for out-of-band handoff; v0 is denylist-only. |

| A4 | Replay/knowledge v1 polish, tracked in issue #60. | 4 | 3 | 3 | 21 | Slug derivation, clean reruns, project-scoped replay proposals, and stronger gates will improve first replay value. |

| A5 | Issue hygiene for completed umbrella issues #23/#25/#26. | 3 | 2 | 1 | 25 | GitHub should reflect that K0-K12 shipped, with remaining work split into #59/#60/#32/#19. |

| A6 | Two intentional baseline-lint orphan warnings. | 2 | 2 | 1 | 20 | Either link/generated blocks should land, or the warnings should be documented as intentional. |

| A7 | Daily export scripts push by default and have minimal dirty-tree/branch locking. | 3 | 3 | 2 | 24 | Fine for manual use; scheduled runs need stronger guardrails if multiple sessions share the checkout. |

50? | A8 | Archive backfill/regenerate remains open as issue #32. | 3 | 4 | 5 | 7 | Useful data-integrity cleanup, but broad and intentionally deferred. |

3. Recommended Sequence

1. Refresh the committed archive intentionally: run an approved `export --all --pdf`, inspect the generated `archive/` diff, and commit only explicit archive paths.
2. Clean up planning state: close or comment on issues #23, #25, and #26 as completed by PRs #45-#57; keep #59, #60, #32, and #19 open.
3. Land issue #60 in small PRs: replay slug reuse from handoff slugging, clean 60? bundle reruns, project-scoped replay proposals, and stronger H1/R2 gates.
4. Land issue #59 before sending replay bundles to another agent for real work.
5. Decide whether the two orphan project pages should get generated/link blocks now or stay as known review warnings.

4. Documentation Updates

This audit prompted immediate docs cleanup:

- README now points to current baseline health and follow-up docs instead of 70? treating the older loop-closure tracker as the active project.
- `docs/README.md` indexes this audit and marks the loop closure as historical rather than active.
- `docs/BASELINE\_LOOP\_CLOSURE.md` now separates the completed original closure proof from backlog rows that remain future work.

5. Honest Limits

- This audit did not run a real archive export or commit generated archive changes.
- 80?- This audit did not execute an out-of-band replay, so `R3-signal` and `R4-value` remain correctly gated.
- This audit did not inspect every archived transcript manually; it relied on the repo's deterministic status, lint, and eval commands plus targeted code and docs review.

13. tool_result

1?# Baseline Knowledge + Replay Plan

```
> **Status:** `DONE` - execution tracker for issues #23, #25, and #26; all K rows landed (PRs #45-#57). **Owner:** `avidullu`. **Created:** `2026-07-07`. **Last updated:** `2026-07-07`
> **Lifecycle:** `DRAFT -> IN PROGRESS -> DONE -> archived`
> **Tracking anchors:** Section 7 is the source of truth; indexed in `docs/README.md`; pointer in `SESSION_HANDOFF.md`.
> **Relation to existing docs:** extends `docs/BASELINE_LOOP_CLOSURE.md`, `docs/CALIBRATION_EFFICACY.md`, and `docs/COMPOSE_STACK.md`; treats issue #19 as a shared provenance dependency.
> **Honesty note:** claims marked `[verified]` were checked against this repo or the linked issue text; `[researched]` items come from external sources in Section 10; `[design]` items are not implemented yet.
```

10?

0. TL;DR

Build the compounding knowledge layer, report-only handoff audit, and replay pipeline as one safety-first program. The shared substrate is a deterministic, human-gated `baseline/SCHEMA.md`, #19-aligned trace fields, marker-owned project pages, and lint gates. Report-only handoff coverage can ship early; any generated knowledge writes or replay egress wait for schema, provenance, ingest-integrity, and redaction contracts.

20?## 1. Problem & goal

****Problem:**** the baseline loop can ingest structured proposals and publish reviewed guardrails, but it does not yet compound project knowledge, mine the repo's own handoff conventions, or replay archived sessions to compare alternate agent outputs. Issues #23, #25, and #26 are three faces of the same product: turn archived work into reviewable knowledge that improves future sessions without giving an LLM silent write access to policy.

****Goal:**** a safe knowledge-and-replay pipeline where:
30?

1. Project pages accumulate durable, cross-linked knowledge with explicit provenance.
2. Session handoffs become normalized evidence records and proposal inputs.
3. Replay packets can be generated deterministically, executed out-of-band by a different agent or panel, and ingested only as human-reviewable proposals.
4. Lint, calibration, and promotion gates prove the loop compounds knowledge instead of rewriting history.

2. Decisions locked

40?

| # | Decision | Source / date | Implication |
|-----|--|---|---|
| D1 | Do `baseline/SCHEMA.md` before generated knowledge writes or replay egress. | #26 + research, 2026-07-07 | Derived pages, proposals, replay bundles, and ingested replay results get ownership, provenance, and lint rules before they write. A report-only handoff coverage audit may ship earlier. |
| D2 | Keep raw sources immutable and derived pages deterministic. | LLM-wiki prior art | |
| D3 | Use marker-block upserts and generated sections; do not let an LLM freely rewrite project pages. | | |
| D3 | Ship handoff mining before replay. | #23/#25 scope comparison | |
| D4 | Replay execution is out-of-band in v1. | #25 + eval tooling research | |
| D5 | Exclude coding sessions from replay v1. | #25 non-goal | |
| D6 | Align lightweight provenance to #19 trace fields now. | #19 dependency | |
| D7 | Do not build search or embeddings for v1. | docs/COMPOSE_STACK.md + #23/#26 non-goals | |
| D8 | Reuse the shipped marker grammar and content-preserving upsert. | | |
| D9 | Keep existing tracker rows as shared capability owners. | docs/BASELINE_LOOP_CLOSURE.md | |
| D10 | Treat replay bundle redaction as a first-class safety deliverable. | PR #45 review, 2026-07-07 | |
| D11 | External proposals need provenance-integrity checks before becoming candidates. | | |

`baseline\_ingest.py`, PR #45 review `[verified]` | Replay/handoff proposal ingest must verify `replay\_of` and evidence trace references against `archive/index.jsonl`, not rely only on human promote review. |

3. Foundation - research and prior art

| Source | Useful takeaway | Design consequence |
|---|---|---|
| LLM-wiki concept `[researched]` | Keep raw sources as immutable input, maintain structured interlinked markdown, and use schema/index/lint conventions. | `baseline/SCHEMA.md`, project-page marker blocks, page indexes, stale/orphan/contradiction lint. |
60?| Reflexion `[researched]` | Verbal reflections can improve later agent behavior without changing model weights. | Treat handoffs and replay judgments as reflection-like evidence, not as automatic policy. |
| Generative Agents `[researched]` | Experiences can be stored, reflected into higher-level summaries, and retrieved for planning. | Separate raw archive, normalized records, synthesized project pages, and future prompt packets. |
| Contextual Experience Replay `[researched]` | Past experiences can be accumulated and synthesized into a memory buffer for training-free improvement. | Replay and handoff outputs should land as structured, reviewable experience records. |
| AgentRR `[researched]` | Recording and summarizing agent trajectories supports replay for similar tasks. | `baseline replay select` should emit deterministic manifests and bundles tied to session identity. |
| AgentHER `[researched]` | Failed trajectories can be relabeled into useful training data when classification and confidence gates are explicit. | Replay ingest needs rubrics, confidence, judge identity, and human gates; no blind promotion. |
| Helicone replay / LangSmith trajectory evals / OpenAI agent evals `[researched]` | Production replay/eval systems rely on traces, graders, and full interaction context. | Because this repo has archived transcripts but not always full tool traces, v1 replay stays conservative. |
| W3C PROV-DM `[researched]` | Provenance models connect entities, activities, agents, derivations, and bundles. | Use the concept of derivation, but keep field names aligned to issue #19's concrete breadcrumb vocabulary. |

4. Architecture

```
70?``mermaid
flowchart LR
    Archive["archive/ + index.jsonl"] --> Schema["baseline/SCHEMA.md"]
    Schema --> Pages["baseline/projects/<slug>/README.md"]
    Schema --> Lint["baseline lint"]

    Handoffs["handoff artifacts"] --> HandoffAudit["baseline handoffs audit"]
    HandoffAudit --> AuditReport["baseline/handoffs/audit.md"]
    Handoffs --> HandoffIndexer["baseline handoffs index"]
    HandoffIndexer --> HandoffIndex["baseline/handoffs/index.jsonl"]
80? HandoffIndex --> Pages
    HandoffIndex --> Proposals["baseline/proposals/*.json"]

    Archive --> ReplaySelect["baseline replay select"]
    ReplaySelect --> Manifest["baseline/replay/manifest.jsonl"]
    Manifest --> Redact["redaction preflight + report"]
    Redact --> Bundle["baseline/replay/bundles/*"]
    Bundle --> Replayer["external agent / panel"]
    Replayer --> ReplayIngest["baseline replay ingest"]
    ReplayIngest --> Proposals
90? Proposals --> Candidates["baseline/candidates/"]
    Candidates --> Calibrate["baseline calibrate"]
    Calibrate --> Promote["baseline promote"]
    Promote --> Published["baseline/global/ + agent slices"]
    Lint --> Pages
    Lint --> Proposals
...

```

4.1 Relation to existing tracker rows and code

100?This doc is the execution tracker for issues #23, #25, and #26, but it must not fork shared baseline-loop work already tracked in `docs/BASELINE\_LOOP\_CLOSURE.md`.

```
This plan	Existing owner	Reconciliation
K3 project-page upserts	TD4 content-preserving promote, shipped in #31	Reuse or generalize
`upsert_promoted_content()` and `parse_promoted_blocks()` instead of writing a second marker-		
block system.		
K4 `baseline lint`	P6 contradiction detection	K4 is the #26 implementation slice for P6
plus link/staleness/orphan checks; close or cross-link P6 when K4 lands.		
K8/K11 replay select/ingest	P5 cross-agent project correlation	Replay provenance should
feed P5 once correlation exists; K8 can start without P5, but K11 must preserve agent/lineage		
fields for it.		
K9/K10 replay redaction and bundles	P10/P11 watchlist + tombstone/redaction design	K10 is
hard-blocked on K9's deterministic redaction preflight. K9 may reuse P10/P11 helpers when they
exist, but this tracker owns the replay-bundle egress gate. |
```

110?### 4.2 Knowledge layer (#26)

Add `baseline/SCHEMA.md` as the contract for derived knowledge:

- Page types: global guardrail, project page, handoff index, replay manifest, proposal, candidate, calibration report.
- Ownership: human-owned prose, generated marker blocks, append-only ledgers.
- Trace/provenance fields aligned to #19: `source`, `source\_file`, `markdown\_path`, `session\_id`, `timestamp`, `project\_slug`, `repo`, `evidence\_anchor`, `evidence\_excerpt`, `transform`, `bundle\_id`, and 120? optional `calibration\_effect`.
- Link rules: every generated block links to source evidence or a proposal id.
- Lint rules: schema conformance, broken links, orphan pages, stale generated blocks, duplicate claims, and contradiction candidates.
- Validation mechanism: `baseline/SCHEMA.md` is the human-readable contract; conformance is enforced by `baseline lint` and command-specific validators. `baseline/proposals/proposal.schema.json` is currently an illustrative example, while `agent\_sessions/baseline\_ingest.py` is the actual proposal validator.

Project pages remain markdown, but generated regions are bounded:

```
130?
```markdown
<!-- baseline:begin id="knowledge.handoff-patterns" -->
...
<!-- baseline:end id="knowledge.handoff-patterns" -->
```
```

The first pass should update only marker-owned blocks. Human prose remains outside generated blocks. Metadata such as `generated\_by` should live inside the block body or in structured sidecars unless K1 deliberately extends the existing 140?parser grammar used by both promote and publish.

4.3 Handoff mining (#23)

Handoff mining should be deterministic and auditable:

- Discover `SESSION\_HANDOFF.md`, `session-handoff.md`, `MEMORY.md` start-here pointers, and agent-specific handoff sections in archived sessions.
- K2 `baseline handoffs audit` writes only `baseline/handoffs/audit.md` as a coverage/freshness report; it does not write pages, proposals, or indexes. 150?- K6 `baseline handoffs index` normalizes records into `baseline/handoffs/index.jsonl`.
- Feed stable patterns into project pages and K7 proposal JSON. Project-page feed dates should change only when the generated feed content changes, so periodic runs do not create date-only diffs.
- K7 `baseline handoffs proposals` converts indexed records for configured or already-scaffolded projects into deterministic proposal JSON under `baseline/proposals/`; it refuses to overwrite hand-written proposals and

- leaves promotion to the existing review/ingest flow.
- Extract structured fields with heading/marker scraping only. This is deterministic for repo handoffs that follow the local convention (`## Next Steps / Open Threads`, ramp-up kit, decisions), but arbitrary formats are best-effort: non-conforming handoffs are flagged with parse warnings, not silently summarized or mis-parsed.
- Derive project slugs explicitly: ``archive/index.jsonl`` carries ``messages``, ``sha256``, and ``markdown`` at top level, but ``cwd`` and ``project`` live under each record's ``metadata``. The current ``metadata.project`` value is a URL-encoded absolute path, so K6/K8 need a decode-and-slug step with tests.

Suggested normalized handoff record:

```
170? ```json
{
  "id": "handoff.agent-sessions.2026-07-07.codex",
  "project_slug": "agent-sessions",
  "project_raw": "%2FC%3A%2FUsers%2Favidu%2FProjects%2FAgent%20Sessions",
  "repo": "https://github.com/avidullu/agent-sessions",
  "source_kind": "repo-handoff",
  "observed_at": "2026-07-07T00:00:00Z",
  "open_threads": ["baseline knowledge + replay design"],
180?  "key_decisions": ["schema before replay"],
  "ramp_up_links": ["docs/BASELINE_KNOWLEDGE_REPLAY_PLAN.md"],
  "parse_warnings": [],
  "trace": [
    {
      "source": "codex",
      "source_file": "SESSION_HANDOFF.md",
      "markdown_path": "SESSION_HANDOFF.md",
      "session_id": null,
      "timestamp": "2026-07-07T00:00:00Z",
190?   "project_slug": "agent-sessions",
      "repo": "https://github.com/avidullu/agent-sessions",
      "evidence_anchor": "## Next Steps / Open Threads",
      "evidence_excerpt": "schema/compiled wiki first, handoff mining as the first producer",
      "transform": "handoff-heading-extraction"
    }
  ]
}
...

```

200?### 4.4 Replay loop (#25)

Replay v1 is selection, packaging, and ingest - not local autonomous execution.

Proposed CLI:

- ``baseline replay select --kind planning --limit 20``
- deterministically picks sessions with enough self-contained context, excludes coding sessions, and writes a manifest.
- ``baseline replay bundle --manifest <path>``
- 210? - runs redaction preflight, writes a redaction report, then creates gitignored prompt/evidence packets with rubric files only if the safety gate passes.
- ``baseline replay ingest --result <path>``
- validates an external replay result, verifies archive references, and converts improvements into structured proposals or candidate records.

The nested CLI shape is intentional in this design for readability. The current baseline CLI is flat (``suggest``, ``promote``, ``publish``, ``ingest``, ``bundle``, etc.), so K7 must either add nested subparsers deliberately or choose flat names such as ``replay-select`` / ``replay-bundle`` and document the convention.

220?

Replay result minimum fields:

- `replay\_of`: original archive/session id.
- `replayer`: agent, model family if known, and lineage different from original.
- `rubric\_version`: versioned scoring prompt or panel rubric.
- `claim`: what the alternate run improves.
- `evidence`: original excerpt ids plus replay output ids.
- `confidence`: numeric or enum, with explanation.
- `recommended\_action`: `proposal`, `watchlist`, or `reject`.

230?

Replay ingest cannot rely on the current shallow `validate\_proposal()` alone.
For `source\_kind = "replay"` or `source\_kind = "handoff"`, ingest must:

- verify `replay\_of`, when present, resolves to an `archive/index.jsonl` record;
- verify each structured trace `markdown\_path` or `session\_id` resolves to an archive record, or report a rejected proposal with the missing reference;
- thread accepted trace fields through `Prediction`, `proposal\_to\_prediction()`, `prediction\_to\_dict()`, and prediction sidecars so provenance survives ingest.

240? Existing free-text `evidence` entries can remain valid for hand-written proposals, but external replay/handoff producers must supply structured trace records before their outputs become candidates.

Daily automation follows #25: it may run only deterministic queueing stages (`replay select` and `replay bundle`) after R1-select, R2-dedup, and R5-safety pass on fixtures/sampled bundles. Replay execution, judging, ingest review, and promotion stay human-triggered.

Replay gate names keep issue #25's vocabulary:

250?

| Gate | Meaning |
|-----------|--|
| R1-select | Reviewed selected sessions are genuinely replayable. |
| R2-dedup | Re-running select/bundle with no new sessions produces zero new manifest entries. |
| R3-signal | At least one replay-derived proposal validates end to end. |
| R4-value | Acceptance rate of `replay.*` candidates is measured against keyword-derived and handoff-derived candidates in the ledger. Pause replay expansion if the first 20 reviewed replay candidates produce zero accepts, or if replay acceptance is less than half the baseline candidate acceptance rate. |
| R5-safety | Bundles are redacted, gitignored, and contain no sampled secrets. |

4.5 Redaction and egress safety

260?

Replay bundles are the only planned artifact that can carry full archived task prompts and original deliverables to an external replayer or judge. That makes redaction a first-class deliverable, not an implementation detail inside K10.

V0 redaction is deterministic and fail-closed:

- Scan task prompts, original deliverables, selected excerpts, filenames, and bundle metadata before writing any external packet.
- Detect common high-confidence secret forms: GitHub tokens (`ghp\_`, `gho\_`, `github\_pat\_`), OpenAI-style `sk-` keys, AWS access keys, private-key blocks, Slack `xox\*` tokens, connection strings with passwords, and environment assignments for names containing `TOKEN`, `SECRET`, `PASSWORD`, `API\_KEY`, or `PRIVATE\_KEY`.
- Replace lower-risk private paths/emails with stable placeholders when doing so does not destroy task meaning; block the bundle when a high-confidence secret appears.
- Write a `redaction-report.json` beside the gitignored bundle with counts, placeholder ids, blocked reasons, source refs, and scanner version.
- Add gitignore coverage for replay bundles before any bundle-writing command exists.

K10 is blocked until this preflight has fixture coverage and R5-safety is passing. If P10/P11 redaction helpers land first, K9 reuses them; otherwise this tracker ships the minimal deterministic scanner as the replay egress gate.

5. Artifacts and contracts

| Path | Writer | Reader | Notes |
|------|---|--|---|
| 290? | `baseline/SCHEMA.md` | Human-reviewed docs PR | all baseline producers Required before generated knowledge writes. |
| | `baseline/projects/<slug>/README.md` | humans + marker-block upserts | agents and reviewers Existing page shape is reused. |
| | `baseline/handoffs/index.jsonl` | `baseline handoffs index` | project page updater, proposal generator Deterministic, append/upsert by source id. |
| | `baseline/handoffs/audit.md` | `baseline handoffs audit` | human reviewer Human-readable gaps and confidence. |
| | `baseline/replay/manifest.jsonl` | `baseline replay select` | bundle command Tracked only if it contains session refs/exclusion reasons and no excerpts. |
| | `baseline/replay/bundles/*` | `baseline replay bundle` | external replayer/panel Gitignored; bundles may contain session excerpts. |
| | `baseline/replay/bundles/*/redaction-report.json` | redaction preflight | human reviewer, R5-safety Gitignored by default; records scanner version, replacements, and blocked reasons. |
| | `baseline/replay/ledger.jsonl` | `baseline replay ingest` | calibration/eval Append-only replay result history. |
| | `baseline/proposals/*.json` | humans, `baseline handoffs proposals`, replay ingest | `baseline ingest` Replay/handoff producers include structured trace fields that ingest validates against `archive/index.jsonl`. |

300?## 6. Honest limits - what this does NOT do

- No embeddings or semantic search in v1.
- No automatic promotion from handoff or replay findings.
- No free-form LLM rewrite of `baseline/projects/\*`.
- No coding-session replay until snapshots or commit/worktree provenance exist.
- No replacement for the existing promote, publish, calibrate, and efficacy gates.

7. Deliverables & progress tracker

310?Legend: `Todo`, `In progress`, `Done`, `Blocked/gated`. One small PR per row.

| ID | Deliverable | Issue | Depends on | Gated? | Status | PR |
|---------|---|-------------|--------------------|--------|--------|-----|
| K0 | Tracked design plan, docs index, and handoff pointer | #23/#25/#26 | - | No | Done | #45 |
| K1 | `baseline/SCHEMA.md` with page types, existing marker grammar, #19-aligned trace fields, and lint/validator contract | #26/#19 | K0 | Yes | Done | #46 |
| K2 | Report-only `baseline handoffs audit` coverage/freshness report; writes `baseline/handoffs/audit.md` only and no page/proposal/index writes | #23 | K0 | No | Done | #47 |
| K3 | Reuse/extend `upsert_promoted_content()` for project-page generated sections | #26 | K1, TD4 #31 | Yes | Done | #48 |
| K4 | `baseline lint` skeleton for schema, links, stale blocks, orphan pages, and P6 contradiction checks | #26 | K1, K3 | Yes | Done | #49 |
| K5 | Proposal + `Prediction` trace-field extension and ingest reference validation against `archive/index.jsonl` | #19/#23/#25 | K1 | Yes | Done | #50 |
| 320? K6 | `baseline handoffs index` discovery records in `baseline/handoffs/index.jsonl` and project-page feed | #23/#26 | K2, K3, K5 | No | Done | #51 |
| K7 | Handoff-derived proposal generation with trace records | #23 | K5, K6 | Yes | Done | #52 |
| K8 | `baseline replay select` deterministic manifests, excluding coding sessions | #25 | K1 | No | Done | #53 |
| K9 | Replay redaction v0: deterministic scanner, redaction report, fixture tests, and bundle gitignore coverage | #25 | K0, P10/P11 design | Yes | Done | #54 |
| K10 | `baseline replay bundle` gitignored packets with rubric files after redaction preflight | #25 | K8, K9 | Yes | Done | #55 |
| K11 | `baseline replay ingest` validates external replay results into proposals/candidates | #25 | K5, K10 | Yes | Done | #56 |
| K12 | Efficacy gates for schema lint, handoff precision, replay R1-R5, and proposal acceptance | #23/#25/#26 | K4, K7, K11 | Yes | Done | #57 |

Additional gate names:

330?- `W1-schema`: `baseline lint` validates marker grammar, generated-block ownership, and schema references.

- `W2-links`: generated cross-links resolve and no project page is orphaned.
- `H1-handoff-precision`: fixture handoff patterns are detected while irrelevant docs are ignored.
- `H2-handoff-freshness`: stale and missing handoffs are reported with reasons.
- `R1-select` through `R5-safety`: replay gates from issue #25. Watchlist/tombstone `E7` remains owned by P11 in `docs/BASELINE\_WATCHLIST\_TOMBSTONES.md`.

8. Related issue triage

340?

| Issue | Fold into this plan? | Reason |
|----------------------------------|----------------------|---|
| #19 - provenance trace logs | Yes, as substrate | Replay and handoff evidence need trace ids; v1 trace fields should be a strict subset of #19's breadcrumb vocabulary. |
| #23 - mine handoffs | Yes, primary | First producer for the knowledge layer. |
| #25 - replay loop | Yes, primary | Second producer after schema, handoff mining, and redaction contracts. |
| #26 - wiki-style knowledge layer | Yes, primary | Foundation for both handoff mining and replay output. |
| #32 - backfill/regenerate | No, boundary | Useful for archive identity quality and any future `baseline/handoffs/index.jsonl --prune` semantics, but too broad for this project; keep it separate unless replay selection needs exact regenerated ids. |

9. Open questions - owner / external

350?

1. Should `baseline/SCHEMA.md` be entirely hand-authored, or should commands also print machine-readable schema fragments from code?
2. Should the default stale threshold be configurable per project, with 90 days as the v1 default?
3. Should K1 formalize `baseline/proposals/proposal.schema.json` as real JSON Schema, or keep validation in Python and treat the file as an example?
Keep validation in Python for K5; the schema JSON remains illustrative.
4. Should structured trace fields live directly on proposal objects, or in a sibling sidecar only if existing proposal writers need strict compatibility?

360? **K5 puts `trace` directly on proposal objects and copies it into `Prediction.trace` / prediction sidecars.**

5. Should K6 write project pages for every discovered raw project slug?
No. K6 persists all discovered handoff candidates in `baseline/handoffs/index.jsonl`, but writes `handoffs.index` marker blocks only to configured or already-scaffolded project pages.
6. Should K7 generated proposal files be auto-promoted?
**No. K7 writes deterministic proposal JSON for configured or existing project pages only, with structured trace and human review via `baseline ingest`; generated proposal overwrites are limited to files already

370? marked with `generated\_by = "baseline handoffs proposals"`.**

7. Which session classes are allowed for replay v1 besides planning, writing, documentation, and research?
8. Should R5-safety allow a manual override for blocked bundles, or should v1 remain strictly fail-closed with no override?

10. Definition of done

- [x] `baseline/SCHEMA.md` exists and `baseline lint` plus command-specific validators enforce it for new producers. (K1, K4)
- 380?- [x] Project pages have deterministic generated blocks with evidence links. (K3, K6 `handoffs.index` feeds carry #19-aligned trace)
- [x] `baseline lint` fails on broken generated links and stale/orphan blocks. (K4; broken links are errors, stale/orphan are review warnings by design)
 - [x] `baseline handoffs audit` finds repo and archive handoff artifacts, reports missing/stale/non-conforming handoffs, and does not require generated page writes. (K2)

- [x] At least one handoff-derived finding reaches candidate/proposal review with #19-aligned trace records. (K7 ? 4 generated handoff proposals that clear the K5 gate)
- 390?- [x] Handoff/replay proposal ingest rejects unresolved `replay\_of`, `markdown\_path`, and `session\_id` references before candidate creation. (K5, K11)
- [x] Redaction preflight is deterministic, fail-closed for high-confidence secrets, covered by fixtures, and records a redaction report for every attempted bundle. (K9, K10)
- [x] `baseline replay select` and `baseline replay bundle` produce reviewed, redacted packets for self-contained non-coding sessions. (K8, K10)
- [x] `baseline replay ingest` accepts an external replay result and converts it into the existing proposal/candidate/calibration flow. (K11)
- 400?- [x] Efficacy gates prove no auto-promotion and no silent page rewrites. (K12: `G-no-autopromote`, `W1/W2` schema/marker/link gates). The **measured** replay-value signal (`R4-value`) stays **gated** until a real out-of-band replay result is ingested ? replay execution is out-of-band by design (D4), so the loop is built and proven safe/deterministic, and value measurement begins with the first real replay run.

11. References

Internal `[verified]`: `docs/BASELINE_LOOP_CLOSURE.md`, `docs/CALIBRATION EFFICACY.md`, `docs/COMPOSE_STACK.md`, `docs/BASELINE_WATCHLIST_TOMBSTONES.md`, `docs/archives/TECH_DEBT_PLAN.md`, `agent_sessions/baseline_agent.py`, `agent_sessions/baseline_ingest.py`, `agent_sessions/baseline_promote.py`, `agent_sessions/baseline_publish.py`, `baseline/proposals/proposal.schema.json`.

Issues `[verified]`: `#23`(<https://github.com/avidullu/agent-sessions/issues/23>), `#25`(<https://github.com/avidullu/agent-sessions/issues/25>), `#26`(<https://github.com/avidullu/agent-sessions/issues/26>), related `#19`(<https://github.com/avidullu/agent-sessions/issues/19>) and `#32`(<https://github.com/avidullu/agent-sessions/issues/32>).

External `[researched]`: `[LLM Wiki gist]`(<https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>), `[Reflexion]`(<https://arxiv.org/abs/2303.11366>), `[Generative Agents]`(<https://arxiv.org/abs/2304.03442>), `[Contextual Experience Replay]`(<https://arxiv.org/abs/2506.06698>), `[AgentRR]`(<https://arxiv.org/abs/2505.17716>), `[AgentHER]`(<https://arxiv.org/abs/2603.21357>), `[Helicone session replay]`(<https://docs.helicone.ai/guides/cookbooks/replay-session>), `[LangSmith trajectory evals]`(<https://docs.langchain.com/langsmith/trajectory-evals>), `[OpenAI agent evals]`(<https://developers.openai.com/api/docs/guides/agent-evals>), `[W3C PROV-DM]`(<https://www.w3.org/TR/prov-dm/>).

Changelog

- 2026-07-07 - Merged PR #56 for K11 and opened PR #57 for K12 efficacy gates: wired W1/W2 (schema/marker/link), H1/H2 (handoff precision/freshness), R1-R5 (replay), and `G-no-autopromote` into `baseline eval` with a `gated` status for prerequisite-pending gates. Real-repo run: 14 passed, 0 failed, 2 gated (R3/R4 pending a real replay result). **This completes the tracker ? all K rows (K0-K12) are Done; project status DONE.**
- 2026-07-07 - Merged PR #55 for K10 and opened PR #56 for K11 `baseline replay ingest`: validates external replay results (replay_of + trace refs resolve to `archive/index.jsonl`), emits `replay.\*` proposals that pass the K5 gate, and appends an append-only `baseline/replay/ledger.jsonl`.
- 2026-07-07 - Merged PR #54 for K9 and opened PR #55 for K10 `baseline replay bundle`: writes gitignored packets (redacted task + original deliverable + rubric + per-bundle redaction report) only for sessions that pass the K9 fail-closed gate.
- 420?- 2026-07-07 - Merged PR #53 for K8 and opened PR #54 for K9 replay redaction v0: deterministic fail-closed secret scanner + `baseline replay redact` preflight writing a valueless redaction report (gate R5-safety); egress remains gitignored.
- 2026-07-07 - Merged PR #52 for K7 and opened PR #53 for K8 `baseline replay select`: deterministic, excerpt-free replay manifest excluding coding sessions (D5), idempotent for gate

R2-dedup.

- 2026-07-07 - Wrapped the session with PR #52 open, CI green, and awaiting review; next resume should review/merge #52 before starting K8.
- 2026-07-07 - Opened PR #52 for K7 handoff-derived proposal generation and the #51 stable generated-date follow-up.
- 2026-07-07 - Merged PR #51 for K6, recorded its non-blocking review follow-ups, and started K7 handoff-derived proposal generation; K7 also keeps `handoffs.index` dates stable when feed content is unchanged.
- 2026-07-07 - Merged PR #50 for K5 and started K6 persistent handoff index/project-page feed work; K6 also folds in the small optional #49 invalid-date lint warning and #50 path-prefix normalization follow-ups.
- 2026-07-07 - Opened PR #51 for K6 persistent handoff index records and configured project-page feeds.
- 2026-07-07 - Merged PR #49 for K4 and started K5 trace reference validation.
- 2026-07-07 - Opened PR #50 for K5 proposal trace reference validation.
- 2026-07-07 - Opened PR #49 for K4 `baseline lint` skeleton.
- 430?- 2026-07-07 - Merged PR #48 for K3 and started K4 `baseline lint` skeleton.
- 2026-07-07 - Opened PR #48 for K3 shared project-page marker-block upsert helpers.
- 2026-07-07 - Merged PR #47 for K2 and started K3 project-page marker-block upsert helpers.
- 2026-07-07 - Opened PR #47 for K2 `baseline handoffs audit` report-only coverage/freshness work.
- 2026-07-07 - Merged PR #46 for K1 and started K2 `baseline handoffs audit` report-only coverage/freshness work.
- 2026-07-07 - Opened PR #46 for K1 by adding `baseline/SCHEMA.md`, wiring schema references into bundle packets, and separating K2 report-only handoff audit from K6 persistent handoff indexing.
- 2026-07-07 - Addressed consolidated PR #45 review by promoting replay redaction to a first-class deliverable, aligning trace fields with #19, adding ingest provenance-integrity validation, factoring shared provenance plumbing, allowing report-only handoff audit before schema writes, and defining replay value kill criteria.
- 2026-07-07 - Addressed PR #45 review by reconciling K rows with P5/P6/P10/P11, reusing shipped marker/upsert code, restoring replay R1-R5 gate names, and clarifying schema validation.
- 2026-07-07 - Opened draft PR #45 for the design tracker.
- 2026-07-07 - Created design tracker, indexed it in docs, updated the handoff pointer, and proposed execution sequence for #23, #25, and #26.

440?

14. tool_result

1?# Compose Stack

`agent-sessions` should stay narrow: durable multi-agent export, private git archive, and engineering baseline. Search, Claude-specific browsing, live capture, and runtime memory should be delegated to tools that already specialize in those jobs.

Layer Ownership

| 10? Layer | Owner | Why | |
|--|---|---|--|
| --- | --- | --- | |
| Retrospective multi-agent export to Markdown/PDF | `agent-sessions` | Custom Windows/WSL paths, niche sources, private git archive | |
| Engineering baseline extraction and promotion | `agent-sessions` | Differentiated local workflow; no mature equivalent | |
| Full-text and semantic search | | | |
| [cass](https://github.com/Dicklesworthstone/coding_agent_session_search) | Multi-provider local history index and search | | |
| Claude browsing/export UX | [claude-code-log](https://github.com/daaaain/claude-code-log) | Claude JSONL to readable HTML/Markdown, TUI-oriented workflow | |
| Raw Claude JSONL sync | [claude-code-sync](https://github.com/perfectrain/claude-code-sync) | Claude-specific cross-machine raw history sync | |
| Live terminal-agent capture | [SpecStory CLI](https://docs.specstory.com/quickstart) | Wrapper-style capture going forward | |
| Runtime memory injection | [agentmemory](https://github.com/rohitg00/agentmemory) | | |

MCP/hooks/REST memory layer for future sessions |

20?## Architecture

```
```mermaid
graph TD
 subgraph sources ["Local Agent Stores"]
 Codex["~/codex/sessions"]
 Claude["~/claude/projects"]
 Grok["~/grok/sessions"]
 Other["Gemini Antigravity / DeepSeek dumps"]
 end

 subgraph capture ["Optional Live Capture"]
 SpecStory["SpecStory CLI"]
 end

 subgraph archive ["agent-sessions"]
 Export["agent_archive.py export"]
 Index["archive/index.jsonl"]
 Markdown["archive/**/*md + PDFs"]
 Baseline["baseline suggest -> candidates -> promote"]
 end

 subgraph discover ["External Search"]
 CassIndex["cass index"]
 CassSearch["cass search / triage"]
 end

 subgraph claudeux ["Claude-Specific Tools"]
 CCL["claude-code-log"]
 CCS["claude-code-sync"]
 end

 subgraph memory ["Runtime Memory"]
 AgentMemory["agentmemory"]
 end

 sources --> Export
 SpecStory -->|future capture| sources
 Export --> Index
 Export --> Markdown
 Markdown --> Baseline
 Markdown --> CassIndex
 CassIndex --> CassSearch
 Claude --> CCL
 Claude --> CCS
 Baseline --> Agents["AGENTS / CLAUDE / VS Code generated slices"]
 AgentMemory -->|inject into future sessions| Agents
```
```

Daily Workflow

70?

1. Pull this repo on each machine.
2. Run the local export script from AUTOMATION.md.
3. Let cass index/search local stores or the exported archive separately.
4. Use claude-code-log when the task is specifically Claude browsing/export.
5. Run `baseline suggest` manually or weekly until promotion is mature.

Scope Boundaries

Build here:

80?

- Pluggable extractors in `agent\_sessions/sources/`.

- TOML source definitions and path templates.
- Markdown/PDF archive and `archive/index.jsonl`.
- Daily export automation for this repo.
- Baseline candidate, calibration, promotion, and generated agent views.
- Custom extractors for Grok, Gemini Antigravity, DeepSeek dumps, Codex, Claude, and other local stores that are not well-covered elsewhere.

Delegate externally:

90?

- Search/indexing to cass.
- Claude-specific TUI and detailed exports to claude-code-log.
- Raw Claude JSONL sync to claude-code-sync.
- Live capture to SpecStory when wrapping future terminal-agent sessions.
- Runtime memory injection to agentmemory.

Non-goals:

- Do not build a SQLite or semantic search engine inside this repo without a new explicit decision.
- Do not compete with Claude-specific UX tools.
- Do not replace runtime memory systems.
- Do not auto-promote baseline policy changes without review.
- Do not commit raw bundles or evidence excerpts unless explicitly forced.

Install Pointers

- cass: project README at [coding_agent_session_search](https://github.com/Dicklesworthstone/coding_agent_session_search)
- claude-code-log: project README at [claude-code-log](https://github.com/daaaain/claude-code-log)
- 110?- claude-code-sync: project README at [claude-code-sync](https://github.com/perfectrain/claude-code-sync)
- SpecStory CLI: [SpecStory quickstart](https://docs.specstory.com/quickstart)
- agentmemory: [agentmemory repo](https://github.com/rohitg00/agentmemory) and [project site](https://agent-memory.dev/)

External tools should be installed beside this repo, not vendored into it.

Related Work

- AUTOMATION.md
- ENGINEERING_BASELINE.md
- 120?- BASELINE_PLANNING.md

15. tool_result

17# Roadmap

This archive is local-first: importers should prefer session data stored on owned CPU/RAM and local disks, including Windows and WSL paths.

Search and live-memory features are intentionally delegated to the compose stack instead of rebuilt here; see COMPOSE_STACK.md.

Future Features

10?

- Daily Codex automation that pulls, exports new sessions, renders PDFs, commits, and pushes only when the archive changes. See AUTOMATION.md.
- Optional hosted/cloud session import adapters if a tool later moves chat history off-device. Prefer official APIs, explicit export folders, or vendor extension hooks over UI scraping.
- VS Code chat export inbox for any manually exported JSON or Markdown session files from machines that expose an official export command.

- Engineering baseline extraction that turns repeated session lessons into reviewable guardrails and agent-specific instruction files.
- 20? Agent-assisted proposal generation that uses bounded, reasonable access to sessions, repos, PRs, issues, CI logs, and local instruction files to draft reviewable baseline candidates.

16. tool_result

1?# Session Handoff

Updated: 2026-07-07 20:43 IST

You Are Here

`C:\Users\avidu\Projects\Agent Sessions` is a private GitHub-backed archive for agent coding sessions. The exporter is the `agent\_sessions` package, with TOML-backed source definitions and per-agent extractors for Codex, Claude, Grok, 10?DeepSeek, Gemini Antigravity, and known VS Code extension locations.

The active tracked project is `docs/BASELINE\_KNOWLEDGE\_REPLAY\_PLAN.md`, covering issues #23, #25, and #26 as one safety-first sequence:

1. schema and marker-owned knowledge pages;
2. deterministic handoff mining as the first producer;
3. replay selection, redaction, bundling, and ingest after provenance gates.

Today landed a long run of small PRs:

20?

- #45 K0: design tracker for #23/#25/#26.
- #46 K1: `baseline/SCHEMA.md` and scaffold/schema wiring.
- #47 K2: report-only `baseline handoffs audit`.
- #48 K3: shared project-page marker-block upsert helpers.
- #49 K4: read-only `baseline lint` skeleton.
- #50 K5: proposal trace propagation and archive-reference validation.
- #51 K6: persistent `baseline/handoffs/index.jsonl` plus `handoffs.index` project-page feeds for configured or existing project pages only.
- #52 K7: `baseline handoffs proposals` deterministic proposal JSON generation 30? with a hand-written-proposal overwrite guard and the #51 stable generated-date follow-up.
- #53 K8: `baseline replay select` deterministic, excerpt-free replay manifest excluding coding sessions (D5); idempotent for gate R2-dedup.
- #54 K9: `baseline replay redact` deterministic fail-closed secret scanner and valueless redaction report; egress remains gitignored.
- #55 K10: `baseline replay bundle` writes gitignored redacted packets (task + deliverable + rubric + report) only for sessions that pass the K9 gate.
- #56 K11: `baseline replay ingest` validates external replay results into `replay.\*` proposals (K5-gated) + an append-only `baseline/replay/ledger.jsonl`.
- 40?- #57 K12: efficacy gates (W1/W2, H1/H2, R1-R5, `G-no-autopromote`) wired into `baseline eval` with a `gated` status; real-repo run 14 pass / 0 fail / 2 gated.

Status: knowledge + replay tracker COMPLETE (K0-K12)

`docs/BASELINE\_KNOWLEDGE\_REPLAY\_PLAN.md` is `DONE` ? every K row landed via PRs #45-#57. The knowledge layer (SCHEMA, marker upserts, lint), handoff mining (audit -> index -> project feeds -> proposals), and the replay loop (select -> redact -> bundle -> ingest) are all implemented, deterministic, and human-gated. `baseline eval` now reports the E + W/H/R gate battery.

50?

The only pending signal is empirical, not code: `R3-signal`/`R4-value` stay `gated` until a real out-of-band replay result is ingested ? replay execution is out-of-band by design (D4). The first real replay run will flip those.

The 2026-07-08 audit is recorded in `docs/WORK\_AUDIT\_2026-07-08.md`; it marks the original baseline loop closure as historical/done and captures the current

follow-up set.

Next Steps / Open Threads

60?

- Optional: archive the tracker doc per its lifecycle once satisfied; run a real replay (hand a `baseline replay bundle` packet to a cross-lineage agent, then `baseline replay ingest` the result) to exercise R3/R4 with live data.
- Follow-up issues: #60 collects replay/knowledge v1 polish (richer replay slugs, clean bundle reruns, project-scoped replay proposals, stronger precision/dedup gates); #59 tracks entropy/vendor/JWT redaction hardening; #32 remains archive backfill/regenerate; #19 remains broader trace/explain work.

Known open boundaries:

70?

- #32 remains the right boundary for archive backfill/regenerate work and any future `baseline/handoffs/index.jsonl --prune` semantics.
- The two `baseline lint` orphan warnings for `baseline/projects/agent-sessions` and `baseline/projects/avidullu` are known warnings, not PR blockers.
- Replay execution stays out-of-band in v1. This repo selects and validates packets/results; it does not autonomously run alternate agents.

Ramp-Up Kit

80?- `docs/BASELINE\_KNOWLEDGE\_REPLAY\_PLAN.md`

- `docs/WORK\_AUDIT\_2026-07-08.md`
- `baseline/SCHEMA.md`
- `docs/ENGINEERING\_BASELINE.md`
- `docs/TEST\_PLAN.md`
- `docs/BASELINE\_LOOP\_CLOSURE.md`
- `docs/CALIBRATION\_EFFICACY.md`
- `docs/COMPOSE\_STACK.md`
- `baseline/handoffs/audit.md`
- `baseline/handoffs/index.jsonl`

90?- `baseline/proposals/handoff.\*.handoff-signals.json`

- `agent\_sessions/baseline\_handoffs.py`
- `agent\_sessions/baseline\_ingest.py`
- `agent\_sessions/baseline\_lint.py`
- `agent\_sessions/baseline\_promote.py`
- `agent\_sessions/cli.py`
- `tests/test\_baseline\_handoffs.py`
- `tests/test\_baseline\_ingest.py`
- `tests/test\_baseline\_lint.py`
- `tests/test\_cli.py`

100?

Key Decisions

- Private GitHub repo: `https://github.com/avidullu/agent-sessions`
- Raw source backups remain ignored by Git by default.
- Exported Markdown/PDF files under `archive/` are committed and pushed.
- Preserve transcript text as-is, even if generated Markdown contains trailing whitespace from original session content.
- Default source paths use template variables instead of user-specific hardcoded Windows/WSL paths.

110?- Knowledge/replay sequencing: schema and deterministic marker-block page writes come before handoff mining or replay egress.

- Handoff split: K2 audit is report-only; K6 owns persistent handoff index and project-page feeds; K7 owns proposal generation.
- Project-page producers reuse the shipped `baseline:begin/end` marker grammar; there is no second marker family and no free-form page rewrite path.
- Trace validation: human proposals may keep free-text evidence, but `source\_kind` of `replay`, `handoff`, or `repo-handoff` requires structured trace and resolvable archive references before candidate sidecars are written.
- K6 handoff index: persist every discovered handoff candidate in

120? `baseline/handoffs/index.jsonl`, but write project-page feeds only for

- configured or already-existing pages.
- K7 generated proposals: deterministic review inputs only, scoped to configured/existing projects, with `generated\_by = "baseline handoffs proposals"` and `source\_kind = "repo-handoff"`. They must pass `baseline ingest --dry-run` and are never auto-promoted.
- K7 stable-date follow-up: `handoffs.index` project-page feeds preserve the existing `generated\_at` value when generated content is unchanged.

17. tool_result

1?# Engineering Baseline

This directory holds reviewed engineering guardrails, project memory, and agent-specific generated views derived from archived sessions and repo evidence.

`SCHEMA.md` is the contract for derived baseline artifacts, marker ownership, trace fields, and validator behavior. Commands that write generated knowledge or replay/handoff outputs should follow that schema before adding new artifact types.

10?

Candidate files are suggestions. Promoted files are reviewed baseline guidance. Generated agent files should stay separate from hand-written project instruction files until their output proves useful.

18. tool_result

1?# Baseline Loop Closure

```
> **Status:** `DONE / HISTORICAL` - original closure proof complete; backlog rows split to
follow-up issues/designs. **Owner:** avidullu. **Created:** 2026-07-05. **Last updated:**
2026-07-08
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived`
> **Tracking anchors:** §7 records the original closure tracker;
`docs/BASELINE_KNOWLEDGE_REPLAY_PLAN.md` and `docs/WORK_AUDIT_2026-07-08.md` carry current
follow-up context.
> **Relation to existing docs:** implements the differentiated path in `docs/COMPOSE_STACK.md`;
closes PRs 5?7 in `docs/BASELINE_PLANNING.md`.
> **Honesty note:** `[verified]` items checked against repo code; backlog items are
intentionally not treated as active closure blockers.
```

10?

0. TL;DR

Close the engineering-baseline loop so `agent-sessions` turns multi-agent session evidence into **reviewed, versioned guardrails** ? not just candidate reports. This repo eats its own cooking: iteration is tracked here using the user's standard `PROJECT\_DOC\_TEMPLATE` pattern (sourced from `badminton-highlight-indexer`), and tool efficacy is measured in `docs/CALIBRATION\_EFFICACY.md`.

1. Problem & goal

20?

Problem: Archive + `baseline suggest` originally existed, but `baseline/global/` was empty and no agent-facing slices were published. Competitors search or capture sessions; none close the promote -> publish -> calibrate loop.

Goal: A measurable end-to-end proof that session history becomes durable engineering policy with human approval and provenance. That original proof is now complete: promoted guardrails exist, agent-facing generated slices exist, calibration feeds later suggestions, and `baseline eval --dry-run` reports E1-E6 30?passing.

Good looks like:

1. At least three guardrails promoted from archive evidence.
2. Generated Codex/Claude slices exist and are usable in a pilot repo.
3. Calibration feedback changes the next `baseline suggest` run.
4. The tool correctly identifies the user's tracked-project-doc standard from archive.

2. Decisions locked

40?

| # | Decision | Source / date | Implication |
|----|---|-------------------------|--|
| D1 | Search/memory delegated to compose stack | `docs/COMPOSE_STACK.md` | Do not build SQLite search here |
| D2 | Baseline promotion stays `strict` by default | `config/baseline.toml` | No auto-promote for policy |
| D3 | Standard project template = `badminton-highlight-indexer` | `PROJECT_DOC_TEMPLATE` | Archive evidence `[verified]` Use as calibration anchor + for this project's \$7 |
| D4 | Efficacy measured in `baseline/calibration/efficacy.toml` | This doc | Every phase has a pass/fail metric |
| D5 | Dogfood on `agent-sessions` first, then pilot repos | `config/baseline.toml` | pilots Prove loop before org scale |

3. Foundation ? archive evidence `[verified]`

50?

Identified via `agent\_archive.py` over `archive/`:

| Signal | Archive hits | Source repo evidence |
|---------------------------------|--|---|
| `PROJECT_DOC_TEMPLATE` | 237 sessions | `badminton-highlight-indexer/docs/PROJECT_DOC_TEMPLATE.md` |
| `session-handoff` / ramp-up kit | 943 sessions | `CLAUDE.md`, `SESSION_HANDOFF.md`, working-agreement skills |
| Tracked \$7 progress rows | Widespread in engine + khelsutra repos | `DECISION_SNAPSHOT_REGRESSION.md`, `CODE_CLEANUP_AUDIT_2026-06-24.md` |

Bundle command used to focus evidence:

60?

```

```powershell
python .\tools\agent_archive.py baseline bundle --focus badminton-highlight-indexer --focus PROJECT_DOC_TEMPLATE --focus session-handoff
```

```

4. Design / architecture

```

```mermaid
graph LR
 Archive["archive/ + index.jsonl"] --> Suggest["baseline suggest"]
 Suggest --> Candidates["baseline/candidates/"]
 Candidates --> Calibrate["baseline calibrate"]
 Calibrate --> Promote["baseline promote"]
 Promote --> Global["baseline/global/ + projects/"]
 Global --> Publish["baseline publish"]
 Publish --> Agents["AGENTS.generated / CLAUDE.generated"]
 Feedback["feedback.toml + efficacy.toml"] -.-> Suggest
 Anchor["calibration_anchors"] -.-> Suggest
```

```

80?Reuse map:

- `agent\_sessions/baseline.py` ? suggest, calibrate, future promote/publish
- `agent\_sessions/baseline\_agent.py` ? evidence bundles
- `config/baseline.toml` ? pilots + calibration anchors
- `baseline/calibration/efficacy.toml` ? measurable closure metrics

5. Threat model / risk table

```
| Risk | Mitigation |
90?|-----|-----|
Promoting secrets from transcripts	Redaction rules in `docs/ENGINEERING_BASELINE.md`; manual promote
Template detection false positive	Calibration anchor + user feedback on `guardrail.tracked-project-docs`
Scope creep into search/TUI	`docs/COMPOSE_STACK.md` scope boundaries
Efficacy metrics gaming	Metrics require promoted files + generated slices, not keyword counts alone
```

6. Honest limits ? what this does NOT do

- Does not replace cass for search or agentmemory for runtime memory.
- Does not auto-apply generated slices into project `AGENTS.md` until trust is earned.
- 100?- Does not prove org-scale onboarding (PR 7) until personal pilot passes efficacy gates.

19. tool_result

17# Baseline Derived Layer Schema

```
> **Status:** `v1 draft`.
> **Owner:** human-reviewed docs PRs.
> **Scope:** files under `baseline/` that are derived from archives, repo
> handoffs, replay packets, proposals, candidates, calibration, or promotion.
```

1. Purpose

10?The baseline layer converts immutable archive and repo evidence into reviewed, versioned guidance. This schema defines the artifact types, ownership rules, marker grammar, provenance fields, and validator contracts that producers must follow before they write generated knowledge.

The schema is intentionally markdown-first. It is the human-readable contract; command-specific validators and future `baseline lint` checks enforce it for new generated producers. `baseline/proposals/proposal.schema.json` remains an illustrative proposal example until proposal validation is formalized in code.

20?## 2. Artifact Types

```
Type	Path	Ownership	Notes
Baseline root	`baseline/README.md`, `baseline/SCHEMA.md`	human	Entry point and contract.
Global guardrails	`baseline/global/*.md`	human prose + marker blocks	Reviewed guidance promoted from candidates.
Agent views	`baseline/agents/**/*.generated.md`	generated	Derived views for specific agents; do not overwrite hand-authored repo instructions.
Project pages	`baseline/projects/<slug>/README.md`	human prose + marker blocks	Durable project knowledge and generated sections.
Candidate reports	`baseline/candidates/*.md`	generated, reviewed by humans	Human-readable reports with sidecars.
Candidate sidecars	`baseline/candidates/*.predictions.json`	generated	Structured prediction payloads and trace data.
30?	Proposal inputs	`baseline/proposals/*.json`	human or bounded producer
Handoff audit	`baseline/handoffs/audit.md`	generated report	K2 audit output only; no pages, proposals, or index writes.
Handoff index	`baseline/handoffs/index.jsonl`	generated records	K6 persistent normalized handoff records; project-page feeds are marker blocks on configured or existing project pages only.
Replay manifest	`baseline/replay/manifest.jsonl`	generated records	Deterministic selected session refs and exclusion reasons, no excerpts.
Replay bundles	`baseline/replay/bundles/*`	generated egress	Gitignored packets that may
```

contain excerpts after redaction preflight. |
| Replay ledger | `baseline/replay/ledger.jsonl` | append-only generated records | Replay-result
history after validated ingest. |
| Prediction/feedback ledgers | `baseline/metacognition/\*.jsonl` | append-only generated records
| Prediction and feedback history. |
| Calibration inputs/reports | `baseline/calibration/\*.toml`, `baseline/calibration/\*.md` |
human + generated reports | Efficacy gates and calibration feedback. |

3. Ownership Rules

40?

- Raw archive data remains immutable input. Baseline producers derive from it; they do not rewrite it.
- Human-authored prose outside generated markers is preserved.
- Generated markdown updates are limited to marker-owned blocks or explicitly generated files.
- Append-only ledgers are rewritten only through atomic upsert semantics that preserve unrelated runs.
- Candidate and proposal artifacts are review inputs. They do not become policy until promotion writes reviewed guidance.

50?- Handoff audit is report-only. Persistent handoff indexing writes

- `baseline/handoffs/index.jsonl` for all discovered records, while project-page feeds update only configured or already-scaffolded project pages.
- Generated handoff proposals may overwrite only proposal files already marked with `generated\_by = "baseline handoffs proposals"`; hand-written proposals are never replaced by that producer.
- Replay bundle egress is blocked unless deterministic redaction passes and bundle paths are gitignored.

4. Marker Grammar

60?

Generated markdown blocks use the existing promotion marker grammar:

```
```markdown
<!-- baseline:begin id="stable.block.id" -->
Generated content.
<!-- baseline:end id="stable.block.id" -->
```
```

Rules:

70?

- Begin and end markers must use the same `id`.
 - IDs are stable dotted or dashed identifiers scoped to the target page.
 - Producers must preserve all content outside owned marker blocks.
 - Producers must not introduce a second marker family for knowledge pages.
 - Project-page producers should render sections with `render\_project\_page\_block()` and write them through `upsert\_project\_page\_content()`, which reuses the same marker parser/upsert path as promoted guardrails.
 - Project-page upserts may remove the scaffold placeholder only when the line exactly matches the known placeholder text; edited placeholder-like prose is
- 80? human-owned content.
- Producers should preserve an existing `generated\_at` value when a marker block's substantive generated content is unchanged, so periodic runs converge to quiet diffs.
 - Metadata such as `generated\_by`, `generated\_at`, and `proposal\_id` belongs in the block body or in structured sidecars unless the shared parser changes.

5. Trace Fields

Generated proposals, candidate sidecars, handoff records, replay records, and 90?future project-page blocks should use this #19-aligned trace vocabulary when a field is available:

| Field | Meaning |
|-------|---------|
| --- | --- |

| `source` | Archive source or producer name, for example `codex`, `baseline handoffs audit`, or `baseline handoffs proposals`. |
 | `source\_file` | Original raw source file or repo handoff path when known. |
 | `markdown\_path` | Repo-relative markdown artifact, usually under `archive/`. |
 | `session\_id` | Session identifier from record metadata when available. |
 | `timestamp` | Observation or source timestamp in ISO 8601 form when available. |
 100?| `project\_slug` | Deterministic project identifier used under `baseline/projects/`. |
 | `repo` | Repository URL or canonical repo id when known. |
 | `evidence\_anchor` | Heading, marker id, excerpt id, or other stable local anchor. |
 | `evidence\_excerpt` | Short evidence quote or summary, after redaction rules. |
 | `transform` | Deterministic transform that produced the record. |
 | `bundle\_id` | Replay or agent bundle identifier when the evidence came from a bundle. |
 | `calibration\_effect` | Optional feedback/calibration note, for example accepted, edited, or rejected. |

Structured proposal fields:

110?- `source\_kind` is optional for human-authored proposals; replay and handoff producers should use `replay`, `handoff`, or `repo-handoff`.
 - K7 handoff-derived proposal files use
 `generated\_by = "baseline handoffs proposals"` and `source\_kind = "repo-handoff"`; they are review inputs, not promotion decisions.
 - `replay\_of`, when present, is the original archive session id and must resolve against `archive/index.jsonl`.

Resolution rules:

120?- `markdown\_path` is repo-relative and uses forward slashes.
 - `session\_id` and `markdown\_path` references from replay or handoff producers must resolve against `archive/index.jsonl` before candidate creation.
 - Hand-written proposals may continue to use free-text `evidence` lists, but external replay and handoff producers must include structured trace records before ingest promotes them into candidates.
 - Trace records should be copied forward into prediction sidecars so later calibration and promotion can cite their derivation.

6. Project Identity

130?

Project page paths use `baseline/projects/<slug>/README.md`.

Slug derivation priority:

1. Configured `config/baseline.toml` pilot slugs and aliases.
2. Canonical repository URL when known.
3. Decoded `metadata.project` or `metadata.cwd` basename.
4. A stable disambiguator when names collide.

140?K6/K8 implementations must include fixture coverage for Windows paths with case differences, spaces, URL-encoded drive paths, hyphen-encoded Claude project paths, and duplicate basenames so two unknown projects do not collapse into one page or handoff record. Configured pilot aliases may intentionally collapse multiple raw paths into one canonical project slug.

7. Validation Contract

`baseline lint` is the schema-wide validator for deterministic checks that can run before generated project-page feeds exist. It returns non-zero for errors; 150?warnings are review signals that later K rows can make stricter as producers start writing richer artifacts.

Required validator behavior:

- Reject malformed marker pairs and duplicate generated block ids in a page.
- Fail generated writes that would alter human-owned prose.

- Fail on broken links inside generated marker blocks.
- Warn on project pages that have neither inbound baseline links nor generated blocks; K6 `handoffs.index` feeds satisfy the generated-block side of that 160? check.
- Warn on stale generated blocks by age and malformed generated dates; later producers should add source-record freshness checks.
- Warn on explicit contradiction markers; deeper P6 duplicate/contradiction analysis can harden this rule.
- Reject replay or handoff proposal ingest when required trace references do not resolve.
- Refuse generated handoff proposal overwrites unless the target is already owned by the same handoff proposal producer.
- Fail replay bundle writes when redaction detects high-confidence secrets. 170?

8. Non-Goals

- No embeddings or semantic search in v1.
- No automatic promotion from handoff, replay, or candidate artifacts.
- No free-form LLM rewrite of project pages.
- No coding-session replay until workspace snapshots or commit/worktree provenance exists.

9. Changelog

180?

- 2026-07-07: Added K12 efficacy gates (W1/W2 schema+link, H1/H2 handoff, R1-R5 replay, `G-no-autopromote`) to `baseline eval`, with a `gated` status for prerequisite-pending gates. Completes the knowledge/replay tracker (K0-K12).
- 2026-07-07: Added K11 `baseline replay ingest`: validates external replay results, emits `replay.\*` proposals that clear the K5 gate, and appends the append-only `baseline/replay/ledger.jsonl`.
- 2026-07-07: Added K10 `baseline replay bundle`: gitignored replay packets (redacted task + deliverable + rubric + per-bundle redaction report) written only for sessions that pass the fail-closed redaction gate.
- 2026-07-07: Added K9 replay redaction v0: deterministic fail-closed secret scanner and `redaction-report.json` (values never recorded); gitignored replay egress.
- 2026-07-07: Added K8 `baseline replay select` deterministic, excerpt-free replay manifest (`baseline/replay/manifest.jsonl`) excluding coding sessions.
- 2026-07-07: Added K7 generated handoff proposal ownership rules and stable project-page generated dates for unchanged feeds.
- 2026-07-07: Added K6 persistent handoff index records and configured project-page `handoffs.index` feeds.
- 2026-07-07: Added K5 proposal trace propagation and ingest reference validation.
- 2026-07-07: Added K4 `baseline lint` severity model and first validator scope.
- 190?- 2026-07-07: Added K3 project-page marker-block helper contract.
- 2026-07-07: Initial schema for K1 of the baseline knowledge/replay tracker.

20. tool_result

1?""Command-line interface for the archive tool.""

```
from __future__ import annotations
```

```
import argparse
```

```
from pathlib import Path
```

```
from .archive import ExportResult, discover_sources, export_sources, pdf_existing,
prune_index_records
```

```
from .archive_status import archive_status
```

```
10?from .baseline import baseline_scaffold, baseline_suggest
```

```
from .config import ArchiveConfig, load_config
```

```
def default_repo_root() -> Path:
    return Path.cwd()
```

```

def _handle_discover(config: ArchiveConfig, args: argparse.Namespace) -> int:
    return discover_sources(config, samples=args.samples, write=args.write)
20?

def _export_summary_lines(
    result: ExportResult,
    *,
    write_pdfs: bool,
    copy_raw_files: bool,
    dry_run: bool,
) -> list[str]:
    lines = [f"Exported {result.exported} session files."]
30?    if result.skipped_sources:
        lines.append("Skipped sources without extractors:")
        lines.extend(f"- {source}" for source in result.skipped_sources)
        if any("(inventory)" in source for source in result.skipped_sources):
            lines.append("Inventory-only sources are expected until transcript files are
available.")
        if result.pdf_missing:
            lines.append("PDF export requested but reportlab is not installed. Run: python -m pip
install reportlab")

        lines.extend(["", "Next steps:"])
        if dry_run:
40?            lines.append("- Dry run only: no archive files were written.")
        else:
            lines.append("- Review `archive/INDEX.md` and `archive/index.jsonl`.")
            if write_pdfs:
                lines.append("- PDFs are written beside Markdown files when `reportlab` is
available.")
            else:
                lines.append("- Markdown files are under `archive/`; rerun with `--pdf` for optional
PDFs.")
            if copy_raw_files:
                lines.append("- Raw backups, if written, are under ignored `raw/`.")
                lines.append("- Check intended changes with `git status --short archive/
docs/DISCOVERY.md`.")
50?            lines.append("- Stage explicit paths only; keep `sources.toml`, `raw/`, and unrelated
files out.")
            lines.append("- For baseline review, run `python tools/agent_archive.py baseline
scaffold` and `baseline suggest`.")
        return lines

def _handle_export(config: ArchiveConfig, args: argparse.Namespace) -> int:
    if not args.all and not args.source:
        raise SystemExit("export requires --all or at least one --source")
    result = export_sources(
        config,
60?        selected=args.source,
        limit=args.limit,
        write_pdfs=args.pdf,
        copy_raw_files=args.copy_raw,
        dry_run=args.dry_run,
    )
    print(
        "\n".join(
            _export_summary_lines(
                result,
70?                write_pdfs=args.pdf,
                copy_raw_files=args.copy_raw,
                dry_run=args.dry_run,
            )
        )
    )

```

```

return 0

def _handle_pdf(config: ArchiveConfig, args: argparse.Namespace) -> int:
80?     if not args.all and not args.source:
            raise SystemExit("pdf requires --all or at least one --source")
        return pdf_existing(config, selected=args.source, limit=args.limit, force=args.force)

def _handle_status(config: ArchiveConfig, args: argparse.Namespace) -> int:
        return archive_status(config, selected=args.source, as_json=args.json)

def _handle_prune(config: ArchiveConfig, args: argparse.Namespace) -> int:
90?     return prune_index_records(config, dry_run=args.dry_run)

def _handle_baseline_scaffold(config: ArchiveConfig, args: argparse.Namespace) -> int:
        return baseline_scaffold(config, dry_run=args.dry_run)

def _handle_baseline_suggest(config: ArchiveConfig, args: argparse.Namespace) -> int:
        return baseline_suggest(
            config,
100?         output=args.output,

```

21. tool_result

1?# Engineering Baseline Planning

Status: draft
Date: 2026-07-05

North Star

Build a portable engineering baseline system that learns from agent sessions, repositories, reviews, CI, docs, and user corrections, then proposes durable 10?guardrails and project knowledge for future work.

The system should support personal repos first, then scale to teams, organizations, and client onboarding. A future user should be able to point it at a new codebase or organization and get a reviewable onboarding plan, a map of important project knowledge, and candidate guardrails for agent-assisted work.

Pilot Scope

Start with a small but representative corpus:

- 20?
- `badminton-highlight-indexer`: mothership project and primary source of tribal knowledge.
 - `muneem`: newer project with useful upserts from newer agents and workflows.
 - Telegram-related repo: workflow and integration-heavy pilot.
 - `avidullu` GitHub account: broader personal repo inventory and cross-project patterns.
 - `KhelSutra` GitHub organization: organization-style pilot and future team sharing candidate.

30?The first implementation should not assume these names are permanent. They are seed candidates for configuration and evaluation.

Product Shape

The baseline has four layers:

1. Evidence: archived sessions, repo docs, code, PRs, issues, CI logs, and local agent instruction files. Search/discovery can use the compose stack, but the archive remains the durable local evidence layer.
2. Candidates: proposed rules, patterns, project facts, and onboarding notes with provenance.
3. Promoted baseline: reviewed guidance accepted into global or project-specific baseline files.
4. Published agent views: generated Codex, Claude, and VS Code-supportive files kept separate from hand-written project instructions until the system earns enough trust.

Suggest Interface

Use Markdown first because it is reviewable in GitHub PRs, easy to edit, and works with any agent.

Suggested layout:

```

```text
baseline/
 candidates/
 2026-07-05-extraction.md
 global/
 engineering-guardrails.md
 repo-workflows.md
 regression-frameworks.md
 prompt-patterns.md
 projects/
 badminton-highlight-indexer/
 overview.md
 decisions.md
 pitfalls.md
 test-and-release.md
 muneem/
 telegram/
 agents/
 codex/AGENTS.generated.md
 claude/CLAUDE.generated.md
 vscode/copilot-instructions.generated.md
```

```

Candidate entries should be structured enough to promote later:

```

```markdown

```

## 22. tool\_result

```

"""Baseline orchestration: scaffold, suggest, and calibrate commands.

```

The baseline subsystem is split into focused modules; this one keeps the top-level commands and the prediction-artifact IO, and re-exports the public API so existing `from agent_sessions.baseline import ...` call sites keep working:

```

- ``baseline_types`` ? shared dataclasses + pure helpers (cycle-free root)
- ``baseline_settings`` ? settings loading, scaffold templates, feedback IO
- ``baseline_predictions`` ? keyword scanning + deterministic predictions
- ``baseline_report`` ? candidate/calibration Markdown rendering
- ``baseline_promote`` ? marker blocks + promotion of accepted guardrails
- ``baseline_calibration`` ? feedback/ledger calibration loop
"""

```

```

from __future__ import annotations

```



```

import datetime as dt
import json
20?import os
from collections import Counter
from dataclasses import replace
from pathlib import Path
from typing import Any, cast

from .baseline_calibration import apply_calibration_loop, load_ledger_entries
from .baseline_predictions import (
 KEYWORD_GROUPS,
 PR_ONLY_REPO_WRITES_TEXT,
30? build_predictions,
 keyword_hits,
 project_evidence,
 project_signal_counts,
 scan_text_signals,
 signal_confidence,
 signal_evidence,
 tracked_project_doc_confidence,
 tracked_project_doc_evidence,
)
40?from .baseline_promote import (
 PROMOTED_PLACEHOLDER,
 PROMOTION_BEGIN,
 PROMOTION_END,
 baseline_promote,
 category_promotion_target,
 global_baseline_header,
 parse_promoted_blocks,
 project_page_header,
 project_page_placeholder,
50? promote_predictions,
 render_project_page_block,
 render_promoted_block,
 select_promotable_predictions,
 upsert_project_page_content,
 upsert_promoted_content,
)
from .baseline_report import (
 render_calibration_summary,
 render_candidate_report,
60? render_id_list,
 render_prediction,
)
from .baseline_settings import (
 BASELINE_CONFIG,
 BASELINE_ROOT,
 agent_readme,
 baseline_files,
 baseline_readme,
 baseline_schema,
70? candidates_readme,
 feedback_example,
 load_baseline_settings,
 load_feedback,
 metacognition_readme,
 project_readme,
 resolve_prediction_sidecar,
)
from .baseline_types import (
 BaselineSettings,
80? Pilot,

```

## 23. tool\_result

## 1?# Calibration Efficacy

How to measure whether `agent-sessions` is working ? not just archiving sessions, but learning the user's engineering patterns and closing the baseline loop.

Machine-readable metrics live in `baseline/calibration/efficacy.toml`. Update status there after each review cycle.

### Why a calibration anchor?

10?

Archive analysis identified a **repeatable standard** the user already applies in production repos:

```
| Anchor | Location | Archive evidence |
|-----|-----|-----|
| Tracked project doc template | `badminton-highlight-indexer/docs/PROJECT_DOC_TEMPLATE.md` |
237 sessions mention `PROJECT_DOC_TEMPLATE` |
| Session handoff + ramp-up kit | `SESSION_HANDOFF.md`, `session-handoff` skills | 943 sessions
mention handoff/ramp-up |
| $7 progress tracker discipline | Precedents in engine + khelsutra repos | Widespread in
archived Claude/Codex sessions |
```

20?The tool proves efficacy when it can **detect ? propose ? promote ? apply** this pattern ? starting with dogfooding on `docs/BASELINE\_LOOP\_CLOSURE.md` in this repo.

### Efficacy gates

```
| ID | Gate | Pass condition | Measures |
|----|-----|-----|-----|
| E1 | Detect | `baseline suggest` emits `guardrail.tracked-project-docs` with evidence
citing archive hits | Tool finds user's template pattern without being told the path |
| E2 | Anchor | `config/baseline.toml` `calibration_anchors` lists `badminton-highlight-
indexer` template | Explicit link between archive inference and known ground truth |
| E3 | Dogfood | `docs/BASELINE_LOOP_CLOSURE.md` exists as a filled tracked project doc |
Tool's iteration uses the user's own template |
30?| E4 | Promote | ?1 rule in `baseline/global/` is non-placeholder promoted text | Loop
produces durable policy, not just candidates |
| E5 | Publish | `baseline/agents/claude/CLAUDE.generated.md` (or codex equivalent) exists |
Agents can consume output |
| E6 | Calibrate | Second `baseline suggest` after `feedback.toml` shows adjusted confidence
or suppressed rejected IDs | Feedback changes behavior |
```

### Review cadence

1. Run `baseline suggest` and check E1.
2. Copy `feedback.example.toml` ? `feedback.toml`; mark `guardrail.tracked-project-docs` and top guardrails.
3. Run `baseline calibrate --feedback baseline/calibration/feedback.toml`.
4. After promote/publish land, re-run suggest and update `efficacy.toml` statuses.
- 40?5. When E1?E6 pass, mark `docs/BASELINE\_LOOP\_CLOSURE.md` \$7 P9 ?.

### Commands

```
```powershell
python .\tools\agent_archive.py baseline suggest
python .\tools\agent_archive.py baseline calibrate --feedback baseline\calibration\feedback.toml
python .\tools\agent_archive.py baseline bundle --focus PROJECT_DOC_TEMPLATE --focus badminton-
highlight-indexer
```
```

50?## Related

- Tracked project: `docs/BASELINE\_LOOP\_CLOSURE.md`

- Template: `docs/PROJECT\_DOC\_TEMPLATE.md`
- Metrics file: `baseline/calibration/efficacy.toml`
- Compose scope: `docs/COMPOSE\_STACK.md`

## 24. tool\_result

```
1?"""Extractor registry keyed by source kind."""

from __future__ import annotations

from collections.abc import Callable
from pathlib import Path

from ..models import ExtractedSession

10?Extractor = Callable[[Path], ExtractedSession]

_EXTRACTORS: dict[str, Extractor] = {}

def register(kind: str) -> Callable[[Extractor], Extractor]:
 def decorator(extractor: Extractor) -> Extractor:
 _EXTRACTORS[kind] = extractor
 return extractor

 20? return decorator

def get_extractor(kind: str) -> Extractor | None:
 import agent_sessions.sources # noqa: F401

 return _EXTRACTORS.get(kind)

def known_kinds() -> tuple[str, ...]:
 30? import agent_sessions.sources # noqa: F401

 return tuple(sorted(_EXTRACTORS))
```

## 25. tool\_result

```
1?"""Shared archive data models."""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Any

10?@dataclass(frozen=True)
class Source:
 name: str
 kind: str
 roots: tuple[Path, ...]
 glob: str = "**/*"
 description: str = ""

@dataclass(frozen=True)
20?class SessionMessage:
 role: str
 text: str
 timestamp: str = ""
```

```
@dataclass(frozen=True)
class ExtractedSession:
 metadata: dict[str, Any]
 messages: list[SessionMessage]
30?
```

## 26. tool\_result

```
- /home/avidullu/projects/Agent/agent-sessions/agent_sessions/sources/
- __init__.py
- claude.py
- codex.py
- deepseek.py
- gemini.py
- grok.py
- registry.py
```

## 27. tool\_result

1?# Engineering Guardrails

Promoted guidance derived from reviewed baseline candidates.

```
<!-- baseline:begin id="guardrail.handoff-and-resume" -->
```

### Handoff And Resume Discipline

```
ID: `guardrail.handoff-and-resume`
Promoted: 2026-07-05
10? **Run:** 2026-07-05-extraction
Confidence: 0.77
Review note: Promote first ? matches SESSION_HANDOFF pattern across pilots.
```

Projects should maintain a short session handoff or start-here file so future agents can resume from real state instead of replaying old conversation.

### Evidence

```
- 151 scanned sessions contain `checkpointing` signals.
- archive/claude-windows/20260701-45ea3a91-654d-4972-9cd3-65f35db54caf-45ea3a91-654d-4972-9cd3-65f35db54caf-9f13f1f.md (192 keyword hits)
- archive/claude-windows/20260622-985cc286-9e1e-4217-87c3-2d418bc4fa9c-985cc286-9e1e-4217-87c3-2d418bc4fa9c-090b1b8.md (173 keyword hits)
20?- archive/claude-windows/20260618-6a82308f-69d6-43a8-9ea4-d8eaf497253d-6a82308f-69d6-43a8-9ea4-d8eaf497253d-d3b76c6.md (171 keyword hits)
- archive/codex-windows/20260702-019f2174-3bdf-73b3-9285-a8db45326c90-rollout-2026-07-02t11-41-33-019f2174-3bdf-73.md (140 keyword hits)
<!-- baseline:end id="guardrail.handoff-and-resume" -->
```

```
<!-- baseline:begin id="guardrail.pr-only-repo-writes" -->
```

### PR-Only Repo Writes

```
ID: `guardrail.pr-only-repo-writes`
Promoted: 2026-07-05
Run: 2026-07-05-extraction
30? **Confidence:** 0.93
Review note: Promote first ? high confidence, strong archive evidence.
```

Agents must not push directly to durable/shared repos. They should branch from the remote base, stage explicit paths, open a PR, and merge only after explicit approval or a scoped umbrella approval. Scoped approval is limited to the named project, PR set, task, and time/context in which it was granted; it must not be reused for adjacent work, self-authored PRs, or later PRs without renewed confirmation.

### Evidence

```
- 333 scanned sessions contain `repo-governance` signals.
- archive/codex-windows/20260702-019f2174-3a4a-7753-9b64-aca102f804-rollout-2026-07-02t11-41-33-019f2174-3a4a-77.md (1118 keyword hits)
- archive/codex-windows/20260702-019f2174-3bdf-73b3-9285-a8db45326c90-rollout-2026-07-02t11-41-33-019f2174-3bdf-73.md (1072 keyword hits)
- archive/claude-windows/20260618-6a82308f-69d6-43a8-9ea4-d8eaf497253d-6a82308f-69d6-43a8-9ea4-d8eaf497253d-d3b76c6.md (820 keyword hits)
40?- archive/claude-
windows/20260703-7fce5410-4c57-4ec9-a054-be898bda22da-7fce5410-4c57-4ec9-a054-be898bda22da-
bd42d4f.md (810 keyword hits)
<!-- baseline:end id="guardrail.pr-only-repo-writes" -->
```

```
<!-- baseline:begin id="guardrail.tracked-project-docs" -->
```

## Tracked Project Docs For Substantial Work

```
ID: `guardrail.tracked-project-docs`
Promoted: 2026-07-05
Run: 2026-07-05-extraction
Confidence: 0.99
50? **Review note:** Matches badminton-highlight-indexer PROJECT_DOC_TEMPLATE; use for
substantial multi-PR work.
```

### 28. tool\_result

```
1?#!/usr/bin/env python3
"""Command wrapper for the agent session archive."""
```

```
from __future__ import annotations
```

```
import sys
from pathlib import Path
```

```
REPO_ROOT = Path(__file__).resolve().parents[1]
10?if str(REPO_ROOT) not in sys.path:
 sys.path.insert(0, str(REPO_ROOT))
```

```
from agent_sessions.cli import main # noqa: E402
```

```
if __name__ == "__main__":
 raise SystemExit(main())
```

### 29. tool\_result

```
1?"""Claude Code session extraction."""
```

```
from __future__ import annotations
```

```
from pathlib import Path
from typing import Any
```

```
from ..models import ExtractedSession, SessionMessage
from ..utils import jsonl_objects, text_from_content
10?from .registry import register
```

```
@register("claude")
```

```
def extract(path: Path) -> ExtractedSession:
 metadata: dict[str, Any] = {"session_id": path.stem, "project": path.parent.name}
 messages: list[SessionMessage] = []
 for obj in jsonl_objects(path):
 if obj.get("sessionId"):
 metadata["session_id"] = obj.get("sessionId")
20? message = obj.get("message")
```

```

 if isinstance(message, dict):
 role = message.get("role") or obj.get("type") or "message"
 content = text_from_content(message.get("content"))
 if content:
 messages.append(SessionMessage(role=role, text=content,
timestamp=obj.get("timestamp", "")))
 continue
 if obj.get("type") == "summary" and obj.get("content"):
 messages.append(
 SessionMessage(role="summary", text=text_from_content(obj.get("content")),
timestamp=obj.get("timestamp", ""))
)
 30?
 return ExtractedSession(metadata=metadata, messages=messages)

```

### 30. tool\_result

```

1?{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\session
s\\2026\\07\\02\\rollout-2026-07-02T11-41-32-019f2174-38b2-7d92-bff9-9b57ab7306e6.jsonl",
"sha256": "15390945c8c4d53627b652eeac6761f25f5f547d8f2e08951bdc5e387c235d1e", "messages": 72,
"markdown": "archive\\codex-windows\\20260702-019f2174-38b2-7d92-bff9-9b57ab7306e6-rollout-2026-
07-02t11-41-32-019f2174-38b2-7d.md", "pdf": "archive\\codex-windows\\20260702-019f2174-38b2-
7d92-bff9-9b57ab7306e6-rollout-2026-07-02t11-41-32-019f2174-38b2-7d.pdf", "raw": null,
"metadata": {"session_id": "019f2174-38b2-7d92-bff9-9b57ab7306e6", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-32-019f2174-38b3-7260-a032-587c94d92d34.jsonl",
"sha256": "d8871cd248166e9b3f16f59849aabb2967f3b3cfd61e21b56439b2d0588f32f", "messages": 148,
"markdown": "archive\\codex-windows\\20260702-019f2174-38b3-7260-a032-587c94d92d34-rollout-2026-
07-02t11-41-32-019f2174-38b3-72.md", "pdf": "archive\\codex-windows\\20260702-019f2174-38b3-
7260-a032-587c94d92d34-rollout-2026-07-02t11-41-32-019f2174-38b3-72.pdf", "raw": null,
"metadata": {"session_id": "019f2174-38b3-7260-a032-587c94d92d34", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-32-019f2174-38dd-7920-9ff4-c09b81ac2ed8.jsonl",
"sha256": "3564e2f6ece49faed920263a58aeada5ed34bdd475e8859bd9d9ac4bef6638ad", "messages": 870,
"markdown": "archive\\codex-windows\\20260702-019f2174-38dd-7920-9ff4-c09b81ac2ed8-rollout-2026-
07-02t11-41-32-019f2174-38dd-79.md", "pdf": "archive\\codex-windows\\20260702-019f2174-38dd-
7920-9ff4-c09b81ac2ed8-rollout-2026-07-02t11-41-32-019f2174-38dd-79.pdf", "raw": null,
"metadata": {"session_id": "019f2174-38dd-7920-9ff4-c09b81ac2ed8", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-32-019f2174-38e5-7980-9505-83230428c846.jsonl",
"sha256": "3220ad5650a8c277e5451d1b75febac0274a79e74bd2555f249998c511993d3b", "messages": 1031,
"markdown": "archive\\codex-windows\\20260702-019f2174-38e5-7980-9505-83230428c846-rollout-2026-
07-02t11-41-32-019f2174-38e5-79.md", "pdf": "archive\\codex-windows\\20260702-019f2174-38e5-
7980-9505-83230428c846-rollout-2026-07-02t11-41-32-019f2174-38e5-79.pdf", "raw": null,
"metadata": {"session_id": "019f2174-38e5-7980-9505-83230428c846", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-32-019f2174-390c-7ce2-8083-44c35ac32ea0.jsonl",
"sha256": "cfe0d698a0780c4ba38c851b4b04e8850c87da0fdb9c50e4120c3044cab70026", "messages": 979,
"markdown": "archive\\codex-windows\\20260702-019f2174-390c-7ce2-8083-44c35ac32ea0-rollout-2026-
07-02t11-41-32-019f2174-390c-7c.md", "pdf": "archive\\codex-windows\\20260702-019f2174-390c-
7ce2-8083-44c35ac32ea0-rollout-2026-07-02t11-41-32-019f2174-390c-7c.pdf", "raw": null,
"metadata": {"session_id": "019f2174-390c-7ce2-8083-44c35ac32ea0", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-32-019f2174-3937-7a93-8a9d-3b54c2d0628a.jsonl",
"sha256": "3d73b2f8e26e94386a02f77799ea0319532130a0fbf6dc75ae9c7326e22df95b", "messages": 2637,
"markdown": "archive\\codex-windows\\20260702-019f2174-3937-7a93-8a9d-3b54c2d0628a-rollout-2026-

```

```
07-02t11-41-32-019f2174-3937-7a.md", "pdf": "archive\\codex-windows\\20260702-019f2174-3937-7a93-8a9d-3b54c2d0628a-rollout-2026-07-02t11-41-32-019f2174-3937-7a.pdf", "raw": null,
"metadata": {"session_id": "019f2174-3937-7a93-8a9d-3b54c2d0628a", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-32-019f2174-3952-7782-9b31-e675621e2eb0.jsonl",
"sha256": "4f4ce9fa94596de5ee070d7146e942614e05ce2aa31e3ea98545045b0a628d32", "messages": 695,
"markdown": "archive\\codex-windows\\20260702-019f2174-3952-7782-9b31-e675621e2eb0-rollout-2026-
07-02t11-41-32-019f2174-3952-77.md", "pdf": "archive\\codex-windows\\20260702-019f2174-3952-
7782-9b31-e675621e2eb0-rollout-2026-07-02t11-41-32-019f2174-3952-77.pdf", "raw": null,
"metadata": {"session_id": "019f2174-3952-7782-9b31-e675621e2eb0", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-32-019f2174-3963-7a90-a81e-85d0ec15c96c.jsonl",
"sha256": "a13b2b3f93a332a01378c5e6c182e996293c9b8d97b99719f695a7eb7187adac", "messages": 595,
"markdown": "archive\\codex-windows\\20260702-019f2174-3963-7a90-a81e-85d0ec15c96c-rollout-2026-
07-02t11-41-32-019f2174-3963-7a.md", "pdf": "archive\\codex-windows\\20260702-019f2174-3963-
7a90-a81e-85d0ec15c96c-rollout-2026-07-02t11-41-32-019f2174-3963-7a.pdf", "raw": null,
"metadata": {"session_id": "019f2174-3963-7a90-a81e-85d0ec15c96c", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-32-019f2174-396f-71a1-99ab-269083c0eccf.jsonl",
"sha256": "6952bdf4851f333f1a74c7852287f86b52e1904f713061f88172c9d5ec1f3f24", "messages": 305,
"markdown": "archive\\codex-windows\\20260702-019f2174-396f-71a1-99ab-269083c0eccf-
rollout-2026-07-02t11-41-32-019f2174-396f-71.md", "pdf": "archive\\codex-
windows\\20260702-019f2174-396f-71a1-99ab-269083c0eccf-
rollout-2026-07-02t11-41-32-019f2174-396f-71.pdf", "raw": null, "metadata": {"session_id":
"019f2174-396f-71a1-99ab-269083c0eccf", "cwd": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer", "model_provider": "openai", "cli_version": "0.142.5", "source": "vscode"}}}
10?{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessio
ns\\2026\\07\\02\\rollout-2026-07-02T11-41-32-019f2174-3994-71a1-8a79-db052ece7f01.jsonl",
"sha256": "3ee749a8fcb565f7eac1b977b06c1e8c9c3994e26bbd175ef894854a747d8550", "messages": 2030,
"markdown": "archive\\codex-windows\\20260702-019f2174-3994-71a1-8a79-db052ece7f01-rollout-2026-
07-02t11-41-32-019f2174-3994-71.md", "pdf": "archive\\codex-windows\\20260702-019f2174-3994-
71a1-8a79-db052ece7f01-rollout-2026-07-02t11-41-32-019f2174-3994-71.pdf", "raw": null,
"metadata": {"session_id": "019f2174-3994-71a1-8a79-db052ece7f01", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-32-019f2174-39bc-7ef3-a52e-f735d1cd827b.jsonl",
"sha256": "4be6fc02fb5e1ce20e2d3107077fc8d140284a78c5872ee84ee321202ecaada8", "messages": 197,
"markdown": "archive\\codex-windows\\20260702-019f2174-39bc-7ef3-a52e-f735d1cd827b-rollout-2026-
07-02t11-41-32-019f2174-39bc-7e.md", "pdf": "archive\\codex-windows\\20260702-019f2174-39bc-
7ef3-a52e-f735d1cd827b-rollout-2026-07-02t11-41-32-019f2174-39bc-7e.pdf", "raw": null,
"metadata": {"session_id": "019f2174-39bc-7ef3-a52e-f735d1cd827b", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-32-019f2174-39c7-7941-84a6-7b6c81c91f4b.jsonl",
"sha256": "f43f87ed3ff0eb8dd0c956ec17a11e8722a6247b406f6bbe52042e7ba926dc68", "messages": 1101,
"markdown": "archive\\codex-windows\\20260702-019f2174-39c7-7941-84a6-7b6c81c91f4b-rollout-2026-
07-02t11-41-32-019f2174-39c7-79.md", "pdf": "archive\\codex-windows\\20260702-019f2174-39c7-
7941-84a6-7b6c81c91f4b-rollout-2026-07-02t11-41-32-019f2174-39c7-79.pdf", "raw": null,
"metadata": {"session_id": "019f2174-39c7-7941-84a6-7b6c81c91f4b", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-32-019f2174-39ee-7ab0-9495-1769dfec0fe.jsonl",
"sha256": "6ee0b2e84815ab701407d3996dc6407b9dc7d8ba0120bfe14a240db227e0bd5e", "messages": 47,
"markdown": "archive\\codex-windows\\20260702-019f2174-39ee-7ab0-9495-1769dfec0fe-
rollout-2026-07-02t11-41-32-019f2174-39ee-7a.md", "pdf": "archive\\codex-
windows\\20260702-019f2174-39ee-7ab0-9495-1769dfec0fe-
```

```
rollout-2026-07-02t11-41-32-019f2174-39ee-7a.pdf", "raw": null, "metadata": {"session_id":
"019f2174-39ee-7ab0-9495-1769dfec0fe", "cwd": "c:\\Users\\avidu\\Projects\\badminton-highlight-
indexer", "model_provider": "openai", "cli_version": "0.142.5", "source": "vscode"}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-32-019f2174-39fd-7fe3-a9d6-704b33b72a6c.jsonl",
"sha256": "69c93aa39f50d133f22cc196686875bc6fd9efe4e1e69320e76b8bb9ea98be92", "messages": 693,
"markdown": "archive\\codex-windows\\20260702-019f2174-39fd-7fe3-a9d6-704b33b72a6c-rollout-2026-
07-02t11-41-32-019f2174-39fd-7f.md", "pdf": "archive\\codex-windows\\20260702-019f2174-39fd-
7fe3-a9d6-704b33b72a6c-rollout-2026-07-02t11-41-32-019f2174-39fd-7f.pdf", "raw": null,
"metadata": {"session_id": "019f2174-39fd-7fe3-a9d6-704b33b72a6c", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-32-019f2174-3a05-7a62-a8ba-9d5d1471204f.jsonl",
"sha256": "fedf38b896e2fec9b1908e8efb3ce30dbc2303a0e3f5e101eba70b1ccce125f5", "messages": 932,
"markdown": "archive\\codex-windows\\20260702-019f2174-3a05-7a62-a8ba-9d5d1471204f-rollout-2026-
07-02t11-41-32-019f2174-3a05-7a.md", "pdf": "archive\\codex-windows\\20260702-019f2174-3a05-
7a62-a8ba-9d5d1471204f-rollout-2026-07-02t11-41-32-019f2174-3a05-7a.pdf", "raw": null,
"metadata": {"session_id": "019f2174-3a05-7a62-a8ba-9d5d1471204f", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-33-019f2174-3a0d-70c1-95bd-8b9f870a11a1.jsonl",
"sha256": "c53ce180c080ad90c1148345f3573f32c210ea7b27ba90b10342ca091d069d54", "messages": 186,
"markdown": "archive\\codex-windows\\20260702-019f2174-3a0d-70c1-95bd-8b9f870a11a1-rollout-2026-
07-02t11-41-33-019f2174-3a0d-70.md", "pdf": "archive\\codex-windows\\20260702-019f2174-3a0d-
70c1-95bd-8b9f870a11a1-rollout-2026-07-02t11-41-33-019f2174-3a0d-70.pdf", "raw": null,
"metadata": {"session_id": "019f2174-3a0d-70c1-95bd-8b9f870a11a1", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-33-019f2174-3a2c-7bb1-a822-f9c7a1e1ecfc.jsonl",
"sha256": "388785a571ef80e936aa451a238452281b651f56cbe5217edd6e7ae2057e899f", "messages": 272,
"markdown": "archive\\codex-windows\\20260702-019f2174-3a2c-7bb1-a822-f9c7a1e1ecfc-
rollout-2026-07-02t11-41-33-019f2174-3a2c-7b.md", "pdf": "archive\\codex-
windows\\20260702-019f2174-3a2c-7bb1-a822-f9c7a1e1ecfc-
rollout-2026-07-02t11-41-33-019f2174-3a2c-7b.pdf", "raw": null, "metadata": {"session_id":
"019f2174-3a2c-7bb1-a822-f9c7a1e1ecfc", "cwd": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer", "model_provider": "openai", "cli_version": "0.142.5", "source": "vscode"}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-33-019f2174-3a3a-7812-a55c-d352f28a4a3d.jsonl",
"sha256": "1b1890b2c9500ee3d10f158317d264abb6243d122e957d8c566ee4b1aa139a44", "messages": 76,
"markdown": "archive\\codex-windows\\20260702-019f2174-3a3a-7812-a55c-d352f28a4a3d-rollout-2026-
07-02t11-41-33-019f2174-3a3a-78.md", "pdf": "archive\\codex-windows\\20260702-019f2174-3a3a-
7812-a55c-d352f28a4a3d-rollout-2026-07-02t11-41-33-019f2174-3a3a-78.pdf", "raw": null,
"metadata": {"session_id": "019f2174-3a3a-7812-a55c-d352f28a4a3d", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}
{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessions\\
\\2026\\07\\02\\rollout-2026-07-02T11-41-33-019f2174-3a58-78d0-b693-2b853628484c.jsonl",
"sha256": "660c6e8bf29774b1d71126c5066cbf8acd85bf9dc6a81a6df40dfbae2b2f3bfff", "messages": 145,
"markdown": "archive\\codex-windows\\20260702-019f2174-3a58-78d0-b693-2b853628484c-rollout-2026-
07-02t11-41-33-019f2174-3a58-78.md", "pdf": "archive\\codex-windows\\20260702-019f2174-3a58-
78d0-b693-2b853628484c-rollout-2026-07-02t11-41-33-019f2174-3a58-78.pdf", "raw": null,
"metadata": {"session_id": "019f2174-3a58-78d0-b693-2b853628484c", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
"cli_version": "0.142.5", "source": "vscode"}}
20?{"source": "codex-windows", "kind": "codex", "source_file": "C:\\Users\\avidu\\.codex\\sessio
ns\\2026\\07\\02\\rollout-2026-07-02T11-41-33-019f2174-3ada-7050-baeb-4fbd73a0d448.jsonl",
"sha256": "8ad45ec3b9dfff3ef34e3d0bd05671172c296f649b58971aeb2f6aa897d39046", "messages": 448,
"markdown": "archive\\codex-windows\\20260702-019f2174-3ada-7050-baeb-4fbd73a0d448-rollout-2026-
07-02t11-41-33-019f2174-3ada-70.md", "pdf": "archive\\codex-windows\\20260702-019f2174-3ada-
7050-baeb-4fbd73a0d448-rollout-2026-07-02t11-41-33-019f2174-3ada-70.pdf", "raw": null,
"metadata": {"session_id": "019f2174-3ada-7050-baeb-4fbd73a0d448", "cwd":
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer", "model_provider": "openai",
```



```
"cli_version": "0.142.5", "source": "vscode"}}
```

### 31. tool\_result

```
<workspace_result workspace_path="/home/avidullu/projects/Agent/agent-sessions">
Found at least 29 matching lines
/home/avidullu/projects/Agent/agent-sessions/archive/INDEX.md
1:# Agent Session Archive
2:
3:Generated: `2026-07-05T16:54:46+00:00`
4:
5:| Source | Kind | Messages | Markdown | PDF |
6:| --- | --- | ---: | --- | --- |
7:| codex-windows | codex | 72 | [20260702-019f2174-38b2-7d92-bff9-9b57ab7306e6-rollout-2026-07-02t11-41-32-019f2174-38b2-7d.md](codex-windows/20260702-019f2174-38b2-7d92-bff9-9b57ab7306e6-rollout-2026-07-02t11-41-32-019f2174-38b2-7d.md) | [PDF](codex-windows/20260702-019f2174-38b2-7d92-bff9-9b57ab7306e6-rollout-2026-07-02t11-41-32-019f2174-38b2-7d.pdf) |
8:| codex-windows | codex | 148 | [20260702-019f2174-38b3-7260-a032-587c94d92d34-rollout-2026-07-02t11-41-32-019f2174-38b3-72.md](codex-windows/20260702-019f2174-38b3-7260-a032-587c94d92d34-rollout-2026-07-02t11-41-32-019f2174-38b3-72.md) | [PDF](codex-windows/20260702-019f2174-38b3-7260-a032-587c94d92d34-rollout-2026-07-02t11-41-32-019f2174-38b3-72.pdf) |
9:| codex-windows | codex | 870 | [20260702-019f2174-38dd-7920-9ff4-c09b81ac2ed8-rollout-2026-07-02t11-41-32-019f2174-38dd-79.md](codex-windows/20260702-019f2174-38dd-7920-9ff4-c09b81ac2ed8-rollout-2026-07-02t11-41-32-019f2174-38dd-79.md) | [PDF](codex-windows/20260702-019f2174-38dd-7920-9ff4-c09b81ac2ed8-rollout-2026-07-02t11-41-32-019f2174-38dd-79.pdf) |
10:| codex-windows | codex | 1031 | [20260702-019f2174-38e5-7980-9505-83230428c846-rollout-2026-07-02t11-41-32-019f2174-38e5-79.md](codex-windows/20260702-019f2174-38e5-7980-9505-83230428c846-rollout-2026-07-02t11-41-32-019f2174-38e5-79.md) | [PDF](codex-windows/20260702-019f2174-38e5-7980-9505-83230428c846-rollout-2026-07-02t11-41-32-019f2174-38e5-79.pdf) |
11:| codex-windows | codex | 979 | [20260702-019f2174-390c-7ce2-8083-44c35ac32ea0-rollout-2026-07-02t11-41-32-019f2174-390c-7c.md](codex-windows/20260702-019f2174-390c-7ce2-8083-44c35ac32ea0-rollout-2026-07-02t11-41-32-019f2174-390c-7c.md) | [PDF](codex-windows/20260702-019f2174-390c-7ce2-8083-44c35ac32ea0-rollout-2026-07-02t11-41-32-019f2174-390c-7c.pdf) |
12:| codex-windows | codex | 2637 | [20260702-019f2174-3937-7a93-8a9d-3b54c2d0628a-rollout-2026-07-02t11-41-32-019f2174-3937-7a.md](codex-windows/20260702-019f2174-3937-7a93-8a9d-3b54c2d0628a-rollout-2026-07-02t11-41-32-019f2174-3937-7a.md) | [PDF](codex-windows/20260702-019f2174-3937-7a93-8a9d-3b54c2d0628a-rollout-2026-07-02t11-41-32-019f2174-3937-7a.pdf) |
13:| codex-windows | codex | 695 | [20260702-019f2174-3952-7782-9b31-e675621e2eb0-rollout-2026-07-02t11-41-32-019f2174-3952-77.md](codex-windows/20260702-019f2174-3952-7782-9b31-e675621e2eb0-rollout-2026-07-02t11-41-32-019f2174-3952-77.md) | [PDF](codex-windows/20260702-019f2174-3952-7782-9b31-e675621e2eb0-rollout-2026-07-02t11-41-32-019f2174-3952-77.pdf) |
14:| codex-windows | codex | 595 | [20260702-019f2174-3963-7a90-a81e-85d0ec15c96c-rollout-2026-07-02t11-41-32-019f2174-3963-7a.md](codex-windows/20260702-019f2174-3963-7a90-a81e-85d0ec15c96c-rollout-2026-07-02t11-41-32-019f2174-3963-7a.md) | [PDF](codex-windows/20260702-019f2174-3963-7a90-a81e-85d0ec15c96c-rollout-2026-07-02t11-41-32-019f2174-3963-7a.pdf) |
15:| codex-windows | codex | 305 | [20260702-019f2174-396f-71a1-99ab-269083c0eccf-rollout-2026-07-02t11-41-32-019f2174-396f-71.md](codex-windows/20260702-019f2174-396f-71a1-99ab-269083c0eccf-rollout-2026-07-02t11-41-32-019f2174-396f-71.md) | [PDF](codex-windows/20260702-019f2174-396f-71a1-99ab-269083c0eccf-rollout-2026-07-02t11-41-32-019f2174-396f-71.pdf) |
16:| codex-windows | codex | 2030 | [20260702-019f2174-3994-71a1-8a79-db052ece7f01-rollout-2026-07-02t11-41-32-019f2174-3994-71.md](codex-windows/20260702-019f2174-3994-71a1-8a79-db052ece7f01-rollout-2026-07-02t11-41-32-019f2174-3994-71.md) | [PDF](codex-windows/20260702-019f2174-3994-71a1-8a79-db052ece7f01-rollout-2026-07-02t11-41-32-019f2174-3994-71.pdf) |
17:| codex-windows | codex | 197 | [20260702-019f2174-39bc-7ef3-a52e-f735d1cd827b-rollout-2026-07-02t11-41-32-019f2174-39bc-7e.md](codex-windows/20260702-019f2174-39bc-7ef3-a52e-f735d1cd827b-rollout-2026-07-02t11-41-32-019f2174-39bc-7e.md) | [PDF](codex-windows/20260702-019f2174-39bc-7ef3-a52e-f735d1cd827b-rollout-2026-07-02t11-41-32-019f2174-39bc-7e.pdf) |
18:| codex-windows | codex | 1101 | [20260702-019f2174-39c7-7941-84a6-7b6c81c91f4b-rollout-2026-07-02t11-41-32-019f2174-39c7-79.md](codex-windows/20260702-019f2174-39c7-7941-84a6-7b6c81c91f4b-rollout-2026-07-02t11-41-32-019f2174-39c7-79.md) | [PDF](codex-windows/20260702-019f2174-39c7-7941-84a6-7b6c81c91f4b-rollout-2026-07-02t11-41-32-019f2174-39c7-79.pdf) |
19:| codex-windows | codex | 47 | [20260702-019f2174-39ee-7ab0-9495-1769dfdec0fe-rollout-2026-07-02t11-41-32-019f2174-39ee-7a.md](codex-
```

windows/20260702-019f2174-39ee-7ab0-9495-1769fdfec0fe-rollout-2026-07-02t11-41-32-019f2174-39ee-7a.md) | [PDF](codex-w  
indows/20260702-019f2174-39ee-7ab0-9495-1769fdfec0fe-rollout-2026-07-02t11-41-32-019f2174-39ee-7a.pdf) |  
20:| codex-windows | codex | 693 | [20260702-019f2174-39fd-7fe3-a9d6-704b33b72a6c-rollout-2026-07-02t11-41-32-019f2174-39fd-7f.md](codex-windows/20260702-019f2174-39fd-7fe3-a9d6-704b33b72a6c-rollout-2026-07-02t11-41-32-019f2174-39fd-7f.md) | [PDF](codex-windows/20260702-019f2174-39fd-7fe3-a9d6-704b33b72a6c-rollout-2026-07-02t11-41-32-019f2174-39fd-7f.pdf) |  
21:| codex-windows | codex | 932 | [20260702-019f2174-3a05-7a62-a8ba-9d5d1471204f-rollout-2026-07-02t11-41-32-019f2174-3a05-7a.md](codex-windows/20260702-019f2174-3a05-7a62-a8ba-9d5d1471204f-rollout-2026-07-02t11-41-32-019f2174-3a05-7a.md) | [PDF](codex-windows/20260702-019f2174-3a05-7a62-a8ba-9d5d1471204f-rollout-2026-07-02t11-41-32-019f2174-3a05-7a.pdf) |  
22:| codex-windows | codex | 186 | [20260702-019f2174-3a0d-70c1-95bd-8b9f870a11a1-rollout-2026-07-02t11-41-33-019f2174-3a0d-70.md](codex-windows/20260702-019f2174-3a0d-70c1-95bd-8b9f870a11a1-rollout-2026-07-02t11-41-33-019f2174-3a0d-70.md) | [PDF](codex-windows/20260702-019f2174-3a0d-70c1-95bd-8b9f870a11a1-rollout-2026-07-02t11-41-33-019f2174-3a0d-70.pdf) |  
23:| codex-windows | codex | 272 | [20260702-019f2174-3a2c-7bb1-a822-f9c7a1e1ecfc-rollout-2026-07-02t11-41-33-019f2174-3a2c-7b.md](codex-w  
indows/20260702-019f2174-3a2c-7bb1-a822-f9c7a1e1ecfc-rollout-2026-07-02t11-41-33-019f2174-3a2c-7b.md) | [PDF](codex-w  
indows/20260702-019f2174-3a2c-7bb1-a822-f9c7a1e1ecfc-rollout-2026-07-02t11-41-33-019f2174-3a2c-7b.pdf) |  
24:| codex-windows | codex | 76 | [20260702-019f2174-3a3a-7812-a55c-d352f28a4a3d-rollout-2026-07-02t11-41-33-019f2174-3a3a-78.md](codex-windows/20260702-019f2174-3a3a-7812-a55c-d352f28a4a3d-rollout-2026-07-02t11-41-33-019f2174-3a3a-78.md) | [PDF](codex-windows/20260702-019f2174-3a3a-7812-a55c-d352f28a4a3d-rollout-2026-07-02t11-41-33-019f2174-3a3a-78.pdf) |  
25:| codex-windows | codex | 145 | [20260702-019f2174-3a58-78d0-b693-2b853628484c-rollout-2026-07-02t11-41-33-019f2174-3a58-78.md](codex-windows/20260702-019f2174-3a58-78d0-b693-2b853628484c-rollout-2026-07-02t11-41-33-019f2174-3a58-78.md) | [PDF](codex-windows/20260702-019f2174-3a58-78d0-b693-2b853628484c-rollout-2026-07-02t11-41-33-019f2174-3a58-78.pdf) |  
26:| codex-windows | codex | 448 | [20260702-019f2174-3ada-7050-baeb-4fbd73a0d448-rollout-2026-07-02t11-41-33-019f2174-3ada-70.md](codex-windows/20260702-019f2174-3ada-7050-baeb-4fbd73a0d448-rollout-2026-07-02t11-41-33-019f2174-3ada-70.md) | [PDF](codex-windows/20260702-019f2174-3ada-7050-baeb-4fbd73a0d448-rollout-2026-07-02t11-41-33-019f2174-3ada-70.pdf) |  
27:| codex-windows | codex | 100 | [20260702-dc9fc71e-8ce0-430b-b84a-21e2ebc23435-rollout-2026-07-02t11-41-33-019f2174-3d41-79.md](codex-windows/20260702-dc9fc71e-8ce0-430b-b84a-21e2ebc23435-rollout-2026-07-02t11-41-33-019f2174-3d41-79.md) | [PDF](codex-windows/20260702-dc9fc71e-8ce0-430b-b84a-21e2ebc23435-rollout-2026-07-02t11-41-33-019f2174-3d41-79.pdf) |  
28:| codex-windows | codex | 62 | [20260702-019f2179-d9ee-7da3-9421-1f409958cad9-rollout-2026-07-02t11-47-41-019f2179-d9ee-7d.md](codex-windows/20260702-019f2179-d9ee-7da3-9421-1f409958cad9-rollout-2026-07-02t11-47-41-019f2179-d9ee-7d.md) | [PDF](codex-windows/20260702-019f2179-d9ee-7da3-9421-1f409958cad9-rollout-2026-07-02t11-47-41-019f2179-d9ee-7d.pdf) |  
29:| codex-windows | codex | 9 | [20260704-019f2ad2-a3aa-7c53-816b-a75517c9130a-rollout-2026-07-04t07-21-19-019f2ad2-a3aa-7c.md](codex-windows/20260704-019f2ad2-a3aa-7c53-816b-a75517c9130a-rollout-2026-07-04t07-21-19-019f2ad2-a3aa-7c.md) | [PDF](codex-windows/20260704-019f2ad2-a3aa-7c53-816b-a75517c9130a-rollout-2026-07-04t07-21-19-019f2ad2-a3aa-7c.pdf) |  
</workspace\_result>

## 32. tool\_result

```
1?[archive]
archive_dir = "archive"
raw_dir = "raw"

[[sources]]
name = "codex-windows"
kind = "codex"
description = "Codex local and archived JSONL sessions on Windows."
roots = [
10? "{home}/.codex/sessions",
 "{home}/.codex/archived_sessions",
]
glob = "**/*.jsonl"

[[sources]]
```

```
name = "claude-windows"
kind = "claude"
description = "Claude Code project JSONL sessions on Windows."
roots = [{"home}/.claude/projects"]
20?glob = "**/*.jsonl"
```

```
[[sources]]
name = "claude-wsl-ubuntu"
kind = "claude"
description = "Claude Code project JSONL sessions in the default WSL distribution."
roots = [{"wsl_home}/.claude/projects"]
glob = "**/*.jsonl"
```

```
[[sources]]
30?name = "gemini-antigravity-windows"
kind = "gemini_antigravity"
description = "Gemini Antigravity transcript JSONL files on Windows."
roots = [{"home}/.gemini/antigravity/brain"]
glob = "**/transcript*.jsonl"
```

```
[[sources]]
name = "grok-wsl-ubuntu"
kind = "grok"
description = "Grok chat history files in the default WSL distribution."
40?roots = [{"wsl_home}/.grok/sessions"]
glob = "**/chat_history.jsonl"
```

```
[[sources]]
name = "deepseek-vscode-windows"
kind = "deepseek_request_dump"
description = "DeepSeek V4 VS Code request dump prompts on Windows."
roots = [
 "{appdata}/Code/User/globalStorage/vizards.deepseek-v4-for-copilot/request-dumps",
]
50?glob = "**/*.msg*.txt"
```

```
[[sources]]
name = "deepseek-vscode-wsl-ubuntu"
kind = "deepseek_request_dump"
description = "DeepSeek V4 VS Code request dump prompts in WSL VS Code Server."
roots = [
 "{wsl_vscode_server}/data/User/globalStorage/vizards.deepseek-v4-for-copilot/request-dumps",
]
glob = "**/*.msg*.txt"
60?
```

```
[[sources]]
name = "copilot-vscode-windows-inventory"
kind = "inventory"
description = "VS Code Copilot Chat storage inventory. Exporter intentionally skips inventory-only sources."
roots = [{"appdata}/Code/User/globalStorage/github.copilot-chat"]
glob = "**/*"
```

```
[[sources]]
name = "copilot-vscode-wsl-ubuntu-inventory"
70?kind = "inventory"
description = "WSL VS Code Server Copilot Chat storage inventory. Exporter intentionally skips inventory-only sources."
roots = [
 "{wsl_vscode_server}/data/User/globalStorage/github.copilot-chat",
]
glob = "**/*"
```

```
[[sources]]
name = "zai-vscode-wsl-ubuntu-inventory"
```

```

kind = "inventory"
80?description = "ZAI VS Code extension inventory. Exporter intentionally skips inventory-only
sources."
roots = [
 "{wsl_vscode_server}/extensions/ltmoerdani.zai-copilot-chat-0.3.1",
]
glob = "**/*"

```

### 33. tool\_result

```

1?"""Keyword signal scanning and deterministic prediction generation.""

```

```

from __future__ import annotations

```

```

import json
import math
import re
from collections import Counter
from typing import Any
10?
from .baseline_settings import BASELINE_CONFIG
from .baseline_types import BaselineSettings, Prediction, TextSignal, confidence
from .config import ArchiveConfig, read_toml
from .utils import archive_markdown_path

```

```

KEYWORD_GROUPS = {
 "repo-governance": (
 "pull request",
20? "pr",
 "merge",
 "approval",
 "direct push",
 "branch",
 "git add",
 "git pull --ff-only",
 "origin/main",
),
 "regression-frameworks": (
30? "pytest",
 "ruff",
 "mypy",
 "coverage",
 "test suite",
 "ci",
 "smoke",
 "regression",
),
 "architecture-decisions": ("architecture", "adr", "trade-off", "pivot", "boundary",
"decision"),
40? "docs-freshness": ("handoff", "readme", "roadmap", "docs", "documentation", "runbook"),
 "checkpointing": ("resume", "checkpoint", "session handoff", "start here", "ramp-up"),
 "metacognition": ("pattern", "memory", "baseline", "salient", "guardrail", "calibrate",
"prediction"),
 "tracked-project-docs": (
 "project_doc_template",
 "tracked project doc",
 "progress tracker",
 "definition of done",
 "decisions locked",
 "one small pr per row",
50? "$7",
),
}

```

```

PR_ONLY_REPO_WRITES_TEXT = (
 "Agents must not push directly to durable/shared repos. They should branch from the remote
base, stage "
 "explicit paths, open a PR, and merge only after explicit approval or a scoped umbrella
approval. Scoped "
 "approval is limited to the named project, PR set, task, and time/context in which it was
granted; it must "
 "not be reused for adjacent work, self-authored PRs, or later PRs without renewed
confirmation."
)
60?

```

### 34. tool\_result

```

exit: 0
3114 archive/index.jsonl

INDEX.md
claude-windows
claude-wsl-ubuntu
codex-windows
deepseek-vscode-windows
deepseek-vscode-wsl-ubuntu
gemini-antigravity-windows
grok-wsl-ubuntu
index.jsonl
...
3115
PDFs:
3114

```

### 35. tool\_result

```

1?# Automation

```

This repo supports a local daily export job. The job is intentionally limited to exporting sessions into `archive/`, committing changed archive files, and pushing to the private GitHub repo. Search indexing and baseline suggestion runs stay separate.

## Daily Export Contract

```

10?The scheduled job should:

```

1. `git pull --ff-only`
2. Optionally run `python tools/agent\_archive.py status`
3. Run `python tools/agent\_archive.py export --all`
4. Optionally include `--pdf`
5. Commit only when `archive/` changed
6. Push only after a successful commit
7. Exit cleanly when there is nothing new

```

20?It should not:

```

- commit `raw/`
- commit machine-local `sources.toml`
- run cass indexing
- run `baseline suggest` daily until promotion is safer
- stage non-archive files

## Windows

```

30?Manual run:

```

```

```powershell
.\scripts\daily-export.ps1
.\scripts\daily-export.ps1 -Pdf
.\scripts\daily-export.ps1 -Source codex-windows -Pdf
```

```

Task Scheduler sketch:

```

40?```powershell
$repo = "C:\Users\avidu\Projects\Agent Sessions"
$action = New-ScheduledTaskAction `
 -Execute "powershell.exe" `
 -Argument "-NoProfile -ExecutionPolicy Bypass -File `"$repo\scripts\daily-export.ps1`" -Pdf"
$trigger = New-ScheduledTaskTrigger -Daily -At 7:30am
Register-ScheduledTask -TaskName "Agent Sessions Daily Export" -Action $action -Trigger $trigger
```

```

WSL/Linux

50?

36. tool_result

```

1?[baseline]
root = "baseline"
candidates_dir = "baseline/candidates"
metacognition_dir = "baseline/metacognition"
ledger_path = "baseline/metacognition/prediction-ledger.jsonl"
feedback_example = "baseline/calibration/feedback.example.toml"

[[pilots]]
slug = "badminton-highlight-indexer"
10?kind = "repo"
aliases = ["badminton-highlight-indexer", "badminton highlight indexer"]
notes = "Mothership project and primary source of tribal knowledge."

[[pilots]]
slug = "muneem"
kind = "repo"
aliases = ["muneem"]
notes = "Newer project with useful agent/workflow upserts."

20?[[pilots]]
slug = "agent-sessions"
kind = "repo"
aliases = ["agent-sessions", "agent sessions"]
notes = "Dogfood repo for baseline loop closure and efficacy calibration."

[[pilots]]
slug = "telegram"
kind = "repo-family"
aliases = ["telegram"]
30?notes = "Integration-heavy pilot; exact repo should be named later."

[[pilots]]
slug = "avidullu"
kind = "github-account"
aliases = ["avidullu"]
notes = "Broader personal repo inventory and cross-project patterns."

[[pilots]]
slug = "khelsutra"
40?kind = "github-organization"
aliases = ["KhelSutra", "khelsutra"]
notes = "Organization-style pilot and future sharing candidate."

```

```

[approval_modes.strict]
description = "PR required and human approval required."
applies_to = ["repo-writes", "baseline-promotion", "destructive-ops", "security-policy",
"architecture-policy"]

[approval_modes.umbrella]
description = "Scoped user approval for a batch of related work."
50?applies_to = ["debugging-batches", "multi-pr-workflows"]

[approval_modes.auto-promote]
description = "Low-risk facts or formatting only, never repo writes or policy changes."
applies_to = ["low-risk-facts", "formatting"]

[approval_modes.observe-only]
description = "Capture evidence and suggestions without publishing into agent-facing
instructions."
applies_to = ["uncertain-patterns", "conflicting-evidence"]

60?# Ground-truth anchors for efficacy calibration (see docs/CALIBRATION_EFFICACY.md).
[[calibration_anchors]]
id = "standard-project-template"
kind = "tracked-project-doc"
source_repo = "badminton-highlight-indexer"
source_path = "docs/PROJECT_DOC_TEMPLATE.md"
local_mirror = "docs/PROJECT_DOC_TEMPLATE.md"
archive_signal = "PROJECT_DOC_TEMPLATE"
archive_hits = 237
notes = "Mothership tracked-project-doc pattern; mirrored here for dogfooding and E2 gate."
70?
[[calibration_anchors]]
id = "session-handoff-ramp-up"
kind = "checkpointing"
source_repo = "badminton-highlight-indexer"
source_path = "SESSION_HANDOFF.md"
local_mirror = "SESSION_HANDOFF.md"
archive_signal = "session-handoff"
archive_hits = 943
notes = "Handoff + ramp-up kit pattern repeated across Claude/Codex/DeepSeek sessions."
80?
[[calibration_anchors]]
id = "baseline-loop-closure-project"
kind = "tracked-project"
source_repo = "agent-sessions"
source_path = "docs/BASELINE_LOOP_CLOSURE.md"
local_mirror = "docs/BASELINE_LOOP_CLOSURE.md"
archive_signal = "baseline loop"
archive_hits = 0
notes = "Dogfood project for closing promote/publish/calibrate loop; $7 is closure tracker."
90?

```

37. tool_result

```

<workspace_result workspace_path="/home/avidullu/projects/Agent/agent-sessions">
Found at least 19 matching lines
/home/avidullu/projects/Agent/agent-sessions/docs/WORK_AUDIT_2026-07-08.md
41:| ID | Item | Impact | Risk | Effort | Priority | Why it matters |

/home/avidullu/projects/Agent/agent-sessions/docs/BASELINE_PLANNING.md
6:## North Star
148:- Work tracking: bugs, follow-ups, deferred ideas, project-specific backlog

/home/avidullu/projects/Agent/agent-sessions/docs/FIRST_USER_SETUP_TRACKER.md
20:## 1. Problem & goal
22:**Problem:** issue #42 reports that the core setup path works, but the user has
27:**Goal:** a first user can run discovery, status, export, baseline scaffold, and

```

/home/avidullu/projects/Agent/agent-sessions/docs/BASELINE_LOOP_CLOSURE.md

19:## 1. Problem & goal

21:**Problem:** Archive + `baseline suggest` originally existed, but

26:**Goal:** A measurable end-to-end proof that session history becomes durable

/home/avidullu/projects/Agent/agent-sessions/docs/BASELINE_WATCHLIST_TOMBSTONES.md

29:## 1. Problem statement

55:### 1.3 User goals (from task)

62:## 2. Goals and non-goals

64:### 2.1 Goals

74:### 2.2 Non-goals (this phase)

236:### 4.1 Purpose

939:**Rejected:** Fingerprint-keyed feedback solves dedup but not **backlog ranking** ? the user's primary watchlist goal. Two-file model (watchlist state + tombstone registry) kept; fingerprint-keyed feedback could complement tombstones later as optional index.

/home/avidullu/projects/Agent/agent-sessions/docs/TEST_PLAN.md

18:## 1. Problem & goal

20:**Problem:** Coverage must stay high as baseline replay, promotion, and linting workflows add new write paths. Any refactor, new extractor, or baseline logic change risks silent regressions without targeted tests.

22:**Goal:** Achieve >80% line coverage via pytest with:

</workspace_result>

38. tool_result

100?

Promotion:

- [] Accept
 - [] Edit and accept
 - [] Reject
 - [] Project-specific only
- ...

Approval Model

110?Approval policy should be explicit and machine-readable:

- `strict`: PR required, human approval required. This applies to shared repo writes, publishing baseline changes, and any destructive or irreversible operation.
- `umbrella`: user grants scoped approval for a batch of related work, such as debugging several findings and opening individual PRs.
- `auto-promote`: allowed only for low-risk facts or formatting improvements after confidence thresholds are met. It must not apply to repo writes, security rules, architecture pivots, or policy changes.
- 120?- `observe-only`: capture evidence and suggestions, but do not publish into agent-facing instructions.

Direct pushes to durable repos should be treated as a cardinal violation unless the repo policy explicitly allows it or the user grants an explicit exception. Merge authorization is context-bound: an instruction such as "merge the PRs" is scoped to the active project or PR set being discussed, and does not carry over to unrelated PRs, self-authored PRs, or later work unless the user renews that approval explicitly.

130?## What To Extract

The baseline should cover all of these dimensions:

- Repo governance: branch creation, PR discipline, merge approval, release etiquette, staging explicit paths.
- Architecture decisions: pivots, trade-offs, ADRs, integration boundaries, data model choices.

- Regression frameworks: test commands, coverage floors, lint/type checks, fixtures, smoke tests, replay harnesses.
- 140?- Coding discipline: local patterns, preferred abstractions, style rules, naming conventions, dependency rules.
- Docs freshness: handoff updates, README drift, runbooks, generated docs, decision logs.
- Tech debt principles: recurring refactors, cleanup thresholds, known risky areas, debt that should not be repeated.
- Checkpointing and resuming: session handoffs, ramp-up kits, open threads, "start here" files.
- Work tracking: bugs, follow-ups, deferred ideas, project-specific backlog items.
- 150?- Prompt and agent patterns: instructions that reliably produce better work, common failure modes, corrective phrasing.

AI-Assisted Proposal Generation

Yes, giving a capable AI agent reasonable access can make the suggestions much crisper. The useful pattern is not "agent decides the baseline." It is "agent helps mine evidence and drafts proposals with citations."

Reasonable access tiers:

39. tool_result

1?# Baseline Candidate Extraction (2026-07-05)

Status: proposed

Generated by: `python .\tools\agent\_archive.py baseline suggest`

Summary

- Indexed sessions: `3114`
- Markdown sessions scanned for text signals: `500`
- 10?- Sources represented: `7`
- Feedback file: `none`

Source Mix

- `claude-windows`: 2222
- `deepseek-vscode-windows`: 752
- `deepseek-vscode-wsl-ubuntu`: 53
- `codex-windows`: 52
- `claude-wsl-ubuntu`: 19
- 20?- `grok-wsl-ubuntu`: 13
- `gemini-antigravity-windows`: 3

Kind Mix

- `claude`: 2241
- `deepseek\_request\_dump`: 805
- `codex`: 52
- `grok`: 13
- `gemini\_antigravity`: 3
- 30?

Pilot Project Signals

- `badminton-highlight-indexer`: 1770
- `muneem`: 294
- `khelestra`: 233
- `avidullu`: 85
- `telegram`: 1

Text Signal Counts

40?

40. tool_result

```
exit: 0
ness/Productivity-Oriented Engineer", "scope": "user-profile", "risk": "medium", "category":
"metacognition", "confidence": 0.63, "status": "proposed", "feedback": "none", "evidence":
["badminton-highlight-indexer: 1770 archive records", "muneem: 294 archive records", "khelsutra:
233 archive records", "avidullu: 85 archive records", "telegram: 1 archive records"], "text":
"The user looks like a builder who blends software engineering, product/business workflows, and
agent automation to improve ptotal 12
-rw-r--r-- 1 avidullu avidullu 230 Jul  8 14:45 README.md
-rw-r--r-- 1 avidullu avidullu 7849 Jul  8 14:45 prediction-ledger.jsonl
```

41. tool_result

1?# pr-review-loop (Claude Code plugin)

A self-pacing PR reviewer. It sleeps ~15 minutes, wakes to review open PRs (validating locally by running the tests), posts an `LGTM` comment when satisfied so the **author** can merge, and auto-stops its own sleep-wake alarm after N consecutive empty polls.

Ships one skill: `/pr-review-loop:start`.

10?## Install

This repo (`avidullu/agent-sessions`) hosts a marketplace named `agent-sessions-tools`. From any Claude Code session:

```
```shell
/plugin marketplace add avidullu/agent-sessions
/plugin install pr-review-loop@agent-sessions-tools
```
```

20?Then start the reviewer in the target repo's session:

```
```shell
/pr-review-loop:start
```
```

?or just ask in natural language: **"be the designated reviewer for PRs on this repo ? LGTM once you validate them, stop after 3 empty polls."** The skill is model-invoked, so it also triggers when you ask to watch/review/babysit PRs.

30?## Parameters

Parsed from your request; sensible defaults otherwise: `repo`, `filter` (tracker / label / branch prefix / title substring), `cadence\_min` (default 15), `idle\_stop` (default 3), `worktree`.

Behavior notes

- Reviews for correctness **and** side effects; validates by running the repo's tests/lint locally, not just reading the diff.
- 40?- Posts LGTM as a **comment** (works even when the acting account can't formally self-approve); the **author merges** ? the reviewer never does.
- The idle counter rides in the wake-up message, so the loop is stateless across firings and survives context compaction.
- New-PR discovery latency ? `cadence\_min`; already-subscribed PRs get real-time push for new commits/comments.

Local development

```
```shell
50?claude --plugin-dir ./plugins/pr-review-loop
```
```

42. tool_result

```
1?"""Grok local session extraction."""
```

```
from __future__ import annotations
```

```
from pathlib import Path
```

```
from typing import Any
```

```
from ..models import ExtractedSession, SessionMessage
```

```
from ..utils import jsonl_objects, text_from_content
```

```
10?from .registry import register
```

```
@register("grok")
```

```
def extract(path: Path) -> ExtractedSession:
```

```
    metadata: dict[str, Any] = {"session_id": path.parent.name, "project":  
path.parent.parent.name}
```

```
    messages: list[SessionMessage] = []
```

```
    for obj in jsonl_objects(path):
```

```
        content = text_from_content(obj.get("content"))
```

```
        if not content:
```

```
20?            continue
```

```
        role = obj.get("type") or "message"
```

```
        messages.append(SessionMessage(role=role, text=content, timestamp=obj.get("timestamp",
```

```
"")))
```

```
    return ExtractedSession(metadata=metadata, messages=messages)
```

43. tool_result

```
<workspace_result workspace_path="/home/avidullu/projects/Agent/agent-sessions">
```

```
Found at least 17 matching lines
```

```
/home/avidullu/projects/Agent/agent-sessions/tests/test_grok_extractor.py
```

```
11:     def test_basic_extraction(self, tmp_path: Path, sample_grok_jsonl: str) -> None:
```

```
25:     def test_type_fallback_role(self, tmp_path: Path) -> None:
```

```
36:     def test_message_fallback_role(self, tmp_path: Path) -> None:
```

```
47:     def test_empty_content_skipped(self, tmp_path: Path) -> None:
```

```
57:     def test_empty_file(self, tmp_path: Path) -> None:
```

```
/home/avidullu/projects/Agent/agent-sessions/tests/test_baseline_agent.py
```

```
22:     def test_selects_by_focus(self) -> None:
```

```
45:     def test_selects_all_when_no_focus(self) -> None:
```

```
67:     def test_respects_max_sessions(self) -> None:
```

```
82:     def test_sorts_by_focus_score(self) -> None:
```

```
105:    def test_per_source_limit(self) -> None:
```

```
124:    def test_creates_record(self, tmp_path: Path) -> None:
```

```
150:    def test_missing_markdown_file(self, tmp_path: Path) -> None:
```

```
169:    def test_session_only(self) -> None:
```

```
174:    def test_repo_read_only(self) -> None:
```

```
178:    def test_write_candidates(self) -> None:
```

```
184:    def test_returns_dict(self) -> None:
```

```
199:    def test_renders(self) -> None:
```

```
</workspace_result>
```

44. tool_result

```
- /home/avidullu/projects/Agent/agent-sessions/baseline/projects/  
- agent-sessions/  
  - README.md  
- avidullu/
```

- README.md
- badminton-highlight-indexer/
 - README.md
- khelsutra/
 - README.md
- muneem/
 - README.md
- telegram/
 - README.md

45. tool_result

1?# Prompt Patterns

Promoted prompt guidance will land here.

46. tool_result

```
<workspace_result workspace_path="/home/avidullu/projects/Agent/agent-sessions">
Found at least 14 matching lines
/home/avidullu/projects/Agent/agent-sessions/agent_sessions/archive.py
3:from __future__ import annotations
29:class ExportResult:
35:def iter_source_files(source: Source) -> Iterable[Path]:
50:def sha256_file(path: Path) -> str:
58:def copy_raw(config: ArchiveConfig, path: Path, source: Source, digest: str) -> Path:
66:def select_sources(config: ArchiveConfig, selected: list[str] | None) -> list[Source]:
76:def _index_path_exists(config: ArchiveConfig, rel: Any) -> bool:
80:def _can_reuse_record(
108:def export_sources(
194:def index_record_key(record: dict[str, Any]) -> tuple[str, str]:
200:def index_identity_key(record: dict[str, Any]) -> tuple[str, ...]:
214:def _posix_index_record(record: dict[str, Any]) -> dict[str, Any]:
225:def read_existing_index_records(config: ArchiveConfig) -> list[dict[str, Any]]:
232:def merge_index_records(existing: list[dict[str, Any]], current: list[dict[str, Any]]) ->
list[dict[str, Any]]:
</workspace_result>
```

47. tool_result

```
{
  "results": [
    {
      "server": "grok_com_github",
      "tools": [
        {
          "tool_name": "grok_com_github__sub_issue_write",
          "description": "Add a sub-issue to a parent issue in a GitHub repository.",
          "score": 6.327215194702148,
          "input_schema": {
            "type": "object",
            "properties": {
              "after_id": {
                "type": "number",
                "description": "The ID of the sub-issue to be prioritized after (either after_id
OR before_id should be specified)"
              },
              "before_id": {
                "type": "number",
                "description": "The ID of the sub-issue to be prioritized before (either
after_id OR before_id should be specified)"
              },
              "issue_number": {
                "type": "number",

```

```

        "description": "The number of the parent issue"
    },
    "method": {
        "type": "string",
        "description": "The action to perform on a single sub-issue\nOptions are:\n-
'add' - add a sub-issue to a parent issue in a GitHub repository.\n- 'remove' - remove a sub-
issue from a parent issue in a GitHub repository.\n- 'reprioritize' - change the order of sub-
issues within a parent issue in a GitHub repository. Use either 'after_id' or 'before_id' to
specify the new position.\n\t\t\t\t\t"
    },
    "owner": {
        "type": "string",
        "description": "Repository owner",
        "x-mcp-header": "owner"
    },
    "replace_parent": {
        "type": "boolean",
        "description": "When true, replaces the sub-issue's current parent issue. Use
with 'add' method only."
    },
    "repo": {
        "type": "string",
        "description": "Repository name",
        "x-mcp-header": "repo"
    },
    "sub_issue_id": {
        "type": "number",
        "description": "The ID of the sub-issue to add. ID is not the same as issue
number"
    }
},
"required": [
    "method",
    "owner",
    "repo",
    "issue_number",
    "sub_issue_id"
]
},
{
    "tool_name": "grok_com_github_triage_issue",
    "description": "Triage an issue by capturing a focused triage rationale and optionally
applying metadata (labels, issue type, and issue fields) in a single operation.\nUse this tool
when:\n- You are triaging a newly opened or untriaged issue for maintainers\n- You need to
categorize the issue (type) and suggest/apply relevant labels\n- You want to record a brief,
structured triage report\n\nDo not use this tool when:\n- The user is asking a general product
question unrelated to triaging a specific issue\n\ntriage_rationale should be concise markdown
aimed at maintainers (not the issue author); it should contain:\n- Summary of the issue\n-
Analysis of the problem or request\n- Suggested next steps or actions.",
    "score": 6.234358310699463,
    "input_schema": {
        "type": "object",
        "properties": {
            "fields": {
                "type": "array",
                "items": {
                    "type": "object",
                    "properties": {
                        "name": {
                            "type": "string",
                            "description": "Field name"
                        }
                    }
                },
                "value": {
                    "description": "Field value (string, number, or other supported type)"
                }
            }
        }
    }
}

```

```

        }
      },
      "required": [
        "name",
        "value"
      ]
    },
    "description": "Custom fields to set on the issue (optional)"
  },
  "issue_number": {
    "type": "number",
    "description": "The issue number to triage",
    "minimum": 1
  },
  "labels": {
    "type": "array",
    "items": {
      "type": "string"
    }
  },
  "description": "Labels to apply to the issue (optional)"
},
"owner": {
  "type": "string",
  "description": "Repository owner (username or organization)",
  "x-mcp-header": "owner"
},
"repo": {
  "type": "string",
  "description": "Repository name",
  "x-mcp-header": "repo"
},
"triage_rationale": {
  "type": "string",
  "description": "The triage comment body containing analysis and rationale. This
will be posted as a comment on the issue."
},
"type": {
  "type": "string",
  "description": "The issue type to set (optional, e.g., 'bug',
'feature_request')"
}
},
"required": [
  "owner",
  "repo",
  "issue_number",
  "triage_rationale"
]
}
},
{
  "tool_name": "grok_com_github_semantic_issue_similarity_search",
  "description": "Find issues that are semantically similar to a given issue using
natural language understanding. Useful for identifying duplicates, related issues, or similar
problems.",
  "score": 6.199577808380127,
  "input_schema": {
    "type": "object",
    "properties": {
      "issue_number": {
        "type": "number",
        "description": "The issue number to find similar issues for",
        "minimum": 1
      },
      "owner": {

```

```

        "type": "string",
        "description": "Repository owner",
        "x-mcp-header": "owner"
    },
    "page": {
        "type": "number",
        "description": "Page number for pagination (min 1)",
        "minimum": 1
    },
    "per_page": {
        "type": "number",
        "description": "Number of similar issues to return per page (default: 3, max:
10)",
        "minimum": 1,
        "maximum": 10
    },
    "repo": {
        "type": "string",
        "description": "Repository name",
        "x-mcp-header": "repo"
    },
    "threshold": {
        "type": "number",
        "description": "Similarity threshold between 0.0 and 1.0 (default: 0.80)",
        "minimum": 0,
        "maximum": 1
    }
},
"required": [
    "owner",
    "repo",
    "issue_number"
]
}
},
{
    "tool_name": "grok_com_github__list_issue_types",
    "description": "List supported issue types for a repository or its owner organization.
When repo is omitted, returns org-level issue types directly.",
    "score": 5.894150733947754,
    "input_schema": {
        "type": "object",
        "properties": {
            "owner": {
                "type": "string",
                "description": "The account owner of the repository or organization.",
                "x-mcp-header": "owner"
            },
            "repo": {
                "type": "string",
                "description": "The name of the repository. When provided, returns issue types
for this specific repository. When omitted, returns org-level issue types directly.",
                "x-mcp-header": "repo"
            }
        },
        "required": [
            "owner"
        ]
    }
},
{
    "tool_name": "grok_com_github__issue_read",
    "description": "Get information about a specific issue in a GitHub repository.",
    "score": 5.894150733947754,
    "input_schema": {

```

```

    "type": "object",
    "properties": {
      "issue_number": {
        "type": "number",
        "description": "The number of the issue"
      },
      "method": {
        "type": "string",
        "description": "The read operation to perform on a single issue.\nOptions
are:\n1. get - Get details of a specific issue.\n2. get_comments - Get issue comments.\n3.
get_sub_issues - Get sub-issues (children) of the issue.\n4. get_parent - Get the parent issue,
if this issue is a sub-issue of another.\n5. get_labels - Get labels assigned to the issue.\n",
        "enum": [
          "get",
          "get_comments",
          "get_sub_issues",
          "get_parent",
          "get_labels"
        ]
      },
      "owner": {
        "type": "string",
        "description": "The owner of the repository",
        "x-mcp-header": "owner"
      },
      "page": {
        "type": "number",
        "description": "Page number for pagination (min 1)",
        "minimum": 1
      },
      "perPage": {
        "type": "number",
        "description": "Results per page for pagination (min 1, max 100)",
        "minimum": 1,
        "maximum": 100
      },
      "repo": {
        "type": "string",
        "description": "The name of the repository",
        "x-mcp-header": "repo"
      }
    },
    "required": [
      "method",
      "owner",
      "repo",
      "issue_number"
    ]
  }
}
],
"total_hidden_tools": 91,
"status": "ready",
"note": null
}

```

48. tool_result

1?# badminton-highlight-indexer

Project-specific promoted baseline notes for `badminton-highlight-indexer` will land here.

<!-- baseline:begin id="handoffs.index" -->

Handoff Signals

Generated by: `baseline handoffs index`
10? **Generated at:** 2026-07-07

Indexed handoff candidates: `62`.

Candidates

- [archive/claude-windows/20260606-55d47d77-6dbb-4afc-9ad0-315df54b08fd-55d47d77-6dbb-4afc-9ad0-315df54b08fd-2aa890e.md](../.../archive/claude-windows/20260606-55d47d77-6dbb-4afc-9ad0-315df54b08fd-55d47d77-6dbb-4afc-9ad0-315df54b08fd-2aa890e.md) session `55d47d77-6dbb-4afc-9ad0-315df54b08fd`; sections: `Next Steps / Open Threads`.
- [archive/claude-windows/20260610-1c6eb3b4-ed47-4cb3-84e8-50b309c6495c-1c6eb3b4-ed47-4cb3-84e8-50b309c6495c-56bfe18.md](../.../archive/claude-windows/20260610-1c6eb3b4-ed47-4cb3-84e8-50b309c6495c-1c6eb3b4-ed47-4cb3-84e8-50b309c6495c-56bfe18.md) session `1c6eb3b4-ed47-4cb3-84e8-50b309c6495c`; sections: -.
- [archive/claude-windows/20260610-2139f383-5578-4e22-831e-8859ad4f69f2-2139f383-5578-4e22-831e-8859ad4f69f2-2e1c23d.md](../.../archive/claude-windows/20260610-2139f383-5578-4e22-831e-8859ad4f69f2-2139f383-5578-4e22-831e-8859ad4f69f2-2e1c23d.md) session `2139f383-5578-4e22-831e-8859ad4f69f2`; sections: -.
- [archive/claude-windows/20260613-d6979332-5869-49a6-bf7e-bd9f4da1954e-d6979332-5869-49a6-bf7e-bd9f4da1954e-560e614.md](../.../archive/claude-windows/20260613-d6979332-5869-49a6-bf7e-bd9f4da1954e-d6979332-5869-49a6-bf7e-bd9f4da1954e-560e614.md) session `d6979332-5869-49a6-bf7e-bd9f4da1954e`; sections: -.
- [archive/claude-windows/20260620-c47f0851-d75f-45b6-9126-e48544381426-c47f0851-d75f-45b6-9126-e48544381426-f7d33b2.md](../.../archive/claude-windows/20260620-c47f0851-d75f-45b6-9126-e48544381426-f7d33b2.md) session `c47f0851-d75f-45b6-9126-e48544381426`; sections: -.
- 20?- [archive/claude-windows/20260621-577fd4df-ed3f-43d3-854d-d63d732e2e07-577fd4df-ed3f-43d3-854d-d63d732e2e07-c27973f.md](../.../archive/claude-windows/20260621-577fd4df-ed3f-43d3-854d-d63d732e2e07-577fd4df-ed3f-43d3-854d-d63d732e2e07-c27973f.md) session `577fd4df-ed3f-43d3-854d-d63d732e2e07`; sections: -.
- [archive/claude-windows/20260621-dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3-dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3-9aef209.md](../.../archive/claude-windows/20260621-dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3-9aef209.md) session `dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3`; sections: -.
- [archive/claude-windows/20260621-df230d9b-4146-4882-a603-82c0eaffc508-df230d9b-4146-4882-a603-82c0eaffc508-0325022.md](../.../archive/claude-windows/20260621-df230d9b-4146-4882-a603-82c0eaffc508-df230d9b-4146-4882-a603-82c0eaffc508-0325022.md) session `df230d9b-4146-4882-a603-82c0eaffc508`; sections: -.
- [archive/claude-windows/20260621-f81fb2b5-e34b-427e-a8a1-a37eb4539a49-f81fb2b5-e34b-427e-a8a1-a37eb4539a49-d7088cf.md](../.../archive/claude-windows/20260621-f81fb2b5-e34b-427e-a8a1-a37eb4539a49-f81fb2b5-e34b-427e-a8a1-a37eb4539a49-d7088cf.md) session `f81fb2b5-e34b-427e-a8a1-a37eb4539a49`; sections: -.
- [archive/claude-windows/20260622-2ba49afa-99f4-41f2-86a2-27a289d432f3-2ba49afa-99f4-41f2-86a2-27a289d432f3-7877e3f.md](../.../archive/claude-windows/20260622-2ba49afa-99f4-41f2-86a2-27a289d432f3-7877e3f.md) session `2ba49afa-99f4-41f2-86a2-27a289d432f3`; sections: -.
- `52` more indexed handoff candidate(s).

Trace

- `source`=claude-windows; `source\_file`=C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\55d47d77-6dbb-4afc-9ad0-315df54b08fd.jsonl;
`markdown\_path`=archive/claude-windows/20260606-55d47d77-6dbb-4afc-9ad0-315df54b08fd-55d47d77-6dbb-4afc-9ad0-315df54b08fd-2aa890e.md; `session\_id`=55d47d77-6dbb-4afc-9ad0-315df54b08fd;
`project\_slug`=badminton-highlight-indexer; `transform`=baseline handoffs index
- `source`=claude-windows; `source\_file`=C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\1c6eb3b4-ed47-4cb3-84e8-50b309c6495c.jsonl;
`markdown\_path`=archive/claude-windows/20260610-1c6eb3b4-ed47-4cb3-84e8-50b309c6495c-1c6eb3b4-ed47-4cb3-84e8-50b309c6495c-56bfe18.md; `session\_id`=1c6eb3b4-ed47-4cb3-84e8-50b309c6495c;
`project\_slug`=badminton-highlight-indexer; `transform`=baseline handoffs index
- 30?- `source`=claude-windows; `source\_file`=C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\2139f383-5578-4e22-831e-8859ad4f69f2.jsonl;

```
`markdown_path`=archive/claude-windows/20260610-2139f383-5578-4e22-831e-8859ad4f69f2-2139f383-5578-4e22-831e-8859ad4f69f2-2e1c23d.md; `session_id`=2139f383-5578-4e22-831e-8859ad4f69f2; `project_slug`=badminton-highlight-indexer; `transform`=baseline handoffs index
```

49. tool_result

- [completed] overview-complete: Synthesize full repo read into structured overview covering idea, feedbacks, comparisons, engineering, vision

50. assistant

****Overarching idea****: A private, local-first, Git-backed system for archiving coding-agent sessions across multiple tools (Claude Code, Codex, Grok, DeepSeek VS Code dumps, Gemini Antigravity, with inventory notes for Copilot/ZAI) and turning that raw history into a ****compounding, reviewable engineering baseline**** of guardrails, project knowledge, handoff patterns, and agent-specific instructions.

It is not primarily a search tool or live memory system. It faithfully exports local stores (Windows + WSL paths) to portable Markdown + optional PDF + a merge-aware `archive/index.jsonl`, then runs deterministic extraction, candidate generation, calibration, promotion, and publishing steps. Everything is human-gated with strong provenance, redaction, and linting. The repo dogfoods its own patterns (AGENTS.md discipline, SESSION_HANDOFF.md, tracked project docs with \$7 trackers, PR-only writes).

Core value proposition: prevent knowledge loss across sessions, machines, and agent vendors; synthesize repeated signals (repo governance, regression practices, handoffs, prompt patterns, project templates) into durable, promotable policy that gets published back into future agent contexts (e.g., `baseline/agents/.../\*.generated.md`).

****Current scale**** (from committed artifacts): ~3,114 indexed sessions in `archive/index.jsonl` (heavily Claude + DeepSeek, plus Codex/Grok/Gemini), producing thousands of .md/.pdf files. Pilots include `badminton-highlight-indexer` (mothership), `muneem`, `agent-sessions` (self), `khehsutra`, etc.

Owner's Vision (avidullu)

From `docs/BASELINE\_PLANNING.md`, `ENGINEERING\_BASELINE.md`, `BASELINE\_KNOWLEDGE\_REPLAY\_PLAN.md`, `CALIBRATION\_EFFICACY.md`, `COMPOSE\_STACK.md`, `SESSION\_HANDOFF.md`, and `config/baseline.toml`:

- ****North star****: A portable engineering baseline system that learns from archived sessions + repos + reviews + CI + user corrections, then proposes durable guardrails and project knowledge. Start personal/dogfood, then scale to teams, orgs, and client onboarding. A new codebase or org should yield reviewable onboarding plans, knowledge maps, and candidate guardrails.
- Strong process philosophy: "One small PR per row", explicit approval modes (strict/umbrella/auto-promote/observe-only), scoped merge authorization, tracked project docs as the standard (inspired by `badminton-highlight-indexer/docs/PROJECT\_DOC\_TEMPLATE.md`, detected via 237+ archive hits), mandatory handoff/ramp-up artifacts.
- Safety-first and anti-silent-LLM-write: Raw archives immutable; generated content limited to marker blocks (`<!-- baseline:begin/end -->`); no auto-promotion of policy; deterministic transforms preferred initially; provenance/trace fields required for handoff/replay producers; redaction gates for any egress.
- Compose philosophy (delegation): Own durable export + baseline extraction/publishing. Delegate search/indexing to external tools (explicitly cass), live capture (SpecStory), Claude-specific UX/sync, and runtime memory injection.
- Measurable loop closure: Efficacy gates E1?E6 (detect ? anchor ? dogfood ? promote ? publish ? calibrate) + newer W/H/R gates for schema/links/handoffs/replay. Calibration feedback (toml) must measurably affect future suggestions. Replay is intentionally out-of-band in v1.
- Multi-machine + automation: Git convergence via index, scheduled daily exports (pull ? export ? commit only archive changes ? push), portable TOML path templates.

The project is highly self-reflective and documented (extensive trackers, audits, schemas).

Engineering Overview

****Structure****:

- ``agent_sessions/``: Core package (models, config, utils, pluggable sources via registry, archive export/indexing/merge logic, rich baseline subsystem).
- ``tools/agent_archive.py`` + ``agent_sessions/cli.py``: Main CLI (``discover``, ``status``, ``export`` --all [--pdf] [--copy-raw]), ``baseline ...`` subcommands).
- Sources: ``claude.py``, ``codex.py``, ``grok.py``, ``deepseek.py``, ``gemini.py`` (simple extractors that turn JSONL/txt into ``ExtractedSession`` with metadata + messages). Registry + decorator pattern. Inventory-only sources are explicitly skipped.
- Baseline pipeline (split across focused modules for testability): ``baseline.py`` (orchestration), ``baseline_predictions.py`` (keyword groups + signal scanning for deterministic predictions), ``baseline_report.py``, ``baseline_promote.py`` (marker parsing + ``upsert_promoted_content`` / project-page helpers), ``baseline_calibration.py``, ``baseline_lint.py``, ``baseline_handoffs.py``, ``baseline_replay*.py``, settings/scaffolding, types.
- Config: ``config/default_sources.toml`` (templated roots), ``config/baseline.toml`` (pilots, approval modes, calibration anchors), optional local ``sources.toml``.
- Outputs: ``archive/`` (MD + PDF + index + INDEX.md), ``baseline/`` (SCHEMA.md, global/*.md with promoted marker blocks, projects/<slug>/README.md with handoff feeds, agents/*/*.generated.md, candidates/ + .predictions.json sidecars, proposals/, handoffs/, replay/ manifests + gitignored bundles + ledgers, metacognition/ ledgers).
- Other: ``plugins/pr-review-loop/`` (Claude Code marketplace plugin for self-pacing PR review + test validation + LGTM), scripts for daily export, extensive tests/.

****Key technical traits**:**

- Deterministic and reviewable: Keyword-driven predictions (repo-governance, regression-frameworks, checkpointing/handoffs, tracked-project-docs, etc.), stable slugs, stable generated dates when content unchanged, provenance traces aligned to #19.
- Safety: Fail-closed redaction (high-confidence secrets block bundles), gitignore for raw/egress, lint rejects broken markers/links + warns on orphans/stale.
- Portability: Path templates (``{home}``, ``{wsl_home}``, ``{appdata}``, etc.), index merge that preserves records from other machines.
- Quality gates: ~97% coverage (576 tests), mypy, ruff, ``baseline lint --dry-run``, ``baseline eval --dry-run`` (reports E + W/H/R + G-no-autopromote gates; some stay "gated" until real data).
- Render/PDF via reportlab (optional).
- Strict Git workflow (per ``AGENTS.md``): small PRs, review, explicit LGTM before merge; no batching everything.

The baseline subsystem evolved through a tracked plan (K0?K12 for knowledge + handoff mining + replay loop, all landed via small PRs #45?#57 as of the 2026-07-07/08 docs). Original loop-closure work is now marked historical.

Feedbacks (Internal + Current State)

The repo contains unusually transparent self-auditing:

- ``docs/WORK_AUDIT_2026-07-08.md`` (most recent): Repo is healthy on engineering gates (tests pass at 96.94% cov, ruff/mypy clean, baseline lint 0 errors/2 warnings, eval 14 pass/0 fail/2 gated). Primary debt is ****archive freshness**** (status showed 1100+ new visible files at audit time ? baseline/replay quality depends on recent sessions). Replay R3/R4 remain correctly gated until a real out-of-band replay result is ingested. Other items: stale planning docs (cleaned), replay polish/redaction hardening, issue hygiene, daily-export guardrails, two intentional orphan project-page warnings.
- ``docs/BASELINE_KNOWLEDGE_REPLAY_PLAN.md``: Full tracker showing disciplined execution of schema ? handoff audit/index/proposals ? replay select/redact/bundle/ingest + efficacy wiring. All K rows done; value measurement pending live data.
- Calibration/efficacy: Anchors on real repeated patterns (PROJECT_DOC_TEMPLATE, handoffs). Feedback loop exists and is required for E6.
- Other signals: Emphasis on provenance integrity in ingest (rejects unresolved ``replay_of``/trace refs vs archive index), redaction as first-class, no silent rewrites.
- Process feedback is baked in via AGENTS.md and repeated promotion of "handoff-and-resume" + "PR-only repo writes" guardrails (high confidence, hundreds of archive hits).
- Strengths observed: High determinism + safety; good multi-machine story; self-consistent dogfooding.
- Open/acknowledged: Freshness requires intentional export+commit cycles; replay still needs external execution + ingest for full signal; archive backfill (#32) deferred as broad.

Recent work focused on completing the knowledge/replay foundation safely before expanding.

Comparison to Similar Options

This project is deliberately narrow and composable (see `docs/COMPOSE\_STACK.md`).

- **cass** (Dicklesworthstone/coding_agent_session_search): Excellent unified TUI/CLI search + archaeology across 11+ providers (Codex, Claude, Gemini, Cursor, Aider, etc.). Fast indexing, session expansion, handoff support. This repo explicitly delegates search/triage to cass and focuses on durable export + derived policy/baseline. Complementary, not competing.
- **observational-memory** (intertwine): Close spiritual peer ? local markdown memory, automatic capture, multi-agent (Claude/Codex/Grok), multi-machine sync, exports. Overlap on archiving; this adds deeper baseline extraction, promotion/publish loop, replay pipeline, agent-specific generated slices, governance markers, and PDF + index convergence.
- **Claude-specific tools** (claude-code-log, claude-code-sync): Better dedicated UX/export/sync for Claude JSONL. This project is multi-vendor + synthesis-focused.
- **Replay/visualization** (e.g., farmanp/recall) and cost/classification tools (tokensave): Narrower or different focus. This repo's replay is conservative (select non-coding sessions, redaction preflight, bundles for *external* replayer/judge, ingest only as proposals).
- **Agent-knowledge / memory systems** (keshrath/agent-knowledge and similar): Git-synced KBs, search, distillation, secrets scrubbing. Overlap on git + scrubbing; this emphasizes deterministic signal extraction from transcripts, human promotion gates, calibration, and publishing *into* agent instructions rather than pure retrieval.
- **Karpathy LLM-wiki / persistent markdown KB patterns** and related self-updating wiki experiments: Very aligned philosophy (raw ? LLM/derived structured MD that compounds, interlinked, persistent across sessions). This implements a more opinionated, gated, provenance-heavy, transcript-specific version with explicit calibration, lint, markers (no free LLM rewrite), and agent publishing. It avoids full LLM-driven wiki maintenance in favor of keyword + human review initially.
- **Broader observability** (LangSmith, Helicone, Phoenix, etc.): Production traces, evals, cost. This is personal/local/offline, transcript-focused, and policy-oriented rather than runtime instrumentation.
- **Handoff skills/plugins** (common in Claude Code/OpenCode ecosystems): Many one-off `/handoff` skills exist. This systematizes discovery + mining at archive scale.

Differentiation/positioning: Best for someone who wants a *durable, reviewable, multi-machine personal engineering policy layer* extracted from their actual agent usage history, with strong process guardrails. Less emphasis on beautiful search UI or fully automatic synthesis compared to cass or LLM-wiki experiments. Very strong on Windows/WSL multi-tool coverage and safety contracts. The plugin marketplace start (pr-review-loop) hints at ecosystem ambitions.

Honest trade-offs: Opinionated (tied to specific workflows like PROJECT_DOC_TEMPLATE + PR discipline); archive freshness requires active maintenance; replay value is gated on external participation; less general than broad search tools until more users adopt.

Overall, this is a sophisticated, high-signal personal infrastructure project with excellent documentation, tests, and self-discipline. It treats agent sessions as a primary data source for long-term engineering improvement rather than ephemeral chat. The recent completion of the knowledge + replay foundation positions it for the "use it in anger with real replay results" phase.

51. user

```
<user_query>
can you write this feedback out as a pdf in the ~/projects directory?
</user_query>
```

52. tool_result

```
- /home/avidullu/projects/Agent/agent-sessions/./
- agent-sessions/
  - agent_sessions/
    - __init__.py
```

- archive.py
- archive_status.py
- baseline.py
- baseline_agent.py
- baseline_calibration.py
- baseline_eval.py
- baseline_handoffs.py
- baseline_ingest.py
- baseline_lint.py
- baseline_predictions.py
- baseline_promote.py
- baseline_publish.py
- baseline_redaction.py
- baseline_replay.py
- baseline_replay_ingest.py
- baseline_report.py
- baseline_settings.py
- baseline_types.py
- cli.py
- config.py
- models.py
- path_templates.py
- render.py
- sources/
 - __init__.py
 - claude.py
 - codex.py
 - deepseek.py
 - gemini.py
 - grok.py
 - registry.py
- utils.py
- AGENTS.md
- archive/
 - claude-windows/
[4444 files in subtree: 2222 *.md, 2222 *.pdf]
 - claude-wsl-ubuntu/
 - 20260606-8af642ca-0e91-47d3-99cf-be49dbaeda69-8af642ca-0e91-47d3-99cf-be49dbaeda69-58492bf.md
 - 20260606-8af642ca-0e91-47d3-99cf-be49dbaeda69-8af642ca-0e91-47d3-99cf-be49dbaeda69-58492bf.pdf
-
- 20260623-c287729c-2d4d-499f-b168-ce1fceb891f-c287729c-2d4d-499f-b168-ce1fceb891f-3712522.md
-
- 20260623-c287729c-2d4d-499f-b168-ce1fceb891f-c287729c-2d4d-499f-b168-ce1fceb891f-3712522.pdf
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a0dbe813153f9151d-2a6574464f34.md
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a0dbe813153f9151d-2a6574464f34.pdf
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a1158fbc4f69f0976-b54caf1054f9.md
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a1158fbc4f69f0976-b54caf1054f9.pdf
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a14d9ffbbcd53d0e5-5c6f0c4a4046.md
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a14d9ffbbcd53d0e5-5c6f0c4a4046.pdf
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a202351c55a0ceb23-53b353a8f617.md
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a202351c55a0ceb23-53b353a8f617.pdf
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a2425ecc6dc062cbe-1807344aab0d.md
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a2425ecc6dc062cbe-1807344aab0d.pdf
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a354d35dc4b2c2b42-de8ab4b5fee1.md
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a354d35dc4b2c2b42-de8ab4b5fee1.pdf
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a405eaebdbeec702b-4031eb7440c9.md
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a405eaebdbeec702b-4031eb7440c9.pdf
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a5598ac50dde6368-dce5b4bbcb6e.md
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a5598ac50dde6368-dce5b4bbcb6e.pdf
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a6164396ee6166ae2-c56867f57498.md
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a6164396ee6166ae2-c56867f57498.pdf
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a7d7084767b9679cf-ce8d079aa4a8.md
 - 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a7d7084767b9679cf-ce8d079aa4a8.pdf

- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a8beca64e2ef9d8aa-2fc3dfca27dd.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a8beca64e2ef9d8aa-2fc3dfca27dd.pdf
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a8c625abef0cba8dc-5df73fff18db.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-a8c625abef0cba8dc-5df73fff18db.pdf
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-adb6a4b6311cd61d7-e254a94f5ef2.md
- 20260629-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-agent-adb6a4b6311cd61d7-e254a94f5ef2.pdf
-

20260629-f7f9e275-96e2-4463-a5a3-65bfee712d00-f7f9e275-96e2-4463-a5a3-65bfee712d00-a13da13.md

-

20260629-f7f9e275-96e2-4463-a5a3-65bfee712d00-f7f9e275-96e2-4463-a5a3-65bfee712d00-a13da13.pdf

- 20260629-journal-journal-5ffea3738007.md
- 20260629-journal-journal-5ffea3738007.pdf
- 20260629-journal-journal-f8743d004df6.md
- 20260629-journal-journal-f8743d004df6.pdf
-

20260630-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-fafc315.md

-

20260630-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-4f364eb7-a600-4a2c-9b37-e5a313b6db9d-fafc315.pdf

- codex-windows/
 - [104 files in subtree: 52 *.md, 52 *.pdf]
- deepseek-vscode-windows/
 - [1504 files in subtree: 752 *.md, 752 *.pdf]
- deepseek-vscode-wsl-ubuntu/
 - [106 files in subtree: 53 *.md, 53 *.pdf]
- gemini-antigravity-windows/
 - 20260603-b83a9b6e-713a-4fa4-abe3-4bf658aab73e-transcript-913e67ffcb39.md
 - 20260603-b83a9b6e-713a-4fa4-abe3-4bf658aab73e-transcript-913e67ffcb39.pdf
 - 20260619-a34a0f82-73e4-484a-8c11-b99c579b48fd-transcript-d3d24f68d3ce.md
 - 20260619-a34a0f82-73e4-484a-8c11-b99c579b48fd-transcript-d3d24f68d3ce.pdf
 - 20260619-a34a0f82-73e4-484a-8c11-b99c579b48fd-transcript_full-fd33952b2a57.md
 - 20260619-a34a0f82-73e4-484a-8c11-b99c579b48fd-transcript_full-fd33952b2a57.pdf
- grok-wsl-ubuntu/
 - 20260529-019e7238-98ee-7bc1-a871-f9f00f5f8b3b-chat_history-1cfba6887d84.md
 - 20260529-019e7238-98ee-7bc1-a871-f9f00f5f8b3b-chat_history-1cfba6887d84.pdf
 - 20260529-019e7267-eb2a-74c1-ba09-d7e7d19fa29e-chat_history-a5b6eb725e5f.md
 - 20260529-019e7267-eb2a-74c1-ba09-d7e7d19fa29e-chat_history-a5b6eb725e5f.pdf
 - 20260530-019e77e5-9c65-7753-b108-772d1732a8ae-chat_history-27ce75dbe75d.md
 - 20260530-019e77e5-9c65-7753-b108-772d1732a8ae-chat_history-27ce75dbe75d.pdf
 - 20260602-019e892b-1982-75d2-875b-a9f82c413a34-chat_history-9301903fea0c.md
 - 20260602-019e892b-1982-75d2-875b-a9f82c413a34-chat_history-9301903fea0c.pdf
 - 20260602-019e892c-ae5f-7020-8261-ee9e4476991d-chat_history-1c0d052177f2.md
 - 20260602-019e892c-ae5f-7020-8261-ee9e4476991d-chat_history-1c0d052177f2.pdf
 - 20260602-019e8938-91ad-7db3-95dd-9377f69ffbfbfd-chat_history-b96a6e2dbd34.md
 - 20260602-019e8938-91ad-7db3-95dd-9377f69ffbfbfd-chat_history-b96a6e2dbd34.pdf
 - 20260602-019e893e-31bf-7b72-ae4a-b50fe5040381-chat_history-db67611dc1ed.md
 - 20260602-019e893e-31bf-7b72-ae4a-b50fe5040381-chat_history-db67611dc1ed.pdf
 - 20260602-019e894e-5540-7b70-a47a-beb676811ea9-chat_history-440ebe6f318a.md
 - 20260602-019e894e-5540-7b70-a47a-beb676811ea9-chat_history-440ebe6f318a.pdf
 - 20260602-019e8969-ba19-7bd0-b5b3-fde7c5445158-chat_history-9722b675cbb5.md
 - 20260602-019e8969-ba19-7bd0-b5b3-fde7c5445158-chat_history-9722b675cbb5.pdf
 - 20260608-019ea0e2-2c13-7d72-b154-54e4f93ea536-chat_history-37a6d8f6914c.md
 - 20260608-019ea0e2-2c13-7d72-b154-54e4f93ea536-chat_history-37a6d8f6914c.pdf
 - 20260608-019ea5ba-d3c8-7723-9207-ca6a4ca2fdd0-chat_history-6b0358ec94b7.md
 - 20260608-019ea5ba-d3c8-7723-9207-ca6a4ca2fdd0-chat_history-6b0358ec94b7.pdf
 - 20260608-019ea5bb-6066-7c91-a074-d721dc1b5b9d-chat_history-8d34dd00289e.md
 - 20260608-019ea5bb-6066-7c91-a074-d721dc1b5b9d-chat_history-8d34dd00289e.pdf
 - 20260614-019ec52b-0ea1-7af2-b1be-6d972bd01936-chat_history-71946cdd51ac.md
 - 20260614-019ec52b-0ea1-7af2-b1be-6d972bd01936-chat_history-71946cdd51ac.pdf
- index.jsonl
- INDEX.md
- baseline/
 - agents/
 - [6 files in subtree: 6 *.md]
 - calibration/
 - [3 files in subtree: 2 *.toml, 1 *.md]

- candidates/
 - [3 files in subtree: 2 *.md, 1 *.json]
- evidence/
 - README.md
- global/
 - [4 files in subtree: 4 *.md]
- handoffs/
 - audit.md
 - index.jsonl
- metacognition/
 - prediction-ledger.jsonl
 - README.md
- projects/
 - [6 files in subtree: 6 *.md]
- proposals/
 - [7 files in subtree: 6 *.json, 1 *.md]
- README.md
- replay/
 - manifest.jsonl
- SCHEMA.md
- config/
 - baseline.toml
 - default_sources.toml
- docs/
 - archives/
 - TECH_DEBT_PLAN.md
 - AUTOMATION.md
 - BASELINE_KNOWLEDGE_REPLAY_PLAN.md
 - BASELINE_LOOP_CLOSURE.md
 - BASELINE_PLANNING.md
 - BASELINE_WATCHLIST_TOMBSTONES.md
 - CALIBRATION_EFFICACY.md
 - COMPOSE_STACK.md
 - DISCOVERY.md
 - ENGINEERING_BASELINE.md
 - FIRST_USER_SETUP_TRACKER.md
 - MULTI_MACHINE.md
 - NEW_MACHINE_SETUP.md
 - PROJECT_DOC_TEMPLATE.md
 - README.md
 - ROADMAP.md
 - TEST_PLAN.md
 - WORK_AUDIT_2026-07-08.md
- plugins/
 - pr-review-loop/
 - README.md
 - skills/
 - [1 file in subtree: 1 *.md]
- pyproject.toml
- README.md
- scripts/
 - daily-export.ps1
 - daily-export.sh
- SESSION_HANDOFF.md
- sources.example.toml
- tests/
 - __init__.py
 - conftest.py
 - test_archive.py
 - test_archive_status.py
 - test_baseline.py
 - test_baseline_agent.py
 - test_baseline_calibration.py
 - test_baseline_eval.py
 - test_baseline_handoffs.py

- test_baseline_ingest.py
- test_baseline_lint.py
- test_baseline_publish.py
- test_baseline_redaction.py
- test_baseline_replay.py
- test_baseline_replay_ingest.py
- test_claude_extractor.py
- test_cli.py
- test_codex_extractor.py
- test_config.py
- test_deepseek_extractor.py
- test_gemini_extractor.py
- test_grok_extractor.py
- test_models.py
- test_module_imports.py
- test_path_templates.py
- test_registry.py
- test_render.py
- test_tool_wrapper.py
- test_utils.py
- tools/
- agent_archive.py

53. tool_result

```
exit: 0
/bin/bash: line 1: python: command not found
reportlab not installed
```

54. tool_result

```
exit: 0
total 28
drwxr-xr-x  7 avidullu avidullu 4096 Jul  7 03:41 .
drwxr-x--- 27 avidullu avidullu 4096 Jul  7 09:58 ..
drwxr-xr-x  3 avidullu avidullu 4096 Jul  8 14:45 Agent
drwxr-xr-x  3 avidullu avidullu 4096 Jul  7 06:06 Telegram
drwxr-xr-x  4 avidullu avidullu 4096 Jun 29 08:20 Verifiers
drwxr-xr-x 12 avidullu avidullu 4096 Jul  7 03:41 khelsutra-guru
drwxr-xr-x  5 avidullu avidullu 4096 Jun 29 09:51 muneem-ai
```

55. tool_result

```
exit: 0
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'reportlab'
/bin/bash: line 1: pip3: command not found
checking python3...
/usr/bin/python3
Python 3.12.3
```

56. tool_result

```
exit: 0
/usr/bin/python3: No module named pip
ensurepip is disabled in Debian/Ubuntu for the system python.
```

Python modules for the system python are usually handled by dpkg and apt-get.

```
apt install python3-<module name>
no user pip
```

57. tool_result


```
exit: 0
checking pdf tools
```

58. tool_result

```
1?"""Markdown and PDF rendering.""
```

```
from __future__ import annotations
```

```
import datetime as dt
import re
import textwrap
from pathlib import Path
```

```
10?from .models import ExtractedSession, Source
from .utils import now_utc
```

```
def markdown_for_session(
    source: Source,
    path: Path,
    session: ExtractedSession,
    digest: str,
    imported_at: str | None = None,
20?) -> str:
    title_bits = [source.name, str(session.metadata.get("session_id") or path.stem)]
    title = " / ".join(x for x in title_bits if x)
    lines = [
        f"# {title}",
        "",
        "## Metadata",
        "",
        f"- Source: `{source.name}`",
        f"- Kind: `{source.kind}`",
30?     f"- Source file: `{path}`",
        f"- SHA-256: `{digest}`",
        f"- Source modified: `{modified_timestamp(path)}`",
        f"- Imported at: `{imported_at or now_utc()}`",
    ]
    for key in sorted(session.metadata):
        value = session.metadata[key]
        if value in (None, ""):
            continue
        lines.append(f"- {key}: `{value}`")
40?     lines.extend(["", "## Transcript", ""])
    if not session.messages:
        lines.append("_No transcript messages were extracted from this file._")
        return "\n".join(lines) + "\n"
    for idx, msg in enumerate(session.messages, 1):
        heading = f"### {idx}. {msg.role or 'message'}"
        if msg.timestamp:
            heading += f" ({msg.timestamp})"
        lines.extend([heading, "", msg.text.rstrip(), ""])
    return "\n".join(lines).rstrip() + "\n"
50?
```

```
def modified_timestamp(path: Path) -> str:
    return dt.datetime.fromtimestamp(path.stat().st_mtime,
dt.timezone.utc).isoformat(timespec="seconds")
```

```
def write_pdf(markdown: str, out_path: Path) -> bool:
    try:
        from reportlab.lib.pagesizes import letter
        from reportlab.pdfgen import canvas
```

```

60?     except Exception:
        return False

    ansi = re.compile(r"\x1b\[ [0-?]*[ -/]*[@-~]")
    pdf = canvas.Canvas(str(out_path), pagesize=letter)
    width, height = letter
    margin = 42
    y = height - margin
    line_height = 11
    max_chars = 96
70?
    def emit(line: str, font: str = "Courier", size: int = 8) -> None:
        nonlocal y
        if y < margin:
            pdf.showPage()
            y = height - margin
        pdf.setFont(font, size)
        safe = ansi.sub("", line).encode("latin-1", "replace").decode("latin-1")
        pdf.drawString(margin, y, safe)
        y -= line_height
80?
    for raw in markdown.splitlines():
        line = raw.rstrip()
        if line.startswith("# "):
            y -= 4
            emit(line[2:], "Helvetica-Bold", 14)
            y -= 4
            continue
        if line.startswith("## "):
            y -= 3
90?            emit(line[3:], "Helvetica-Bold", 11)
            y -= 2
            continue
        if line.startswith("### "):
            y -= 2
            emit(line[4:], "Helvetica-Bold", 9)
            continue
        if not line:
            y -= line_height
            continue
100?        for wrapped in textwrap.wrap(line, width=max_chars, replace_whitespace=False) or
[""]:
            emit(wrapped)
    pdf.save()
    return True

```

59. tool_result

```

exit: 0
no venv in repo
no global pip list

```

60. tool_result

```

exit: 0
python3-reportlab - ReportLab library to create PDF documents using Python3
/usr/bin/python3
problem_report.py

```

61. tool_result

```

exit: 1
sudo: a terminal is required to read the password; either use the -S option to read from
standard input or configure an askpass helper

```

sudo: a password is required

62. tool_result

```
exit: 0
trying to find pip
done
```

63. tool_result

```
exit: 0
File "<string>", line 1
import pkgutil; print([m for m in pkgutil.iter_modules() if \"report\" in m.name.lower()])
                        ^
SyntaxError: unexpected character after line continuation character
```

64. tool_result

```
exit: 0
---
ii  groff-base                                1.23.0-3build2                                amd64
GNU troff text-formatting system (base system components)
python-reportlab-doc - Documentation for the ReportLab Python library (PDF format)
python3-django-wkhtmltopdf - Django module with views for HTML to PDF conversions (Python 3)
python3-fpdf - Simple & fast PDF generation for Python
```

65. tool_result

```
exit: 0
```

66. tool_result

```
exit: 0
fpdf not: No module named 'fpdf'
fpdf2 style: No module named 'fpdf'
```

67. tool_result

```
exit: 0
no gs
no gs bin
File "<string>", line 1
import subprocess, os; print(os.popen(\"apt list --installed 2>/dev/null | grep -E
pdf|groff\").read())
                        ^
SyntaxError: unexpected character after line continuation character
```

68. tool_result

```
exit: 0
groff-base/noble,now 1.23.0-3build2 amd64 [installed,automatic]
python3-problem-report/noble-updates,noble-security,now 2.28.1-0ubuntu3.8 all
[installed,automatic]
---
```

69. tool_result

```
exit: 0
GNU groff version 1.23.0
Copyright (C) 2022 Free Software Foundation, Inc.
GNU groff comes with ABSOLUTELY NO WARRANTY.
You may redistribute copies of groff and its subprograms
under the terms of the GNU General Public License.
```

For more information about these matters, see the file
named COPYING.

called subprograms:

```
groff: fatal error: cannot load 'DESC' description file for device 'pdf'
---
groff: fatal error: cannot load 'DESC' description file for device 'pdf'
groff -Tpdf not functional or no pdf device
```

70. tool_result

```
exit: 0
python3-reportlab:
  Installed: (none)
  Candidate: 4.1.0-4
  Version table:
    4.1.0-4 500
    500 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages
python3-fpdf:
  Installed: (none)
  Candidate: 2.7.8-2
  Version table:
    2.7.8-2 500
    500 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages
```

71. tool_result

```
exit: 0
```

72. tool_result

The file /tmp/generate_agent_sessions_overview.py has been created.

73. tool_result

```
exit: 0
PDF written to: /home/avidullu/projects/agent-sessions-feedback.pdf
Size: 10598 bytes
```

74. tool_result

```
exit: 0
-rw-r--r-- 1 avidullu avidullu 10598 Jul  8 14:52 /home/avidullu/projects/agent-sessions-
feedback.pdf
```

75. tool_result

```
exit: 0
-rw-r--r-- 1 avidullu avidullu 11K Jul  8 14:52 /home/avidullu/projects/agent-sessions-
feedback.pdf
/home/avidullu/projects/agent-sessions-feedback.pdf: PDF document, version 1.4, 3 page(s)
---
PDF info (if tools available):
pdfinfo not present (expected)
```

76. tool_result

```
exit: 0
temp artifacts cleaned
-rw-r--r-- 1 avidullu avidullu 11K Jul  8 14:52 /home/avidullu/projects/agent-sessions-
feedback.pdf
```

77. assistant

****Done.****

I generated a 3-page PDF containing the full repository overview (overarching idea, owner's vision, engineering details, internal feedbacks/current state from the audits, and comparisons to similar tools like cass, observational-memory, etc.).

****Location:**** `/home/avidullu/projects/agent-sessions-feedback.pdf`

- Size: 11 KB
- Format: PDF 1.4 (3 pages)
- Cleanly formatted with title, sections, bullets, and a comparison table

Temporary build artifacts (isolated venv + generator script) were created in `/tmp` and removed after successful generation.

The PDF is ready for you at the top of `~/projects`. Let me know if you'd like any adjustments to the content, filename, styling, or placement.